다듬(DaDeum) 완전정복

0장. 이 책을 읽는 법과 전체 지도
  0.1 이 책의 약속
  0.2 다듬은 무엇인가
  0.3 전체 지도
  0.4 이 책의 구성
1장. 코드 기초: 이 프로젝트를 읽는 데 필요한 것만
  1.1 프로그램은 무엇인가
  1.2 변수: 값에 이름 붙이기
  1.3 자료형: 문자열, 숫자, 참/거짓
  1.4 조건문: 만약 ~라면
  1.5 반복문: 여러 번 하기
  1.6 목록·집합·사전
  1.7 함수: 이름 붙인 작업 묶음
  1.8 클래스와 dataclass: 묶음 자료
  1.9 모듈과 import
  1.10 정규식: 글자 모양 찾기
  1.11 정렬과 람다
  1.12 예외 처리
  1.13 이 장에서 못 다룬 것 (나오면 그때 설명합니다)
  1.14 확인 문제
2장. 한글이 컴퓨터 안에서 어떻게 생겼나
  2.1 컴퓨터는 글자를 번호로 저장한다
  2.2 11,172는 어디서 나왔나
  2.3 받침 번호를 꺼내는 한 줄
  2.4 실제 코드: `app/hangul.py`

2.5 이걸로 조사를 어떻게 고르나
2.6 율/률도 같은 계산
2.7 숫자는 왜 판단하지 않는가
2.8 어절 경계 찾기
2.9 확인 문제
3장. 형태소 분석: 문장을 조각내고 이름표 붙이기
3.1 왜 필요한가
3.2 용어
3.3 실제 분석 결과
3.4 품사 태그표 (다듬 코드에 나오는 것들)
3.5 같은 글자, 다른 분석: 다듬이 품사를 보는 이유
3.6 Kiwi가 틀린 문장을 어떻게 읽는가
3.7 요청 한 번에 분석은 한 번만: 캐시
3.8 Kiwi 준비 운동
3.9 문장 나누기와 토큰 자르기 도우미 (`app/morphology.py`)
3.10 확인 문제
4장. 서버와 API: 요청 하나가 지나가는 길
4.1 서버와 클라이언트
4.2 API란
4.3 주문서와 음식의 모양: `schemas.py`
4.4 서버 뼈대: `app/main.py`
4.5 레이트리밋: `ratelimit.py`
4.6 `/api/check` 한 줄씩
4.7 긴 글: `analyze_long`
4.8 `/api/health` : 지금 어느 층이 켜져 있나
4.9 직접 따라 하기
4.10 확인 문제
5장. 엔진 뼈대: 후보를 모으고 정리하는 구조
5.1 한 문장으로
5.2 왜 “하나의 흐름”인가: 만들어진 이유
5.3 자료: `RawIssue`
5.4 검사기 등록: `GENERATORS`
5.5 `analyze()` 한 줄씩
5.6 병합: `merge_and_dedupe` (엔진의 심장)

5.7 출처 우선순위표: `sources.py`
5.8 마지막 청소: `_drop_noise_adjacent_to_fixed_typos`
5.9 예문 전체 추적
5.10 확인 문제
6장. 규칙 검사기: 문법을 코드로 옮긴 것들
6.0 규칙 검사기의 공통 설계
6.1 jamo.py: 흩어진 자모 합치기
6.2 dictionary.py + dict_rules.json: 사전 규칙 474개
6.3 yul_ryul.py: 율/률
6.4 josa.py: 조사 (365줄, 가장 복잡한 규칙 검사기)
6.5 dependent_noun.py: 의존명사 띄어쓰기
6.6 particle_spacing.py: 조사는 앞말에 붙인다
6.7 quotation.py: 간접 인용 `-라고` 겹침
6.8 question_mark.py: 물음표
6.9 honorific.py: 사물 존대
6.10 animacy_particle.py: 에게 / 에
6.11 negative_polarity.py: 부정 호응
6.12 tense.py: 시제 호응
6.13 redundant_causative.py: 불필요한 `-시키다`
6.14 subject_predicate.py: 주술 호응
6.15 morph_rules.py: 형태소 배열 보기
6.16 consistency.py: 문체 일관성 (유일하게 글 전체를 보는 검사기)
6.17 이 장에서 가져갈 것
6.18 확인 문제
7장. 머신러닝 기초와 혼동쌍 모델
7.1 규칙 방식과 학습 방식
7.2 문제를 “둘 중 하나 고르기”로 바꾸기
7.3 학습 데이터: 정답이 공짜로 생기는 방법
7.4 특징: 글자 조각 세기 (TF-IDF)
7.5 로지스틱 회귀: 조각마다 투표하기
7.6 학습 설정 (`ml/train.py`)
7.7 얼마나 믿을까: 시험과 문턱
7.8 배포 형식 `.ddm`
7.9 추론: 규칙이 먼저, 모델은 나중에 (`checkers/confusion.py`)

7.10 여섯 모델의 성적표 (`meta.json` 실제 값)
7.11 이 방식의 한계
7.12 확인 문제
8장. 단어 경계 모델 (gyeol-boundary)
8.1 풀려는 문제
8.2 전체 구조
8.3 단계 ①: 후보 찾기 (`app/boundary.py`)
8.4 단계 ②-a: dense(아주 길게 붙인 덩어리) 판별
8.5 왜 dense에서는 모델을 안 쓰는가
8.6 dense에서 버리는 자리: `_keep_dense`
8.7 단계 ②-b: 모델 판정 (dense가 아닐 때)
8.8 단계 ③: 가드
8.9 학습 데이터 만들기 (`ml/boundary_data.py`)
8.10 학습과 문턱 고르기 (`ml/train_boundary.py`)
8.11 실제로 돌려 본 결과
8.12 한계 (코드에 적힌 실측 기록)
8.13 확인 문제
9장. 오타 · 어절 사전 · 언어모델 · 이상탐지
9.1 어절 사전 (`app/lexicon.py`): 181만 개의 "아는 말" 목록
9.2 키보드 거리 (`app/keystrokes.py`): "손가락이 한 번 헛디딘 것"
9.3 오타 검사 (`checkers/typo.py`): 여섯 개의 문
9.4 이 오타를 "고칠" 것인가 "물어볼" 것인가
9.5 언어모델 gyeol-lm (`app/ngram_lm.py`): "어느 쪽이 더 한국어다운가"
9.6 이상탐지 (`app/anomaly.py`): "이 어절은 낯설다"
9.7 시도했다가 접은 것
9.8 확인 문제
10장. 띄어쓰기: 가장 큰 검사 영역
10.1 문제의 모양
10.2 3단계 판정: 필수 / 권장 / 허용 (`spacing_tiers.py`)
10.3 전체 구조: 네 갈래가 한 목록으로
10.4 ① 삽입: BiGRU 태거
10.5 ② 삭제: Kiwi의 `space()`
10.6 ③ 규칙 (`spacing.py`)
10.7 ④ 추가 규칙 (`spacing_extra.py`)

10.8 실행 결과 정리 (제가 직접 돌림)
10.9 이 장에서 가져갈 것
10.10 확인 문제
11장. 평가: 좋은지 어떻게 아는가
11.1 먼저 용어
11.2 평가 데이터 네 종류
11.3 실제 결과 (제가 돌린 값)
11.4 품질 파이프라인: `python -m eval.pipeline`
11.5 이 장의 교훈 정리
11.6 직접 돌려 보기
11.7 확인 문제

0장. 이 책을 읽는 법과 전체 지도

0.1 이 책의 약속

이 책은 **코드를 한 줄도 모르는 사람**이 끝까지 읽으면 다듬(DaDeum)의 코드를 전부 설명할 수 있게 되는 것을 목표로 합니다. 그래서 모든 장이 같은 순서로 진행됩니다.

순서	묻는 것	예
1	**무엇을** 하는가 (목적)	"받침을 보고 을/를을 고른다"
2	**왜** 필요한가 (이유)	"이 규칙이 없으면 `학교을` 을 못 잡는다"
3	**어떻게** 하는가 (로직)	실제 코드와 한 줄씩 풀이
4	**어떻게 확인했는가** (근거)	실제로 돌려 본 결과, 측정 수치
5	**무엇이 문제였고 어떻게 풀었는가** (해결)	실패 사례와 고친 방법

코드 인용은 모두 실제 저장소(`zmflm0110/dadeum`, master 6af0ea5)에서 가져왔습니다. 설명에 나오는 실행 결과는 제가 그 코드를 직접 돌려서 얻은 값입니다. 돌려 보지 않은 것은 "읽은 바로는"이라고 적습니다.

> 이 책에서 가장 중요한 습관 하나: **"그렇다더라"로 넘어가지 않고, 입력을 넣고 출력을 보는 것.** 다듬 프로젝트 자체가 그 습관으로 만들어졌습니다. (13장에서 그 이야기를 합니다.)

0.2 다듬은 무엇인가

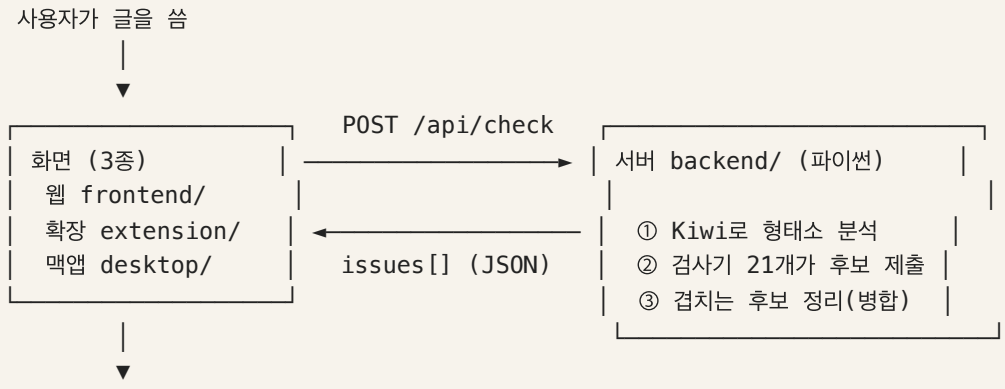
한국어 글을 넣으면 **맞춤법·띄어쓰기·조사·문법 오류에 밑줄을 긋고, 고친 말을 제안하는 프로그램**입니다. 웹 페이지, 크롬 확장, 맥 앱 세 가지 모습이 있고, 셋 다 같은 "엔진"을 씁니다. 전부 내 컴퓨터 안에서 돌고 외부 AI를 부르지 않습니다.

한 문장을 넣으면 이렇게 됩니다.

```
입력:  학교을 가는길에 친구를 만낫는데 그렇게 하면 안되요.

출력:  2-3   '을'     → '를'      (조사, 확실)
       25-26 '안'     → '안 '     (띄어쓰기, 확실)
       26-28 '되요'   → '돼요'    (맞춤법, 확실)
       13-17 '만낫는데' → (없음)   (이상한 말, 애매)
```

0.3 전체 지도



폴더	한 줄 설명	이 책의 장
`backend/app/`	교정 엔진 본체	4-10장
`backend/ml/`	모델을 학습시키는 코드	7-9장
`backend/eval/`	엔진이 좋은지 재는 도구	11장
`backend/tests/`	자동 테스트 997개	11장
`core/`	화면 세 개가 공유하는 상태기계	12장
`frontend/` `extension/` `desktop/`	화면	12장

0.4 이 책의 구성

장	내용
1	코드 기초: 이 프로젝트를 읽는 데 필요한 파이썬 문법만
2	한글이 컴퓨터 안에서 어떻게 생겼나 (받침 계산의 원리)
3	형태소 분석: 문장을 조각내고 이름표를 붙이기
4	서버와 API: 요청 하나가 지나가는 길
5	엔진 뼈대: 후보를 모으고 정리하는 구조
6	규칙 검사기: 조사·자모·사전·의존명사 등
7	머신러닝 기초와 혼동쌍 모델
8	단어 경계 모델
9	오타·사전·언어모델·이상탐지
10	띄어쓰기
11	평가: 좋은지 어떻게 아는가
12	화면과 공유 코어
13	개발 과정과 교훈
14	용어집, 직접 돌려 보기, 연습문제

앞 장을 건너뛰면 뒤 장의 용어를 못 알아볼 수 있습니다. 처음엔 순서대로 읽으세요.

1장. 코드 기초: 이 프로젝트를 읽는 데 필요한 것만

다듬의 엔진은 **파이썬**으로 쓰여 있습니다. 파이썬 전체를 배울 필요는 없고, 이 프로젝트에 실제로 나오는 문법만 알면 됩니다. 각 항목마다 "다듬 코드의 어디에 나오는지"를 같이 적었습니다.

1.1 프로그램은 무엇인가

프로그램은 **입력을 받아 정해진 순서대로 처리해서 출력을 내는 글**입니다. 요리 레시피와 같습니다.

```
입력: "학교을 갔다"  →  [처리: 받침을 보고 을/를 고르기]  →  출력: "학교를 갔다"
```

컴퓨터는 사람처럼 "대충" 이해하지 못합니다. 모든 단계를 정확히 적어 줘야 합니다. 다듬 코드가 길어 보이는 이유는 "이 경우는 이렇게, 저 경우는 저렇게"를 전부 적었기 때문입니다.

1.2 변수: 값에 이름 붙이기

```
text = "학교을 갔다"
count = 3
```

`text`라는 이름표를 값에 붙인 것입니다. `=`는 "같다"가 아니라 **"오른쪽 값을 왼쪽 이름에 넣어라"**입니다.

다듬 예: `app/checkers/confusion.py`에는 `_DOE_FORCES_DOE = set("고게기니나냐네는면며자거든어었겠시")`가 있습니다. "되 뒤에 이 글자들이 오면 무조건 '되'"라는 목록에 이름을 붙인 것입니다.

1.3 자료형: 문자열, 숫자, 참/거짓

종류	예	다듬에서
문자열 `str`	`"학교"`	입력 글, 제안, 설명 메시지
정수 `int`	`26`	글자 위치(start, end)
실수 `float`	`0.97`	확률, 문턱
참/거짓 `bool`	`True`	조건 검사 결과
없음 `None`	`None`	“답이 없다” (예: 받침을 알 수 없는 글자)

문자열의 번호(인덱스)는 0부터 셉니다.

```
text = "학교을 갔다"
text[0]      # '학'
text[2]      # '을'
text[0:2]    # '학교'   (0번부터 2번 "직전"까지)
```

다듬이 말하는 `start=2, end=3`은 “2번 글자부터 3번 직전까지”, 즉 `text[2:3]` = `'을'` 입니다. 이 “끝은 포함하지 않는다” 규칙이 프로젝트 전체에서 일관되게 쓰입니다.

1.4 조건문: 만약 ~라면

```
if idx == NO_BATCHIM_INDEX:
    answer = "를"
else:
    answer = "을"
```

`==`는 “같은가?”를 묻는 기호입니다(`=`와 다릅니다). 다듬 코드 거의 모든 줄이 이런 조건의 연속입니다.

`and`(둘 다), `or`(하나라도), `not`(반대)로 조건을 묶습니다.

```
if actual in (no_batchim, has_batchim) and actual != expected:
    ...   # 조사가 둘 중 하나이면서, 기대한 것과 다르면
```

1.5 반복문: 여러 번 하기

```
for ch in "학교을":
    print(ch)
# 학
# 교
# 을
```

`for 이름 in 목록:`은 목록의 원소를 하나씩 꺼내 아래 들여쓴 줄을 실행합니다. **들여쓰기(맨 앞 공백)가 문법입니다.** 파이썬은 괄호 대신 들여쓰기로 "여기까지가 for의 안쪽"을 표시합니다.

`continue`는 "이번 차례는 건너뛰고 다음으로", `break`는 "반복을 완전히 끝내기"입니다. 다음의 검사기들은 `if (오탐 위험 조건): continue`로 위험한 자리를 건너뛰는 모양이 아주 많습니다.

1.6 목록·집합·사전

```
issues = [ ... ]            # 목록(list): 순서 있음, 같은 값 여러 개 가능
seen = {3, 7, 9}            # 집합(set): 순서 없음, 중복 없음, "있나?" 확인이 빠름
rule = {"pattern": "결재", "replacement": "결제"}   # 사전(dict): 이름표 → 값
```

	만들기	꺼내기	다듬 예
list	`[1, 2]`	`a[0]`	검사기가 낸 `issues`
set	`{1, 2}`	`3 in a`	`seen_spans` : 이미 본 위치
dict	`{"k": v}`	`d["k"]`	`dict_rules.json`의 규칙 한 개

리스트 컴프리헨션은 "목록을 걸러서 새 목록 만들기"를 한 줄로 쓴 것입니다.

```
actionable = [i for i in issues if i.confidence != "low"]
# 풀어 쓰면:
actionable = []
for i in issues:
    if i.confidence != "low":
        actionable.append(i)
```

1.7 함수: 이름 붙인 작업 묶음

```python
def final_index(ch):
    """받침 번호를 돌려준다."""
    if not is_hangul_syllable(ch):
        return None
    return (ord(ch) - 0xAC00) % 28
```

- `def 이름(입력):`으로 만듭니다. `return`은 결과를 돌려주고 함수를 끝냅니다.
- 따옴표 세 개 `"""..."""`는 **설명문(독스트링)**입니다. 실행에 영향이 없고 읽는 사람을 위한 것입니다. 다듬 코드는 이 설명이 매우 깁니다. 왜 이렇게 했는지, 어떤 실패가 있었는지가 거기 적혀 있습니다.
- `#` 뒤는 한 줄 주석입니다.

함수를 쓰는 것을 **호출**이라고 합니다: `final_index("교")` → `0`.

타입 힌트

```python
def check_josa(text: str, kiwi: Kiwi) -> list[RawIssue]:
```

`: str`은 "이 입력은 문자열", `-> list[RawIssue]`는 "결과는 RawIssue의 목록"이라는 **표시**입니다. 실행에 영향은 없고 읽기 쉽게 하려는 것입니다. `str | None`은 "문자열이거나 None"입니다.

1.8 클래스와 dataclass: 묶음 자료

관련된 값 여러 개를 하나로 묶고 싶을 때 씁니다. 다듬의 핵심 자료인 `RawIssue`가 이렇게 생겼습니다(`app/checkers/base.py`).

```python
from dataclasses import dataclass

@dataclass
class RawIssue:
    start: int
    end: int
    category: str
    source: str
    original: str
    suggestions: list[str]
    message: str
    confidence: str
```

"지적 한 건"은 위치(start, end), 종류(category), 출처(source), 원문(original), 고칠 말(suggestions), 설명(message), 확신도(confidence)로 이루어집니다.

```
r = RawIssue(start=2, end=3, category="josa", source="josa_rule",
             original="을", suggestions=["를"], message="...", confidence="h
         igh")
r.original      # '을'  ← 점(.)으로 안의 값을 꺼냄
r.suggestions[0]  # '를'
```

`@dataclass`는 "이 클래스의 생성자와 출력 모양은 자동으로 만들어 줘"라는 표시입니다. `@dataclass(frozen=True)`는 "한 번 만들면 값을 못 바꾼다"입니다(`Generator`, `Source`가 이렇습니다).

1.9 모듈과 import

파일 하나가 모듈입니다. 다른 파일의 것을 가져다 쓰려면 `import`합니다.

```
from .base import RawIssue          # 같은 폴더의 base.py에서 RawIssue를
from ..hangul import final_index    # 한 단계 위 폴더의 hangul.py에서
from kiwipiepy import Kiwi          # 설치된 외부 라이브러리에서
```

점(`.`)의 개수는 폴더 거리입니다. 한 개는 같은 폴더, 두 개는 위 폴더입니다.

1.10 정규식: 글자 모양 찾기

정규식(regular expression)은 "이런 모양의 글자를 찾아라"를 적는 작은 언어입니다. 다듬의 사전 규칙 474개가 전부 정규식입니다.

기호	뜻	예
`abc`	그 글자 그대로	`결재`
`A|B`	A 또는 B	`결재|결제`
`[가-힣]`	한글 한 글자	
`.`	아무 글자	
`+`	앞의 것 1번 이상	`[가-힣]+`
`^` `$`	문자열의 처음, 끝	
`(?<=X)`	"바로 앞에 X가 있을 때만" (앞을 확인만 하고 소비하지 않음)	
`(?=X)`	"바로 뒤에 X가 있을 때만"	
`(?!X)`	"바로 뒤에 X가 없을 때만"	

다듬의 실제 규칙(`data/dict_rules.json`)입니다.

```
(?<=카드로 )결재        → 결제
```

"`카드로` 바로 뒤에 오는 `결재`"만 찾아서 `결제`로 바꿉니다. 앞의 `카드로`는 확인만 하고 바꾸는 범위에는 들어가지 않습니다. 그래서 `서류를 결재`는 건드리지 않습니다. **문맥을 정규식 안에 넣어서** 같은 철자의 다른 쓰임을 구별하는 것입니다.

파이썬에서는 이렇게 씁니다.

```python
import re
pat = re.compile(r"(?<=카드로 )결재")
for m in pat.finditer("카드로 결재했다"):
    print(m.start(), m.end(), m.group(0))   # 4 6 결재
```

1.11 정렬과 람다

```python
ordered = sorted(issues, key=lambda i: (i.start, -(i.end - i.start)))
```

`sorted`는 목록을 정렬합니다. `key=`는 "무엇을 기준으로 줄 세울지"입니다. `lambda i: ...`는 이름 없는 한 줄짜리 함수입니다. 위 기준은 `(시작 위치, 길이의 음수)`이므로 **시작이 빠른 것이 먼저, 시작이 같으면 긴 것이 먼저** 옵니다. 5장의 병합이 이 정렬 위에서 돌아갑

니다.

1.12 예외 처리

```
try:
    return kiwi.analyze(word)[0][1]
except Exception:
    return 0.0
```

`try` 안에서 오류가 나도 프로그램이 죽지 않고 `except` 안쪽을 실행합니다. 다음은 "분석 한 번이 실패해도 서버 전체가 죽으면 안 된다"는 원칙으로 곳곳에 씁니다.

1.13 이 장에서 못 다룬 것 (나오면 그때 설명합니다)

- `yield` (값을 하나씩 내보내는 함수): 8장 경계 후보
- 데코레이터 `@router.post(...)` : 4장 서버
- `async` / `await` (JavaScript): 12장 화면

1.14 확인 문제

1. `text = "안녕하세요"` 일 때 `text[1:3]` 은? (답: `녕하`)
2. `start=4, end=6` 인 지적이 가리키는 글자를 `"카드로 결재했다"` 에서 꺼내면? (답: `결재`)
3. `[i for i in [1,2,3,4] if i > 2]` 의 결과는? (답: `[3, 4]`)

2장. 한글이 컴퓨터 안에서 어떻게 생겼나

다듬의 조사 검사(`을/를`, `이/가`…)는 한 가지 계산에 기대고 있습니다. **"이 글자에 받침이 있는가?"를 산수로 알아내는 것**입니다. 이 장은 그 산수를 처음부터 설명합니다.

2.1 컴퓨터는 글자를 번호로 저장한다

컴퓨터 안에는 글자가 없고 **번호**만 있습니다. 글자마다 번호를 하나씩 정해 둔 표를 **유니코드**라고 합니다. 파이썬에서 `ord("가")`는 글자의 번호를, `chr(번호)`는 번호의 글자를 돌려줍니다.

```
ord("가")   # 44032  (16진수로 0xAC00)
ord("힣")   # 55203  (16진수로 0xD7A3)
chr(44032)  # '가'
```

완성된 한글 글자(가, 각, 간, …, 힣)는 **44032번부터 55203번까지 11,172개**가 빈틈없이 줄지어 있습니다.

2.2 11,172는 어디서 나왔나

한글 한 글자는 세 부분으로 이루어집니다.

부분	이름	가짓수	예 (학)
첫소리	초성	19	ㅎ
가운뎃소리	중성	21	ㅏ
끝소리(받침)	종성	28 (받침 없음 포함)	ㄱ

19 × 21 × 28 = **11,172**. 모든 조합이 번호표에 하나씩 들어 있는 것입니다.

번호는 이 순서로 매겨져 있습니다.

```
(글자 번호 − 44032) = (초성 번호 × 21 + 중성 번호) × 28 + 종성 번호
```

사전에서 "초성 → 중성 → 종성" 순으로 정렬한 것과 같습니다. 같은 초성, 같은 중성인 글자들이 종성 순서대로 28개씩 붙어 있습니다.

```
가(종성 0, 받침 없음) 각(1: ㄱ) 갂(2: ㄲ) 갃(3: ㄳ) 간(4: ㄴ) ... 갛(27: ㅎ)
```

2.3 받침 번호를 꺼내는 한 줄

위 식에서 **28로 나눈 나머지가 종성 번호**입니다.

```
받침번호 = (ord(글자) - 0xAC00) % 28
```

`%`는 나머지입니다. 실제로 계산해 보았습니다.

글자	번호	번호 − 44032	% 28	뜻
가	44032	0	**0**	받침 없음
각	44033	1	**1**	ㄱ
힣	55203	11171	**27**	ㅎ
교	44368	336	**0**	받침 없음
학	54617	…	**1**	ㄱ
을	51012	…	**8**	ㄹ

종성 번호표는 이렇습니다(0은 받침 없음).

```
0:없음 1:ㄱ 2:ㄲ 3:ㄳ 4:ㄴ 5:ㄵ 6:ㄶ 7:ㄷ 8:ㄹ 9:ㄺ 10:ㄻ 11:ㄼ 12:ㄽ 13:ㄾ 14:ㄿ
15:ㅀ 16:ㅁ 17:ㅂ 18:ㅄ 19:ㅅ 20:ㅆ 21:ㅇ 22:ㅈ 23:ㅊ 24:ㅋ 25:ㅌ 26:ㅍ 27:ㅎ
```

2.4 실제 코드: `app/hangul.py`

이 파일 전체가 38줄입니다. 핵심만 옮깁니다.

```
HANGUL_BASE = 0xAC00      # '가'
HANGUL_LAST = 0xD7A3      # '힣'
JONGSEONG_COUNT = 28      # 받침 가짓수 (0 = 받침 없음)

NO_BATCHIM_INDEX = 0
NIEUN_INDEX = 4           # ㄴ – 을/를 예외
RIEUL_INDEX = 8           # ㄹ – 로/으로 예외

def is_hangul_syllable(ch):
    return len(ch) == 1 and HANGUL_BASE <= ord(ch) <= HANGUL_LAST

def final_index(ch):
    if not is_hangul_syllable(ch):
        return None
    return (ord(ch) - HANGUL_BASE) % JONGSEONG_COUNT
```

읽는 법:

1. `is_hangul_syllable` : “글자 하나이면서 번호가 44032~55203 사이인가?” 한글 완성 글자인지 확인합니다.
2. `final_index` : 한글이 아니면 `None` (모른다)을 돌려주고, 한글이면 받침 번호를 돌려줍니다.

왜 한글이 아닐 때 `None` 인가? 숫자·영문·기호는 받침이 있는지 알 수 없기 때문입니다. 이 엔진의 원칙은 “모르면 가만히 있기”이고, `None` 이 그 신호입니다. 부르는 쪽은 `None` 을 받으면 판단을 포기합니다.

2.5 이걸로 조사를 어떻게 고르나

조사는 앞말의 마지막 글자 받침에 따라 짝이 정해집니다.

받침 없음(0)	받침 있음
를	을
는	은
가	이
와	과
로	으로

`학교을` 의 마지막 글자는 `교` , 받침 번호는 0, 따라서 `를` **이어야 합니다**. 그런데 `을` 이 적혀 있으니 오류입니다.

`로/으로`만 예외가 하나 있습니다. **ㄹ 받침(8번)은 받침 없는 것처럼 `로`를 씁니다.** `물로`, `서울로`가 그렇습니다. 코드에서는 `rieul_exception=True`일 때 `idx == RIEUL_INDEX`도 "받침 없음"으로 취급합니다.

```
def _expected_particle(host_last_char, no_batchim, has_batchim, rieul_exce
        ption=False):
    idx = _final_index(host_last_char)
    if idx is None:
        return None           # 숫자/영문/기호: 모른다
    no_batchim_case = idx == NO_BATCHIM_INDEX or (rieul_exception and idx
        == RIEUL_INDEX)
    return no_batchim if no_batchim_case else has_batchim
```

2.6 율/률도 같은 계산

`합격률`, `비율`: 모음이나 ㄴ 받침 뒤에는 `율`, 그 밖에는 `률`을 씁니다(맞춤법 제11항). `NIEUN_INDEX = 4`가 이 규칙의 예외를 위한 상수입니다. 같은 `final_index` 하나로 두 규칙을 처리합니다.

2.7 숫자는 왜 판단하지 않는가

`3을`일까 `3를`일까? 3을 "삼"(ㅁ 받침)으로 읽으면 `을`, "쓰리"로 읽으면 `를`입니다. 코드에는 숫자를 읽는 소리로 푸는 표(`_DIGIT_FINAL`)가 만들어져 있는데 **실제로 쓰지 않습니다.**

```
_DIGIT_FINAL = {"0": 21, "1": 8, "2": 0, "3": 16, ...}  # 영/일/이/삼/…
```

한 번 켜 보고 실측했더니 재현율은 0.850 → 0.866으로 올랐지만 **정문 오탐이 0.05% → 0.15%로 3배**가 됐습니다. 원인은 읽는 법이 하나가 아니라는 것입니다.

문장	읽는 법	현재 표기가 맞는가
언리얼 엔진 3**를**	"쓰리"	맞음 (오류 아님)
충정로 3**가**	"가"(街)는 조사가 아니라 지명 단위	오류 아님
여우원숭이 5**과**	"과"(科)는 분류 단위	오류 아님

그래서 결정은 "문맥 신호가 생기기 전에는 숫자 뒤 조사를 판단하지 않는다"였고, 읽기 유틸은 나중을 위해 남겨 두었습니다. **여기서 배울 것:** 맞출 수 있는 걸 일부러 안 맞춘 것입니다. 이 프로젝트에서 "침묵"은 실패가 아니라 선택입니다.

2.8 어절 경계 찾기

`hangul.py`에는 하나가 더 있습니다.

```
def eojeol_bounds(text, i):
    """i번 글자가 속한, 공백으로 나뉜 한 덩어리의 [시작, 끝)."""
    start = i
    while start > 0 and not text[start - 1].isspace():
        start -= 1
    end = i
    while end < len(text) and not text[end].isspace():
        end += 1
    return start, end
```

`i`에서 왼쪽으로 공백이 나올 때까지, 오른쪽으로 공백이 나올 때까지 걸어갑니다. `while`은 "조건이 참인 동안 반복"입니다.

```
text = "나는 학교를 갔다",  i = 5 (교)
왼쪽으로: 4(학) → 3(공백) 멈춤 → start = 4
오른쪽으로: 5 → 6(를) → 7(공백) 멈춤 → end = 7
결과 (4, 7) = "학교를"
```

2.9 확인 문제

1. `"뽀"`의 받침 번호는? (초성 ㅃ, 중성 ㅗ, 종성 없음 → 0)
2. `"밥을"`에서 `을`이 맞는가? (`밥`의 받침 ㅂ=17, 있음 → `을` 맞음)
3. `"서울로"`가 맞는 이유는? (`울`의 받침 ㄹ=8 → 예외로 `로`)

3장. 형태소 분석: 문장을 조각내고 이름표 붙이기

3.1 왜 필요한가

`학교를`과 `학교에서`와 `학교는`은 글자로는 다른 단어처럼 보이지만 모두 `학교` + 조사입니다. 교정기가 "조사 `을`이 맞는가"를 보려면 먼저 **어디까지가 명사이고 어디부터가 조사인지** 알아야 합니다. 이것을 해 주는 프로그램이 **형태소 분석기**이고, 다듬이 쓰는 것은 **Kiwi**(kiwipiepy 0.23.2, LGPL-3.0)입니다.

> 중요: Kiwi는 다듬이 만든 것이 아닙니다. 다듬의 규칙 검사기 대부분이 Kiwi의 분석 결과 위에서 돌아갑니다. Kiwi가 틀리게 읽으면 그 검사기도 틀립니다. 이 장 뒤쪽이 "Kiwi를 믿을 때와 못 믿을 때"입니다.

3.2 용어

용어	뜻	예
어절	공백으로 나뉜 덩어리	`학교를`
형태소	뜻이나 문법 기능이 있는 가장 작은 조각	`학교` / `를`
품사 태그	형태소의 종류 이름표	`NNG`, `JKO`
어간	활용하는 말의 변하지 않는 부분	`먹`(먹다)
어미	어간 뒤에 붙어 문법 기능을 하는 조각	`었`, `다`, `던`
토큰	분석기가 돌려주는 조각 한 개 (형태소 하나에 해당)	

3.3 실제 분석 결과

`kiwi.tokenize(text)`는 토큰의 목록을 돌려줍니다. 토큰 하나에는 `form`(글자), `tag`(품사), `start`(시작 위치), `len`(길이)이 있습니다. 직접 돌려 본 결과입니다.

```
학교를 갔다
  학교 /NNG  start=0 len=2
  를   /JKO  start=2 len=1
  가   /VV   start=4 len=1
  었   /EP   start=4 len=1
  다   /EF   start=5 len=1
```

- `학교/NNG` : 일반명사. `를/JKO` : 목적격 조사.
- `갔다`가 `가/VV` + `었/EP` + `다/EF`로 쪼개졌습니다. `가다`의 `가`(어간) + 과거 `-았/었-`(선어말어미) + 종결어미 `다`.
- 주목: **`가`와 `었`의 start가 둘 다 4**입니다. `갔`이라는 한 글자 안에 두 형태소가 녹아 있기 때문입니다. 또 과거 어미의 원래 모양은 `았`인데 Kiwi는 `었`으로 정규화해 돌려줍니다. 그래서 **`form`이 원문 글자와 항상 같지는 않습니다.** 원문과 대조해야 할 때 코드가 `text[pos:pos+1]`로 다시 확인하는 이유입니다(`confusion.py`의 `든` 잠금).

3.4 품사 태그표 (다듬 코드에 나오는 것들)

태그	이름	예
NNG	일반명사	학교, 친구
NNP	고유명사	손병호, 서울
NNB	의존명사	것, 수
NP	대명사	나, 이것
NR	수사	하나
VV	동사	먹-, 가-
VA	형용사	예쁘-, 많-
VX	보조용언	있-(먹고 있다)
VCP / VCN	이다 / 아니다	
MAG	일반부사	너무, 매우
MM	관형사	이런, 모든
JKS / JKO / JKB / JKG	주격 / 목적격 / 부사격 / 관형격 조사	가, 를, 에, 의
JX / JC	보조사 / 접속조사	은/는, 와/과
EP	선어말어미	었, 시
EF	종결어미	다, 요
EC	연결어미	고, 는데, 든
ETM	관형사형 전성어미	는, 던, ㄴ
ETN	명사형 전성어미	음, 기
XSN / XSV / XSA	명사 / 동사 / 형용사 파생 접미사	-적, -하, -스럽
SF	문장 끝 부호	. ? !
SN / SL / SH	숫자 / 외국어 / 한자	

`-I`, `-R` 이 붙은 태그(`VA-I`, `VV-R`)는 불규칙 활용 표시입니다. 다듬은 `pos_base(tag)` 로 `-` 뒤를 떼어 씁니다.

```python
def pos_base(tag):
    return tag.split("-")[0]     # 'VV-R' -> 'VV'
```

3.5 같은 글자, 다른 분석: 다듬이 품사를 보는 이유

글자만 봐서는 구분할 수 없고 품사를 봐야 답이 갈리는 경우가 이 프로젝트에 많습니다. 같은 `든`인데 Kiwi가 읽는 방식에 따라 이렇게 갈립니다.

```
봤든 사람   → 보/VV  었/EP  든/ETM  사람/NNG     든이 '관형사형 어미'(ETM)
먹었던 밥   → 먹/VV  었/EP  던/ETM  밥/NNG
```

`든`을 ETM(명사를 꾸미는 어미)으로 읽은 자리는 문법상 `던`이어야 합니다. `든`은 연결어미(EC)라서 명사를 꾸미지 못하기 때문입니다. 이 한 줄이 `confusion.py`의 `_adnominal_den_spans`입니다. "확률이 아니라 문법으로 확정"하는 예입니다.

3.6 Kiwi가 틀린 문장을 어떻게 읽는가

오류가 들어간 문장을 Kiwi는 "모르는 말"로 처리하거나 엉뚱하게 읽습니다. 실제로 이렇습니다.

```
학교을 갔다   → 학교/NNG  을/NNG  가/VV ...
```

`을`**이 조사(JKO)가 아니라 명사(NNG)로** 읽혔습니다. 틀린 조사를 조사로 인정하지 않은 것입니다. 그래서 조사 검사기는 "조사로 태그된 것을 찾는" 방식으로 만들 수 없습니다. 오류가 있는 바로 그 자리가 조사로 태그되지 않으니까요.

> `josa.py` 주석의 뜻(의역): 틀린 조사는 Kiwi가 다른 것으로 다시 태깅하기 때문에(`학교을`의 `을`이 NNG가 된다), 태그만 훑으면 실제 오류를 전부 놓친다.

그래서 방식을 뒤집습니다. **명사 토큰이 끝나는 바로 다음 글자**를 직접 보고, 그 글자가 조사 후보(`을/를/은/는/이/가/와/과/로/으로`)이면서 받침 규칙과 어긋나는지 봅니다. 6장에서 코드로 봅니다.

3.7 요청 한 번에 분석은 한 번만: 캐시

검사기가 21개인데 각자 `kiwi.tokenize(text)`를 부르면 같은 문장을 21번 분석합니다. 느립니다. 두 장치가 있습니다.

① **`tokenize_cached`** (**`app/kiwi_singleton.py`**)

```
@lru_cache(maxsize=8)
def tokenize_cached(analyzer, text):
    return analyzer.tokenize(text)
```

`@lru_cache`는 "같은 입력이 오면 계산 없이 저번 결과를 돌려주라"는 표시입니다(최근 8개 기억). 주의 문구가 달려 있습니다: **"반환된 목록은 공유되므로 읽기 전용으로 쓸 것."** 한 검사기가 목록을 고치면 다른 검사기가 오염됩니다.

② **`_SharedAnalysis`** (**`app/engine.py`**)

요청 하나 동안만 `tokenize` 결과를 나눠 쓰는 얇은 포장입니다. 검사기들은 `kiwi.tokenize(text)`를 평소처럼 부르는데, 포장이 중간에서 캐시를 끼워 줍니다. 요청마다 새로 만들어서 요청 사이에 메모리가 쌓이지 않습니다.

3.8 Kiwi 준비 운동

```
kiwi = Kiwi(load_default_dict=True, load_typo_dict=False, load_multi_dict=
        False)
kiwi.tokenize("다듬")       # 서버가 뜰 때 한 번 돌려 둔다
```

- Kiwi의 사전을 어디까지 적재할지는 **메모리와 정확도의 교환**입니다. 파일 안 주석에 실측표가 있습니다.

설정	메모리	부팅	FPR(오탐률)
사전 전부	578MB	0.94초	.0009
다중어 사전 끔 (현재)	429MB	0.46초	.0009
기본 사전도 끔	344MB	0.33초	.0014

다중어 사전을 끄면 정밀도·오탐률이 소수 넷째 자리까지 같고 메모리가 149MB 줄어서 껐습니다. 기본 사전은 켜 둡니다. 껐을 때 오탐률이 .0009 → .0013으로 오르는데, 이 엔진이 가장 신경 쓰는 게 오탐이라 "조용히 맞바꾸지 않는다"는 판단입니다.

- 마지막 줄 `kiwi.tokenize("다듬")`은 **미리 한 번 돌려서** 내부 구조를 만들어 두는 것입니다. 안 하면 첫 사용자가 120MB 할당과 약 300ms 지연을 떠안습니다.

3.9 문장 나누기와 토큰 자르기 도우미 (`app/morphology.py`)

```
def sentence_slices(tokens):
    """SF(문장 끝 부호) 뒤마다 끊어 (시작 토큰 번호, 끝 토큰 번호) 목록을 만든다."""
```

문서 전체를 한꺼번에 받으므로 "문장의 마지막 어미"를 보는 검사는 문장 경계를 알아야 합니다. 이 함수가 토큰 목록을 문장 단위로 자릅니다.

```
def tokens_within(tokens, start, end):
    """[start, end) 안에 들어 있는 토큰만."""
```

어절 하나만 따로 분석하지 않고 **문장 분석에서 잘라 쓰는** 이유가 주석에 있습니다: 어절만 따로 분석하면 태그가 달라질 수 있기 때문입니다(`계시`는 문장 안에서 VV, 혼자일 때 VX). 교정이 조용히 안 먹히는 일이 생깁니다.

```
def join_changed(kiwi, morphemes, original):
    """고친 형태소 목록을 kiwi.join으로 글자로 되돌린다. 바뀐 게 없으면 None."""
```

이 세 개가 형태소 기반 교정(`morph_rules.py` 등 6개 검사기)의 공통 부품입니다. "어간을 고친 뒤 `kiwi.join`으로 활용형을 다시 만든다"는 방식이라, 문자열 규칙이 `잊혀지`만 알던 자리의 **모든 활용형**(잊혀졌다, 잊혀지는, 잊혀질…)을 한 규칙으로 잡습니다.

3.10 확인 문제

1. `학교를 갔다`에서 `를`의 품사는? (JKO, 목적격 조사)
2. 왜 `갔다`에서 `가`와 `었`이 같은 start를 가지나? (한 글자에 두 형태소가 녹아 있어서)
3. 틀린 조사를 찾을 때 태그가 JKO인 토큰을 찾으면 안 되는 이유는? (틀린 조사는 NNG 등으로 재해석되어 JKO로 안 잡힘)

4장. 서버와 API: 요청 하나가 지나가는 길

4.1 서버와 클라이언트

식당에 비유하면 이렇습니다.

식당	다듬
손님	**클라이언트**: 웹 페이지, 크롬 확장, 맥 앱
주방	**서버**: `backend/`의 파이썬 프로그램
주문서	**요청(request)**: "이 글을 검사해 줘"
음식	**응답(response)**: 지적 목록
메뉴판의 정해진 주문 방식	**API**

손님(화면)은 직접 요리하지 않습니다. 주문서만 써서 주방에 넘기고 음식을 받아 보여 줍니다. 그래서 **교정 로직은 서버 하나에만** 있고, 화면 세 개는 같은 서버를 부릅니다. 서버는 내 컴퓨터 안(`localhost:8000`)에서 돌기 때문에 글이 인터넷으로 나가지 않습니다.

4.2 API란

API는 "이렇게 주문하면 이렇게 답한다"는 약속입니다. 다듬의 API는 둘뿐입니다.

주소	방식	하는 일
`/api/check`	POST	글을 받아 지적 목록을 돌려줌
`/api/health`	GET	"살아 있나, 어느 층이 켜져 있나" 확인

POST는 "내용을 담아 보낸다", GET은 "그냥 달라고 한다"입니다.

4.3 주문서와 음식의 모양: `schemas.py`

```python
class CheckRequest(BaseModel):
    text: str = Field(max_length=20_000)

class Issue(BaseModel):
    id: int
    start: int
    end: int
    category: Category          # "spacing" | "spelling" | "josa" | "gramm
        ar" | "style"
    source: str
    original: str
    suggestions: list[str]
    message: str
    confidence: Confidence      # "high" | "medium" | "low"

class CheckResponse(BaseModel):
    issues: list[Issue]
    elapsed_ms: dict[str, float] = {}
```

- `BaseModel`(pydantic)은 "이 모양이 아니면 받지 않는다"는 검문소입니다. 모양이 틀린 주문서가 오면 서버가 안으로 들이지 않고 **422** 오류를 돌려줍니다.
- `max_length=20_000`: 글은 2만 자까지만. 원인은 서버 보호입니다. 2만 자는 서버 한 명을 약 4초 붙잡는 길이입니다.
- `Literal[...]`(글자 고정 목록): `category`는 다섯 값 중 하나여야 합니다.

실제로 있었던 사고: `style`이 빠져서 서버가 죽었다

`schemas.py` 맨 위 주석에 적힌 이야기입니다.

> 문체 일관성 검사기(`consistency.py`)는 처음부터 `category="style"`을 냈는데, 스키마의 `Category` 목록에 `style`이 없었습니다. 문체가 섞인 글을 검사하면 **`/api/check`가 500 오류로 죽었습니다**(2026-08-30 실측).

왜 테스트가 못 잡았나? 두 측정이 서로 다른 길을 탔기 때문입니다.

측정	길	결과
D 경로 게이트	검사기를 **직접** 불러서 잼	재현율 1.000 (정상으로 보임)
corpus-suite	**API를 거쳐서** 잼	예외로 죽어 0.000

두 숫자가 정반대인데 아무도 비교하지 않았습니다. 이후 "체커가 내는 모든 category가 스키마에 있는지 소스를 훑어 대조하는 계약 테스트"(`tests/test_category_contract.py`)가 생겼습니다. **교훈:** 같은 것을 두 군데서 다른 방법으로 재면, 둘이 어긋나는지 서로 대조해야 합니다.

4.4 서버 뼈대: `app/main.py`

```
app = FastAPI(title="다듬 API", version=APP_VERSION)
app.add_middleware(CORSMiddleware, allow_origins=..., allow_methods=["GE
        T","POST"], ...)
app.middleware("http")(rate_limit_middleware)
app.include_router(api_router, prefix="/api")
app.mount("/core", StaticFiles(directory=CORE_DIR), name="core")
app.mount("/", StaticFiles(directory=FRONTEND_DIR, html=True), name="front
        end")
```

- **FastAPI**: 파이썬으로 서버를 만드는 틀(프레임워크)입니다.
- **미들웨어**: 요청이 본 로직에 닿기 전에 지나가는 관문입니다. 여기서는 CORS와 레이트 리밋이 관문입니다.
- **라우터**: 주소별로 어느 함수가 맡을지 정한 배정표입니다. `prefix="/api"`라서 `check` 함수는 `/api/check`가 됩니다.
- **mount**: 폴더를 통째로 웹 주소에 연결합니다. 웹 앱의 `index.html`, `app.js`도 이 서버가 내줍니다.

등록 순서가 동작입니다. 주석의 경고: 정적 파일 mount는 "무엇이든 받는" 그물이라서 `/api` 라우터보다 먼저 등록하면 `/api/check` 요청까지 삼켜 버립니다. 그래서 라우터를 먼저 등록합니다.

CORS

브라우저는 보안상 "다른 곳에서 온 페이지가 이 서버를 부르는 것"을 기본으로 막습니다. 크롬 확장은 `chrome-extension://` 출처라서 막힙니다. 서버는 쿠키·로그인이 없으므로 모두 허용(`*`)이 안전하다고 판단했고, 운영에서는 `ALLOWED_ORIGINS` 환경변수로 좁힐 수 있습니다.

캐시 재검증 미들웨어

```
@app.middleware("http")
async def revalidate_static(request, call_next):
    response = await call_next(request)
    if not request.url.path.startswith("/api"):
        response.headers["Cache-Control"] = "no-cache"
    return response
```

정적 파일은 기본으로 캐시 지시가 없어서 브라우저가 낡은 `app.js`를 쓰는 일이 있었습니다(배포 후에도 옛 화면). `no-cache`는 "저장은 하되 쓰기 전에 서버에 확인"이라는 뜻이고, 안 바뀌었으면 가벼운 304 응답으로 끝납니다.

4.5 레이트리밋: `ratelimit.py`

한 사용자가 서버를 독점하지 못하게 합니다. 검사는 CPU를 쓰기 때문입니다.

```
IP마다: 60초 창 안에서 60번까지. 61번째부터 429 오류 + "몇 초 뒤 재시도" 안내.
```

작동 방식은 단순합니다.

```
_buckets[key] = (window_start, count)     # IP → (창 시작 시각, 횟수)
...
if now - window_start >= WINDOW:          # 창이 끝났으면 새로 시작
    window_start, count = now, 0
count += 1
if count > REQUESTS_PER_WINDOW:
    return JSONResponse(..., status_code=429, headers={"Retry-After":
        ...})
```

- `threading.Lock()`: 동시에 여러 요청이 `_buckets`를 고치면 값이 꼬이므로 한 번에 한 명만 들어가게 하는 자물쇠입니다.
- 5분마다 오래된 키를 청소합니다(메모리가 계속 늘지 않게).
- **한계가 코드에 적혀 있습니다.** 메모리 안에서만 세므로 서버가 여러 대면 공유가 안 됩니다. 한 대로 쓰는 지금은 충분하다는 판단입니다.

어느 IP인가: `clientip.py`

프록시 뒤에서는 모든 요청의 접속자가 프록시로 보입니다. 원래 IP는 `X-Forwarded-For` 헤더에 있는데, **이 헤더는 누구나 위조할 수 있습니다.** 위조를 믿으면 요청마다 헤더를 바꿔 한도를 무한히 우회합니다.

설정	동작
기본	헤더를 믿지 않고 직접 접속한 IP만 사용
`TRUSTED_PROXIES=IP목록`	접속자가 그 목록일 때만 헤더를 믿음 (권장)
`TRUST_PROXY=1`	옛 방식(누구 헤더든 믿음), 경고 출력

예전엔 기본값이 "믿음"이라 우회가 가능했고, 기본을 "안 믿음"으로 뒤집었습니다. **안전한 쪽이 기본값**이어야 한다는 원칙입니다.

4.6 `/api/check` 한 줄씩

```
@router.post("/check", response_model=CheckResponse)
def check(request: CheckRequest, x_dadeum_client: str | None = Header(defa
        ult=None)):
    _note_client(x_dadeum_client)
    text = request.text
    if not text.strip():
        return CheckResponse(issues=[])

    result = analyze_long(text)
    return CheckResponse(
        issues=[build_issue(idx, r) for idx, r in enumerate(result.issue
        s)],
        elapsed_ms=result.elapsed_ms,
    )
```

1. `@router.post("/check")` : 데코레이터. "아래 함수를 `/check` POST 주문 담당으로 등록하라."
2. `request: CheckRequest` : 검문소를 통과한 주문서가 들어옵니다.
3. `x_dadeum_client` : 헤더 `X-Dadeum-Client`를 읽습니다. 확장이 "나 여기 있다"고 알리는 용도입니다(4.8).
4. 빈 글이면 바로 빈 목록.
5. `analyze_long(text)`가 엔진을 돌립니다(5장).

6. `build_issue(idx, r)`로 내부 자료(`RawIssue`)를 응답 모양(`Issue`)으로 바꾸고, 목록 번호를 `id`로 붙입니다.

실제 응답은 이런 JSON입니다(예문 `학교을 갔다`, 제가 `check()`를 직접 불러 얻은 값).

```
{"issues": [{"id": 0, "start": 2, "end": 3, "category": "josa",
             "source": "josa_rule", "original": "을", "suggestions": ["를"],
             "message": "받침 여부를 확인하면 '를'이 맞는 조사입니다.",
             "confidence": "high"}],
 "elapsed_ms": {"layer1": 4278.2, "layer2": 2.0}}
```

- `message`의 끝 `이/가`는 **제안한 조사 자신의 받침**으로 고릅니다(`_issue()`의 `marker`). `를`은 받침 ㄹ이 있어 `'를'이`, `가`는 받침이 없어 `'가'가`가 됩니다. 설명문조차 같은 받침 계산으로 맞춰 쓰는 것입니다.
- `layer1`이 4278ms로 큰 것은 **서버를 막 띄운 뒤 첫 요청**이라서입니다(사전·모델을 처음 읽는 비용). 두 번째부터는 한 자릿수 ms입니다. 이 값은 `/check` 응답에 그대로 실려서 "어느 층이 시간을 쓰는지" 보는 용도입니다.

4.7 긴 글: `analyze_long`

```
CHUNK_LIMIT = 4000

def analyze_long(text, use_l2=True):
    if len(text) <= CHUNK_LIMIT:
        return analyze(text, use_l2)
    ...
    for start, end in _chunk_bounds(text, CHUNK_LIMIT):
        part = analyze(text[start:end], use_l2)
        for r in part.issues:
            issues.append(replace(r, start=r.start + start, end=r.end + st
        art))
```

4000자를 넘으면 **문장 경계에서** 잘라 조각마다 따로 돌리고, 위치(start, end)를 원문 기준으로 되돌립니다(조각의 시작 위치를 더함).

왜 필요했나? 예전에는 스키마 상한이 5,000자였고, 클라이언트들이 글을 자르지 않고 통째로 보냈습니다. **긴 문서는 422로 튕겨서 검사가 아예 안 됐습니다**(데스크톱·확장·웹 모두 해당). 이후 엔진이 직접 나눠 돌리게 바꿨고 상한을 2만 자로 올렸습니다.

`_chunk_bounds`의 핵심:

```
for sent in _kiwi.split_into_sents(text):
    if sent.end - start > limit and cur > start:
        bounds.append((start, cur))      # 여기까지 한 조각
        start = cur
    while sent.end - start > limit:      # 마침표 없이 이어진 아주 긴 덩어리
        bounds.append((start, start + limit))
        start += limit
    cur = sent.end
```

- Kiwi의 `split_into_sents`가 문장 경계를 줍니다.
- 문장이 상한을 넘기면 그 문장만 억지로 4000자씩 자릅니다.

한계(주석에 있음): 문체 일관성은 글 전체를 봐야 하는데 조각 안에서만 판정합니다.

4.8 `/api/health`: 지금 어느 층이 켜져 있나

```
{"status": "ok", "version": ..., "model_family": "gyeol-1",
 "layer2_pairs": ["an", "doe", "den", "roseo"],
 "boundary_model": True, "lexicon": 1806050,
 "spacing_tagger": False, "extension_live": False}
```

왜 이렇게 자세한가: 이 엔진의 층은 전부 "선택"입니다. 모델 파일이 없거나 torch가 없으면 그 층만 조용히 빠집니다. 그러면 "내 컴퓨터에서는 잡히는데 배포본에서는 안 잡힌다" 같은 일이 생기고, 원인을 찾기 어렵습니다. `/api/health`가 각 층의 적재 여부를 답해 주므로 **결과가 다를 때 가장 먼저 볼 곳**입니다.

`extension_live`는 맥 앱이 읽습니다. 맥 앱은 크롬에서 확장이 담당하므로 "확장이 살아 있을 때만" 비켜섭니다. 확장이 `check`/`health` 요청마다 `X-Dadeum-Client: extension` 헤더를 달고, 서버가 마지막으로 본 시각을 기억해 **180초 안이면 살아 있다**고 답합니다. (설치 안 된 확장을 믿고 맥 앱이 비켜서면 크롬에서 아무 교정도 안 나오는 일이 있었습니다. 그것을 막는 장치입니다.)

4.9 직접 따라 하기

서버를 띄우고 직접 주문해 볼 수 있습니다.

```
cd dadeum/backend
source .venv/bin/activate
uvicorn app.main:app --port 8000

# 다른 터미널에서
curl -s localhost:8000/api/health
curl -s -X POST localhost:8000/api/check \
     -H "Content-Type: application/json" \
     -d '{"text": "학교을 갔다"}'
```

4.10 확인 문제

1. 글이 25,000자이면 어떤 응답이 올까? (422: 스키마 상한 2만 자 초과)
2. 같은 IP가 1분에 61번 요청하면? (61번째부터 429 + Retry-After)
3. 서버가 `style`로 죽었던 이유는? (스키마의 Category 목록에 style이 없었음)

5장. 엔진 뼈대: 후보를 모으고 정리하는 구조

6~10장에서 개별 검사기를 하나씩 봅니다. 그 전에 **검사기들을 감싸는 틀**을 이해해야 합니다. 이 틀이 `backend/app/engine.py`(183줄)와 `backend/app/checkers/base.py`, `sources.py`입니다.

5.1 한 문장으로

> **검사기 21개가 각자 "여기 틀린 것 같다"는 후보를 내면, 겹치는 후보를 우선순위로 정리해서 한 목록으로 돌려준다.**

> 문장 → ① 분석 → ② 후보 → ③ 병합 → 지적 목록

5.2 왜 "하나의 흐름"인가: 만들어진 이유

`engine.py` 맨 위 설명문에 이유가 적혀 있습니다.

> 이 흐름이 예전엔 두 군데에 따로 적혀 있었다: `app/api.py`의 `/check`와 `eval/qa/common.py`의 `run_local`. 한쪽에만 검사기를 더한 채로 어긋나, **문체 일관성 재현율이 평가에서 0으로 찍힌 적이 있다**(측정 도구가 제품과 다른 코드를 밟았다).

즉 "제품이 돌리는 길"과 "성적을 재는 길"이 서로 다른 복사본이었습니다. 제품에는 검사기를 넣었는데 평가 쪽에는 안 넣어서, 평가는 "그 검사기는 아무것도 못 잡는다"고 보고했습니다. 지금은 둘 다 `engine.analyze()` 한 함수만 부릅니다. **검사기를 더하거나 빼는 곳이 `GENERATORS` 목록 한 줄**입니다.

> 이 원칙의 뿌리는 4장에서 본 `style` 사건과 같습니다. "재는 길과 쓰는 길이 같아야 한다."

5.3 자료: `RawIssue`

모든 검사기가 같은 모양으로 후보를 냅니다(`checkers/base.py`).

필드	뜻	예
`start`, `end`	원문 위치 `[start, end)`	`2`, `3`
`category`	종류	`josa`
`source`	어느 규칙이 냈나	`josa_rule`
`original`	원문의 그 부분	`을`
`suggestions`	고칠 말 목록 (비어 있을 수 있음)	`["를"]`
`message`	사람에게 보여 줄 설명	
`confidence`	`high` / `medium` / `low`	`high`

`suggestions`가 비어 있다는 것은 "여기가 수상하다"만 말하고 고칠 말은 없다는 뜻입니다. 이상탐지, 문체 일관성이 그렇습니다.

5.4 검사기 등록: `GENERATORS`

```python
@dataclass(frozen=True)
class Generator:
    name: str
    layer: int      # 1 = 규칙·로컬 모델, 2 = 혼동쌍 분류기
    run: Callable[[str, object], list[RawIssue]]

GENERATORS = [
    Generator("spacing", 1, check_spacing),
    Generator("dictionary", 1, lambda t, k: check_dictionary(t)),
    Generator("morph", 1, check_morph_rules),
    Generator("josa", 1, check_josa),
    Generator("jamo", 1, check_jamo),
    ...
    Generator("confusion", 2, lambda t, k: check_confusion(t)),
    Generator("boundary", 2, check_boundary),
]
```

모든 검사기는 **같은 약속**을 지킵니다: `(글, Kiwi) → 후보 목록`. 어떤 검사기는 Kiwi가 필요 없어서 `lambda t, k: check_dictionary(t)`처럼 `k`를 버립니다.

`layer`는 소요 시간을 층별로 나눠 재기 위한 꼬리표입니다(응답의 `elapsed_ms`). 이름이 "2층"이지만 실행 순서가 아니라 **분류**입니다. 목록 순서대로 하나씩 실행됩니다.

21개의 이름과 담당

이름	담당	장
spacing	띄어쓰기 (태거 + 삭제 + 규칙 30여 개)	10
dictionary	사전 규칙 474개 + `에요/예요`	6
morph	형태소 고쳐 활용형 되살리기	6
josa	받침에 따른 조사	6
jamo	흩어진 자모 합치기	6
dependent_noun	의존명사 띄어쓰기	6
particle_spacing	조사를 앞말에 붙이기	6
yul_ryul	율/률	6
quotation	간접 인용 `-라고` 겹침	6
question_mark	의문문 부호	6
honorific	사물 존대	6
animacy	에게/에	6
negative_polarity	결코·전혀 + 부정	6
tense	시제 호응	6
causative	불필요한 `-시키다`	6
subject_predicate	주술 호응	6
consistency	문체 섞임	6
typo	키보드 오타	9
anomaly	낯선 어절	9
confusion	되/돼 등 혼동쌍 모델	7
boundary	단어 경계 모델	8

5.5 `analyze()` 한 줄씩

```
def analyze(text, use_l2=True):
    shared = _SharedAnalysis(_kiwi)
    candidates = []
    elapsed = {1: 0.0, 2: 0.0}
    for gen in GENERATORS:
        if gen.layer == 2 and not use_l2:
            continue
        t0 = perf_counter()
        candidates += gen.run(text, shared)
        elapsed[gen.layer] += perf_counter() - t0
    return Analysis(
        issues=merge_and_dedupe(candidates),
        elapsed_ms={"layer1": ..., "layer2": ...},
    )
```

1. `_SharedAnalysis(_kiwi)` : 이 요청 동안만 쓰는 분석 공유 캐시(3장).
2. `for gen in GENERATORS` : 검사기를 **등록된 순서대로** 하나씩.
3. `candidates += gen.run(text, shared)` : 후보를 한 바구니에 쏟아 넣습니다. 이 시점에는 정리하지 않습니다.
4. `perf_counter()` : 시계. 실행 전후를 재서 층별 시간을 모읍니다.
5. 마지막에 `merge_and_dedupe` 로 한 번에 정리합니다.

`use_l2=False` 로 2층을 끄고 1층만 돌릴 수 있습니다(평가에서 층별 기여를 따로 재기 위해).

> **순서는 동작의 일부입니다.** 병합은 "같은 자리·같은 길이면 먼저 온 쪽이 남는" 안정 정렬이라서, 목록 순서를 바꾸면 출력이 바뀝니다. 그래서 목록 순서도 코드의 일부로 취급합니다.

5.6 병합: `merge_and_dedupe` (엔진의 심장)

검사기들은 서로를 모릅니다. 그래서 같은 자리를 둘 이상이 지적하는 일이 흔합니다. 예문 `안되요` 에서는 사전 규칙이 `되요→돼요` 를, 혼동쌍 모델이 `되→돼` 를 같은 자리에 냈습니다. 둘 다 보여 주면 겹친 밑줄이 되므로 **하나만** 남깁니다.

알고리즘

```
ordered = sorted(issues, key=lambda i: (i.start, -(i.end - i.start)))
kept = []
cursor = -1
for issue in ordered:
    if issue.start >= cursor:           # 앞의 것과 안 겹침
        kept.append(issue); cursor = issue.end
        continue
    prev = kept[-1]                     # 겹침 → 승부
    if _has_fix(issue) != _has_fix(prev):
        if _has_fix(issue):
            kept[-1] = issue; cursor = issue.end
        continue                        # (제안 있는 쪽이 이김)
    if SOURCE_PRIORITY[issue.source] > SOURCE_PRIORITY[prev.source]:
        kept[-1] = issue; cursor = issue.end
return _drop_noise_adjacent_to_fixed_typos(kept)
```

규칙을 말로 풀면

1. **정렬**: 시작 위치가 빠른 순, 같으면 긴 것 먼저.
2. 하나씩 보면서, 직전에 남긴 것과 **안 겹치면 남깁니다.**
3. **겹치면 승부**:
    - **(가)** 한쪽만 고칠 말이 있으면 → **고칠 말이 있는 쪽**이 이긴다.
    - **(나)** 둘 다 있거나 둘 다 없으면 → **출처 우선순위 숫자가 큰 쪽**.
    - **(다)** 우선순위도 같으면 → **먼저 온 쪽**이 남는다(바꾸지 않으므로).

여섯 가지 상황을 직접 돌려 본 결과

#	입력	결과	이유
1	`(0–2 dict)` `(5–7 josa)`	둘 다 남음	안 겹침
2	`(26–28 dict, 제안)` `(26–27 ml_confusion, 제안)`	**dict**	둘 다 제안, 우선순위 5 > 4
3	`(30–32 anomaly, 제안 없음)` `(30–32 typo, 제안 있음)`	**typo**	(가) 제안 있는 쪽
4	`(0–2 kiwi_space)` `(0–2 tagger_space)`	**kiwi_space**	우선순위 0 = 0, 먼저 온 쪽
5	`(2–3 josa)` `(0–5 dict)`	**(0-5 dict)**	긴 쪽이 먼저 정렬됨, josa(2)는 dict(5)에 짐
6	`(0–3 dict 희한, high)` `(3–4 kiwi_space, medium)`	**dict만**	아래 5.8

왜 “제안 있는 쪽이 먼저”인가 — 실제 사고

코드 주석(2026-08-30)입니다.

> 교정을 만들지 않는 층(이상탐지, 문체 일관성)은 “여기가 수상하다”만 말한다. 그런 지적이 어절 전체를 덮으면 그 안의 진짜 교정을 밀어낼 수 있다. 실제로 그랬다: `자률신경` **을 이상탐지가 통째로 잡아** `률→율` **교정이 사라졌고** 회귀 레지스트리가 그걸 잡아냈다. 우선순위 숫자만으로는 못 막는다. **조언은 교정에 양보한다.**

이상탐지의 우선순위가 0이고 `yul_ryul_rule`도 0이라 숫자만으로는 비길 때가 있었던 것입니다. 그래서 “숫자 비교 이전에 제안 유무로 먼저 가른다”는 규칙이 앞에 들어갔습니다.

5.7 출처 우선순위표: `sources.py`

58개 출처가 한 표에 적혀 있습니다. 구조는 이렇습니다.

```python
@dataclass(frozen=True)
class Source:
    name: str       # "josa_rule"
    layer: str      # "rule" | "model" | "advice"
    priority: int
    basis: str      # 근거 설명
```

층(layer)의 뜻

층	뜻
`rule`	결정적 규칙. 같은 입력이면 늘 같은 출력
`model`	통계·학습 모델 (혼동쌍, 태거, 경계 판정기 등)
`advice`	교정을 만들지 않고 "확인해 보라"만 말함

우선순위

값	출처	이유
6	jamo, quotation, honorific, animacy, negative_polarity, tense, causative, agreement	문법 규칙이 문맥을 보고 확정한 것. 가장 특수하고 신뢰 높음
5	dict_rule	국립국어원 기준 사전 규칙
4	ml_confusion	혼동쌍 학습 모델 (형태론 확정 포함)
3	dependent_noun_rule, particle_spacing_rule	의존명사·조사 붙이기
2	josa_rule	받침 조사
1	typo_rule, boundary_model	오타·경계 모델
0	나머지 (띄어쓰기 세부 규칙 대부분, kiwi_space, tagger_space, consistency, anomaly)	기본값

원칙: **"더 특수하고 더 선별된 근거가 일반적인 것을 이긴다."**

표 없이 쓰던 시절의 사고

`sources.py` 맨 위 주석입니다.

> 예전에는 우선순위가 `base.py` 의 딕셔너리에만 있었고, 2026-09-13에 추가한 띄어쓰기 규칙 11종은 **그 표에 없는 채로 기본값 0으로 돌았다.** 출처를 더하는 곳을 한 군데로 모으고, **표에 없는 출처가 나오면 테스트가 실패하게 했다** (`tests/test_engine.py`).

"규칙을 더했는데 우선순위를 안 적어서 조용히 0이 된" 사고입니다. 기본값에 의존하면 아무도 모르게 틀립니다. 그래서 **"모르는 출처가 나오면 시끄럽게 실패"**로 바꿨습니다.

```
def priority(name):
    s = SOURCES.get(name)
    return s.priority if s else 0
```

(이 함수 자체는 여전히 기본값 0을 주지만, 테스트가 "엔진이 내는 모든 source가 표에 있다"를 검증합니다.)

5.8 마지막 청소: `_drop_noise_adjacent_to_fixed_typos`

```
anchors = [i for i in issues if i.source == "dict_rule" and i.confidence =
        = "high" and i.suggestions]
return [issue for issue in issues
        if not (issue.source in ("kiwi_space", "tagger_space", "kiwi_typ
        o")
                and issue.confidence != "high"
                and any(issue.start == a.end or issue.end == a.start for a
        in anchors))]
```

상황: `희안하게` 는 맞춤법 오류(`희한하게`)입니다. 사전 규칙이 `희안→희한` 을 high로 잡습니다. 그런데 틀린 철자 때문에 Kiwi가 이 단어를 엉뚱하게 해석해서 바로 옆에 "`하` 를 따로 띄우자"는 쓸데없는 제안이 같이 나옵니다.

해결: 확실한 사전 규칙 교정의 **바로 옆에 붙어 있는** 애매한 띄어쓰기·오타 후보는 버립니다. 맞춤법을 고치고 나면 그 전제(틀린 철자)가 사라지므로 쓸데없는 제안이 되기 때문입니다. `i.start == a.end or i.end == a.start` 는 "앞뒤로 딱 붙어 있다"를 뜻합니다.

5.9 예문 전체 추적

```
학교을 가는길에 친구를 만낫는데 그렇게 하면 안되요. 매읽 그 영화 봤든 사람이 너무많아서 놀랐다.
```

② 후보 (검사기가 낸 것, 실제 실행)

```
[spacing    ] 47–48 '많'  → ' 많'   high  degree_adverb_rule
[spacing    ] 25–26 '안'  → '안 '   high  negation_context_rule
[spacing    ]  6–7  '길'  → ' 길'   low   tagger_space
[dictionary ] 26–28 '되요' → '돼요'  high  dict_rule
[josa       ]  2–3  '을'  → '를'    high  josa_rule
[typo       ] 30–32 '매읽' → '매일'  low   typo_rule
[anomaly    ] 13–17 '만낫는데' → []  low   anomaly
[anomaly    ] 30–32 '매읽'  → []    low   anomaly
[confusion  ] 26–27 '되'  → '돼'    high  ml_confusion
[confusion  ] 39–40 '든'  → '던'    high  ml_confusion
```

후보 10개 (이 환경은 띄어쓰기 태거가 켜져 있습니다. 꺼진 환경에서는 `길` 항목이 없습니다).

③ 병합 뒤 (8개)

```
 2–3   '을'  → '를'    high  josa
 6–7   '길'  → ' 길'   low   spacing  (tagger)
13–17  '만낫는데' → [] low   spelling (anomaly)
25–26  '안'  → '안 '   high  spacing
26–28  '되요' → '돼요'  high  spelling (dict_rule)
30–32  '매읽' → '매일'  low   spelling (typo_rule)
39–40  '든'  → '던'    high  grammar (ml_confusion)
47–48  '많'  → ' 많'   high  spacing
```

빠진 두 개: - `(26–27 되→돼, ml_confusion)` : `(26–28 되요→돼요, dict_rule)`에 졌습니다. 정렬에서 같은 start(26)이면 긴 쪽(26-28)이 먼저 오고, 짧은 쪽은 겹치므로 우선순위 5 > 4로 탈락. - `(30–32 매읽, anomaly)` : `typo_rule`이 고칠 말을 가졌으므로 탈락(가).

한 가지 눈여겨볼 점: 서로 다른 길이 같은 곳을 가리킨다

`안되요`는 두 가지가 동시에 맞습니다: `안` 뒤에 공백(`안 되요`)과 `되요→돼요`. 둘은 위치가 달라서(25-26, 26-28) 겹치지 않고 **둘 다 남습니다.** "모두 고치기"가 앞에서부터 하나씩 적용하면 `안 돼요`가 됩니다(12장).

5.10 확인 문제

1. 겹치는 두 후보가 모두 제안이 있고 우선순위가 같으면? (먼저 정렬된 것이 남음, 정렬 키는 시작 위치·긴 것 먼저)
2. 이상탐지(제안 없음, 우선순위 0)가 사전 규칙(제안 있음)과 겹치면 누가 이기나? (사전 규칙. 우선순위 비교 이전에 “제안 있음”이 이긴다)
3. `GENERATORS`의 순서를 바꾸면 안 되는 이유? (같은 자리·같은 길이·같은 우선순위일 때 먼저 온 쪽이 남기 때문)

6장. 규칙 검사기: 문법을 코드로 옮긴 것들

이 장은 모델(학습)이 아니라 **사람이 규범을 읽고 코드로 옮긴 규칙**을 다룹니다. "같은 입력이면 늘 같은 출력"이라 이해하기 쉽고, 그만큼 **예외 처리**가 코드의 대부분입니다. 이 장의 모든 실행 결과는 제가 검사기 함수를 직접 불러 얻은 값입니다.

6.0 규칙 검사기의 공통 설계

거의 모든 검사기가 같은 세 단계를 밟습니다.

```
① 후보 찾기   : 형태소 태그나 정규식으로 "여기가 의심스럽다"를 찾는다
② 함정 거르기 : 오탐이 날 만한 자리를 allow-list / frame / 가드로 제외한다
③ 지적 만들기 : RawIssue 를 만든다 (필요하면 kiwi.join 으로 고친 말을 조립)
```

코드를 읽을 때 **분량의 80%는 ②**입니다. ①은 보통 서너 줄이고, ②에 "이런 경우는 정문이다"라는 실측 사연이 줄줄이 붙어 있습니다. 이 프로젝트에서 규칙을 쓰는 일은 "규칙을 만드는 일"이 아니라 **"규칙이 틀리게 발동하는 자리를 하나씩 막는 일"**입니다.

공통 부품

부품	위치	역할
`tokenize_cached(kiwi, text)`	`kiwi_singleton.py`	형태소 분석(요청당 1번)
`eojeol_bounds(text, i)`	`hangul.py`	i번 글자가 속한 어절의 `[시작, 끝)`
`final_index(ch)`	`hangul.py`	받침 번호
`tokens_within(tokens, s, e)`	`morphology.py`	구간 안의 토큰만
`join_changed(kiwi, morphs, orig)`	`morphology.py`	고친 형태소를 글자로 조립
`sentence_slices(tokens)`	`morphology.py`	문장 단위로 자름

두 가지 고치는 방식

방식	어떻게	어디서
문자열 치환	`을`→`를` 처럼 정한 글자로 바꿈	josa, dictionary, yul_ryul
형태소 재조립	형태소 목록을 고치고 `kiwi.join`으로 활용형을 되살림	honorific, tense, negative_polarity, morph_rules …

재조립은 "포기할 것이다"를 "포기하지 않을 것이다"로 바꿀 때처럼, 활용이 얽힌 자리를 **Kiwi의 결합 규칙에 맡기는** 방법입니다. 활용마다 규칙을 따로 적지 않아도 됩니다.

6.1 jamo.py: 흩어진 자모 합치기

목적: `ㅂㅐ고프다`처럼 글자가 합쳐지지 않은 것을 `배고프다`로. **왜:** 입력기(IME)가 조합에 실패해서 `ㅂ`과 `ㅐ`가 따로 남는 일이 있습니다.

로직

정규식으로 **"자음 다음에 바로 모음"**인 모양을 찾습니다.

```
CHO  = "ㄱㄲㄴㄷㄸㄹㅁㅂㅃㅅㅆㅇㅈㅉㅊㅋㅌㅍㅎ"      # 초성 19개
JUNG = "ㅏㅐㅑㅒㅓㅔㅕㅖㅗㅘㅙㅚㅛㅜㅝㅞㅟㅠㅡㅢㅣ"   # 중성 21개
_PATTERN = re.compile(f"[{CHO}] ?[{JUNG}](?:[{JONG[1:]}](?![{JUNG}]))?")
```

의미: 초성 한 글자 + (공백 있어도 됨) + 중성 한 글자 + (받침 있어도 됨). 합칠 때는 2장의 공식을 거꾸로 씁니다.

```
def _compose(cho_char, jung_char, jong_char=""):
    cho_idx  = CHO.index(cho_char)
    jung_idx = JUNG.index(jung_char)
    jong_idx = JONG.index(jong_char) if jong_char else 0
    return chr(0xAC00 + (cho_idx * 21 + jung_idx) * 28 + jong_idx)
```

`ㅂ`(초성 7번), `ㅐ`(중성 1번) → `(7*21+1)*28 = 4144` → `0xAC00 + 4144` = `배`.

함정 두 개

1. `ㅋㅋㅋ`, `ㅠㅠ`**는 건드리지 않는다.** 자음+자음, 모음+모음이라 "자음 다음 모음" 모양이 아닙니다. 인터넷 웃음 표현을 오류로 잡지 않기 위한 설계입니다.
2. **받침 vs 다음 글자의 초성:** `ㅁㅜㄹㅣ`는 `물`+`ㅣ`이 아니라 `무`+`리`입니다. 받침 후보 뒤에 모음이 오면 받침이 아니라 다음 글자의 초성이므로 `(?![{JUNG}])`로 막습니다.

실행 결과

```
'ㅂㅐ고프다'      → ㅂㅐ→['배'][medium]
'ㅋㅋㅋ 웃기다'   → (없음)
'ㅁㅜㄹ을 마셨다' → ㅁㅜㄹ→['물'][medium]
```

신뢰도가 `medium`인 이유: 규칙이 확실하지만 사용자가 일부러 쓴 것일 수 있어서(자모를 쓴 말장난 등) high로는 올리지 않았습니다.

6.2 dictionary.py + dict_rules.json: 사전 규칙 474개

목적: 틀리게 쓰기 쉬운 표기를 목록으로 잡는다. **방식:** 정규식 474개를 하나씩 적용합니다.

규칙 한 개의 모양 (`app/data/dict_rules.json`)

```
{"pattern": "(?<=카드로 )결재",
 "replacement": "결제",
 "category": "spelling",
 "confidence": "high",
 "message": "대금을 치르는 것은 '결제(決濟)'입니다. '결재(決裁)'는 상급자가 안건을 승인하는
        것입니다."}
```

통계(제가 파일을 읽어 센 값): **474개**, 종류별 맞춤법 318 / 문법 151 / 띄어쓰기 5, 신뢰도별 high 443 / medium 23 / low 8.

코드 전체의 골격

```
for rule in _RULES:
    for m in rule["_compiled"].finditer(text):
        suggestions = [m.expand(rule["replacement"])] if rule["replacemen
        t"] else []
        issues.append(RawIssue(start=m.start(), end=m.end(), ...,
                               source="dict_rule", ...))
```

- `finditer` : 글 안에서 패턴이 맞는 자리를 전부 찾습니다.
- `m.expand(replacement)` : 치환 문자열 안의 `\1` 같은 그룹 참조를 채웁니다. 규칙이 "일부는 그대로 두고 일부만 바꾸기"를 할 수 있게 해 줍니다.
- `replacement`가 비어 있으면 제안 없이 지적만 합니다(low 규칙들).

핵심 기법: 문맥을 정규식에 넣기

`결재/결제` 는 글자만 봐서는 못 가릅니다. 그래서 **앞말을 확인하는** `(?<=...)`를 씁니다.

```
카드로 결재했다   → 결제   (카드로 → 대금)
서류를 결재했다   → (없음) (서류를 → 승인, 맞음)
```

실제로 돌려 본 결과입니다. 두 번째 문장은 건드리지 않습니다.

실제 사고: 정문을 고치던 규칙 9개

README에 기록된 사건입니다. 규칙별로 정문 표본에서 발동한 것을 사람이 읽었더니 **정문을 고치고 있던 규칙이 9개**였습니다. 전체 오탐률 0.0009에는 하나도 안 나타났습니다.

규칙	무엇을 고치려 들었나
`들께 → 들게`	"거룩한 스님**들께** 귀의합니다" (`들` + `께` : 높임 조사)
`희안 → 희한`	조선 공신 **성희안** (사람 이름)
`금새 → 금세`	**황금새**벽당 (`황금` + `새벽`)
`역활 → 역할`	**교역활**동 (`교역` + `활동`)

해결: 단어 경계를 정규식에 넣었습니다(`(?<![가-힣])금새` : "금새 앞에 한글이 오면 안 된다"). 이름처럼 글자로 구분이 안 되는 두 개는 신뢰도를 `low`로 내려 자동 적용을 껐습니다.

작은 발견

`금새`를 실행해 보니 같은 자리가 **두 번** 나옵니다(`dict_rule` 2건). `dict_rules.json`에 같은 규칙이 두 줄 들어 있기 때문입니다. 병합(5장)이 겹치는 것을 하나로 줄여 주므로 사용자에게는 영향이 없지만, 파일 정리 후보입니다.

같은 파일 안의 `에요/예요`

`받침 없는 말 + 에요 → 예요`는 받침을 계산해야 해서 정규식 사전에 못 적습니다. 코드(`_eyo_issues`)에서 따로 처리합니다. 예외도 코드에 있습니다.

- `이에요`(받침 있는 말 뒤), `아니에요`는 정문
- 뒤에 문장부호가 오지 않으면 고치지 않음 (위키 실측 62건 중 인명·지명 3건이 문장부호 없이 이어졌음)

6.3 yul_ryul.py: 율/률

규칙: 앞 글자가 모음이거나 ㄴ 받침이면 `율`, 그 밖의 받침이면 `률`.

```
def _expected(prev_char):
    idx = final_index(prev_char)
    if idx is None: return None
    return "율" if idx in (NO_BATCHIM_INDEX, NIEUN_INDEX) else "률"
```

두 관문

관문	어떤 경우	방법
1	단어 끝 `성공율`, `실패률`	Kiwi가 `율/률`을 접미사(XSN)로 읽은 자리만
2	단어 안 `자률신경`	**언어모델 점수**로 판단: 고친 쪽이 Kiwi 점수로 5점 이상 더 자연스러울 때만

2번이 필요한 이유: `자률신경`은 단어 중간이라 Kiwi가 접미사로 읽지 않습니다. 그런데 `율리우스`처럼 **원래 `율`로 시작하는 이름**도 중간 위치로 보일 수 있어서 점수 차이로 가립니다. 주석의 실측: `자률신경 −37 vs 자율신경 −21`(고친 쪽이 16점 좋음), `율리우스 −23 vs 률리우스 −32`(바꾸면 9점 나빠짐).

```
'성공율이 높다'       → 율→['률'][high]
'비률이 높다'         → 률→['율'][high]
'자률신경'            → 률→['율'][high]
'율리우스 카이사르'   → (없음)
```

6.4 josa.py: 조사 (365줄, 가장 복잡한 규칙 검사기)

2장에서 받침 번호로 조사를 고르는 원리를 봤습니다. 이 파일은 그 위에 **"어디가 조사 자리인가"를 찾는 다섯 경로**가 얹혀 있습니다.

왜 다섯 경로인가

Kiwi는 틀린 조사를 조사로 안 읽습니다(3장). 그래서 "조사로 태그된 것"을 찾을 수 없고, 상황마다 다른 길로 조사 자리를 찾아야 합니다.

경로	상황	방법	예
(1)	명사 바로 뒤	명사 토큰이 끝나는 자리의 **다음 글자**를 직접 봄	`학교을`
(2)	조사/접미사에 겹쳐 붙은 보조사	그 단어 끝 글자 기준	`회사에서은`
(3)	괄호 뒤	**괄호 앞** 글자를 숙주로	`몰리에르(Moliere)은`
(4)	닫는 따옴표 뒤	따옴표 **안쪽** 마지막 글자를 숙주로	`《비바리》은`
(5)	조사가 붙어 통째로 명사 하나로 읽힌 어절	언어모델 점수 차이 4점 이상일 때만	`진도은`

핵심 함수들

```
def _check_particle_at(text, end, host_last_char):
    # 로/으로 먼저 (2글자 앞을 봐야 해서)
    expected = _expected_particle(host_last_char, "로", "으로", rieul_except
        ion=True)
    ...
    for no_batchim, has_batchim in [("가","이"), ("를","을"), ("는","은"),
        ("와","과")]:
        expected = _expected_particle(host_last_char, no_batchim, has_batc
        him)
        actual = text[end:end+1]
        if actual in (no_batchim, has_batchim) and actual != expected \
           and _particle_boundary_ok(text, end+1, actual):
            return _issue(...)
```

읽기: 숙주 마지막 글자의 받침으로 “기대하는 조사”를 계산하고, 실제 글자가 그 쌍 중 하나인데 기대와 다르면 오류입니다.

`_particle_boundary_ok` : 조사 뒤는 경계여야 한다

`가수이다` 의 `이` : 바로 앞 `수` 는 받침이 없으니 “가”여야 한다고 오판할 수 있습니다. 그러나 `이` 뒤에 `다` 가 이어집니다. **진짜 조사 뒤에는 공백·문장부호·끝**이 와야 하므로 한글이 바로 오면 조사가 아니라고 봅니다. 예외: `로/으로` 뒤에 `서/써` 가 오는 것(`로서` , `로써`)은 허용.

`_ending_tail_positions` : `뭔가` , `건가` 는 조사가 아니다

`뭔가` 는 `뭐` + `ㄴ가` (어미)입니다. 받침 ㄴ 뒤에 `가` 가 오므로 “받침이 있으니 `이` 여야 한다”고 오판합니다. 실측: 구어 정문 3,000문장에서 나온 high 오탐 25건 중 17건이 이 부류였습니다.

가르는 법이 흥미롭습니다. 태그만으로는 못 가릅니다(진짜 오류 `학생가` 도 `가/VV + 어/EC` 로 읽혀 EC가 걸림). 대신 **어미의 형태**를 봅니다.

```
뭔가   → 어미 형태 'ㄴ가'  첫 글자가 종성 자모(U+11AB) → 앞 음절에 융합 → 조사 아님
학생가 → 어미 '어'         완성형 음절 → 그냥 붙은 조사 자리
```

`fused = "ㄱ" <= head <= "" or "ㄱ" <= head <= "ㅣ"` 가 그 판정입니다: “첫 글자가 종성 자모 범위인가”.

실행 결과

```
'학교을 갔다'                    → 을→['를'][high]
'사과이 맛있다'                  → 이→['가'][high]
'서울으로 갔다'                  → 으로→['로'][high]     ← ㄹ 받침 예외
'물로 씻다'                      → (없음)                ← 맞게 쓴 문장
'가수이다'                       → (없음)                ← 경계 가드
'몰리에르(Moliere)은 작가다'     → 은→['는'][high]       ← 괄호 앞 글자 '르' 기준
'《비바리》은 영화다'            → 은→['는'][high]       ← 따옴표 안 '리' 기준
'진도은 갔다'                    → 은→['는'][high]       ← 경로 (5)
'3를 샀다'                       → (없음)                ← 숫자는 판단 보류
```

경로 (5)의 안전장치

글자만 보고 고치면 위험합니다: `활동가`, `여행가`는 조사가 붙은 꼴과 글자가 같은 **진짜 단어**입니다. 그래서 "고친 쪽이 Kiwi 언어모델에서 4점 이상 더 그럴듯"할 때만 고칩니다. 실측 여유: 진짜 오류 4.8~11.7, 진짜 단어 1.3. 또 `은/는`, `을/를` 네 개로만 한정합니다. 넓히면 외국 이름이 오탐합니다(`도자이 선 → 도자가`).

6.5 dependent_noun.py: 의존명사 띄어쓰기

규칙(맞춤법 제42항): 의존명사는 띄어 씁니다. `먹을것` → `먹을 것`. **의존명사:** 혼자서는 쓰이지 못하고 꾸미는 말 뒤에서만 쓰이는 명사(것, 수, 뿐, 데, 줄, …).

기본 로직

```
if tok.tag != "NNB" or tok.form not in DEPENDENT_NOUNS: continue   # 의존명사 목록
prev = tokens[i-1]
after_determiner = (prev.tag == "MM" and tok.form in {"것","거"} and prev.form not in {"이","그","저",...})
if prev.tag != "ETM" and not after_determiner: continue            # 관형사형 어미 뒤만
if tok.start != prev.start + prev.len: continue                    # 이미 띄어 있음
```

"앞이 관형사형 어미(ETM)일 때만" 확정합니다. 이 한 줄이 `먹을 것`의 `을`(ETM) 뒤 `것`(NNB)을 잡습니다.

단어별 가드 (사연이 하나씩 있다)

단어	가드	이유(코드 주석 근거)
리	`-ㄹ 리 없다/있다` 틀일 때만	`갈릴리` 를 `갈리+ㄹ+리` 로 읽는 오분석
지	시간 경과 틀(`떠난 지 3년`)일 때만	`필요한 것이 없는지` 의 `지` 는 어미
데	뒤에 조사가 붙거나 지배 용언이 올 때만, 쉼표 바로 뒤면 제외	`활동하는데 실내와…` 의 `데` 는 연결어미
뿐	뒤에 `더러` 가 오면 제외	`-ㄹ뿐더러` 는 연결어미
걸	문장 끝이거나 `그렇다/그러다` 가 뒤따르면 제외	`공부할걸 그랬다` 의 `-ㄹ걸` 은 종결어미
이것·그것	`이/그/저` + `것` 이면 제외	대명사 한 단어
차	아예 목록에서 제외	목적(`연구차`)과 기회(`갔던 차에`)가 규칙이 달라 태그로 못 가름

실행 결과

```
'먹을것이 없다'          → 것→[' 것'][high]
'할수 있다'              → 수→[' 수'][high]
'이것을 먹어라'          → (없음)        ← 대명사
'갈릴리에서'             → (없음)        ← 틀 용언 없음
'떠난지 사흘'            → (없음)        ← 틀의 단위 목록에 '사흘'이 없음
'떠난지 3년이 지났다'    → 지→[' 지'][high]
'없는데, 괜찮다'         → (없음)        ← 쉼표
'공부할걸 그랬다'        → (없음)        ← -ㄹ걸 어미
'이런걸 봤다'            → 걸→[' 걸'][high]   ← 구어 '걸'(= 것을)
```

`떠난지 사흘` 을 못 잡는 것은 한계입니다. `_JI_FRAME_NOUNS` 에 `사흘` 이 없기 때문입니다. "정문을 건드리느니 놓친다"는 선택의 결과입니다.

6.6 particle_spacing.py: 조사는 앞말에 붙인다

규칙: 조사는 앞말에 붙여 씁니다. `2006년 부터` → `2006년부터`.

핵심: Kiwi가 떨어져 있어도 `부터`를 조사(`J*` 태그)로 읽어 줍니다. 그래서 **태그가 곧 판정**입니다. `밖에`는 이 방법이 `하나 밖에 없다`(조사 `밖에`, 붙여야 함)와 `문 밖에 서 있다`(명사 `밖` + `에`, 띄는 게 맞음)를 가려 줍니다.

```
'하나 밖에 없다'    →  밖에→['밖에'][high]    (original 에 앞의 공백이 포함됨)
'문 밖에 서 있다'   → (없음)
```

목록은 "동음이의가 없는 조사"만

`밖에, 커녕, 부터, 까지, 처럼, 한테, 에게, 께서, 마다, 조차, 마저, 치고, 더러, 보고서` 등. **일부러 뺀 것:** `보다`(동사 `보다`·부사도 됨), `만`(`삼십 만`의 단위), `로`. 분석이 한 번 틀리면 서로 다른 두 낱말을 붙여 버리기 때문입니다.

한 가지 가드: 격조사끼리는 겹치지 않는다

`에게 해`는 바다 이름(에게해)이라 `에게`가 조사로 읽혀도 붙이면 안 됩니다. "격조사 바로 뒤에 격조사가 오는 모양이면 조사로 읽은 분석이 틀린 것"이라는 문법 사실로 막습니다.

6.7 quotation.py: 간접 인용 `-라고` 겹침

`온다라고 말했다` → `온다고 말했다`. 종결어미 `-다` 뒤에 `-라고`를 또 붙인 겹침입니다.

```python
if tok.form != "라고" or tok.tag != "JKQ": continue      # 인용 조사
prev = tokens[i-1]
if prev.tag != "EF": continue                             # 앞이 종결어미
if prev.start + prev.len != tok.start: continue           # 사이에 아무것도 없음
```

직접 인용과의 구분: `"온다"라고`는 사이에 닫는 따옴표가 있어서 `prev.start + prev.len != tok.start`가 되고 건너뜁니다.

```
'그는 온다라고 말했다'        → 라고→['고'][high]
'그는 "온다"라고 말했다'      → (없음)
'그가 온다고 말했다'          → (없음)
```

6.8 question_mark.py: 물음표

`지금 몇 시야.` → `몇 시야?`. 의문형 종결어미가 있어도 `나야.`(평서문)와 구별해야 합니다.

두 조건 모두 필요합니다. 1. 문장 끝 어미가 의문 어미(`냐, 니, 나, 야, 가, 까, 느냐`) 중 하나이고, 2. **같은 문장 안에 의문사**(`어디, 누구, 몇, 왜, 언제, 어떻게, …`)가 있다(조사가 아닌 것만).

```
'지금 몇 시야.'      → .→['?'][high]
'어디까지 왔니.'     → .→['?'][high]
'나야.'               → (없음)      ← 의문사 없음
```

6.9 honorific.py: 사물 존대

`커피 나오셨습니다` → `나왔습니다`. 높임 `-시-`는 사람에게만 씁니다.

어려운 점

"주어가 사람인가 사물인가"는 어느 품사 태그에도 없습니다. 그래서 **명시적 단어 목록**(`data/animacy.json`의 `things`, `inanimate`)에 있는 명사일 때만 발동합니다. 목록에 없는 명사는 침묵합니다(허용 목록 방식).

로직

1. 토큰이 `시/EP`이거나 종결어미 `세요/으세요`이면 후보.
2. `_honorified_thing`: 가까운 주어(주격 조사가 붙은 명사, 또는 조사 없이 용언 앞에 놓인 명사)를 찾아 그것이 사물 목록에 있는지 확인.
3. 고칠 말은 **그 어절만** 형태소 재조립: `시`를 빼거나, `세요`를 `습니다`로 바꿈.

함정들(코드에 모두 적혀 있음)

상황	처리
`-어 주세요`, `-지 마세요`	사람에게 부탁하는 말이라 높임이 맞다 → 건너뜀
`-ㄹ게요` 안의 `시` (`기다리실게요`)	부탁 형태 `-어 주세요`로 바꿈
`계시다` (말씀이 계시다)	`시` 형태소가 없는 특수 높임말이라 따로: `있으시다`로
`주문하신 커피`	`주문`이 주어가 아니라 파생 동사의 일부라서 → 건드리지 않음
`사장님 말씀이 계시겠습니다`	`님`이 앞에 있어도 가까운 주어(`말씀`)가 대상

```
'커피 나오셨습니다'          → 나오셨습니다→['나왔습니다'][high]
'주문하신 커피 나왔습니다'   → (없음)      ← 정문의 높임은 살린다
'사장님이 오셨습니다'        → (없음)      ← 사람
'품절이세요'                 → 품절이세요→['품절입니다'][high]
```

6.10 animacy_particle.py: 에게 / 에

유정물(사람·동물)에는 `에게`, 무정물에는 `에`를 씁니다. `동생에 선물을 주었다` → `동생에게`.

한계가 곧 설계: Kiwi는 유정/무정을 모릅니다. 네 문장 모두 `NNG + JKB`로 똑같이 분석합니다. 그래서 명사 목록(`animate`, `inanimate`)을 씁니다.

비대칭 설계

방향	조건
`에게` → `에` (무정물 명사 뒤의 `에게`)	명사가 목록에 있으면 **바로**
`에` → `에게` (유정물 명사 뒤의 `에`)	명사가 목록에 있고 **그 문장에 수여 동사(주다 등)가 있을 때만**

두 번째가 더 까다로운 이유: `에`에는 여격 말고도 장소(`학교에 갔다`), `~에 대하여` 등 의미가 많습니다. 주석: "나머지를 하나하나 제외하는 건 끝이 없다(`에 대하여`, `에 준하여`, `에 선정되다`, `에 미치다` …). **교정 쪽을 허가제로 바꿨다.**"

```
'동생에 선물을 주었다'      → 에→['에게'][high]
'꽃에게 물을 주었다'        → 에게→['에'][high]
'고양이에 사료를 줬다'      → 에→['에게'][high]
'학교에 갔다'                → (없음)
```

6.11 negative_polarity.py: 부정 호응

`결코, 전혀, 좀처럼, 도저히, 도무지, 여간, 차마, 그다지, 미처`는 부정과 함께 쓰입니다. `그는 결코 포기할 것이다` → `포기하지 않을 것이다`.

로직 순서

1. 이 부사(MAG)를 찾는다.
2. 문장 끝까지의 구간에 부정 표지(`않, 못, 안, 없, 아니, 말…` 또는 `…없다`로 끝나는 말)가 있으면 정문 → 끝.
3. 허용되는 서술어(`전혀 다르다`, `전혀 새롭다` 등)가 있으면 정문 → 끝.
4. `미처 –기 전에` 틀이면 정문 → 끝.
5. 부사가 지배하는 **첫 서술어**를 찾는다(3어절 이내).
6. 고치기: `–ㄹ 수 있다`는 `있다`→`없다`, 나머지는 서술어 뒤에 `–지 않–`을 붙인다.

```
'그는 결코 포기할 것이다'          → 포기할→['포기하지 않을'][high]
'도저히 이해할 수 있다'             → 있다→['없다'][high]        ← 수 있다는 없다로
'전혀 다르다'                       → (없음)                      ← 허용 서술어
'그는 결코 포기하지 않았다'         → (없음)
```

코드에 남은 사연

`_clause_end`의 주석: Kiwi는 문장 중간에서도 `좋진`을 `좋/VA + 지/EF + ㄴ/JX`처럼 EF(종결어미)로 읽을 때가 있습니다. 그 EF를 문장 끝으로 믿으면 **뒤에 이어지는 부정을 못 보고** 멀쩡한 문장을 부정 없다고 오판합니다. 그래서 "EF 뒤에 문장 끝 부호가 따라올 때(또는 글의 끝일 때)만 진짜 문장 끝"으로 바꿨습니다. (git 기록에도 `fix: stop the polarity rule from reversing three readings` 커밋이 있습니다. 이 주석과 같은 수정인지는 제가 diff를 확인하지 않았습니다.)

6.12 tense.py: 시제 호응

`어제 친구를 만난다` → `만났다`. 시간 부사와 서술어의 시제가 정면으로 어긋날 때만 잡습니다.

모델: 시간 표지 × 서술어 시제

시간 표지	값	예
과거 부사	PAST	어제, 그저께, 방금, 아까, 예전
과거 명사	PAST	작년, 지난주, 지난달
미래	FUTURE	내일, 모레, 내년, 이따가, 다음 주
`N 전`	PAST 또는 **DATED_PAST**	조금 전 / 9만 년 전

서술어 시제	판정
`었/았/였`	PAST
`겠`, `-ㄹ 것이다`	FUTURE
그 외 용언	PRESENT

모순이 되는 쌍만 지적합니다.

```
_CONTRADICTIONS = {(PAST, PRESENT), (PAST, FUTURE), (DATED_PAST, FUTURE),
        (FUTURE, PAST)}
```

`DATED_PAST`(오래된 시점)는 `PRESENT`와 모순이 아닙니다. 백과사전식 글에서 "9만 년 전에 출현한다"처럼 역사적 현재를 쓰기 때문입니다. 코드 주석의 측정 근거입니다.

함정

- **관형절 안의 부사**는 주절 서술어와 무관합니다(`5만 년 전에 시작되는 시대로 연결된다`). 그래서 ETM을 절 경계로 봅니다.
- `다음 주`는 이미 이야기한 시점을 기준으로도 쓰입니다(`그 다음주까지 머물렀다`). `그/이/저` 관형사가 앞에 있거나 `도`가 뒤따르면 판단 안 함.
- `이다`(VCP)는 사건이 없어서 제외(`몇 년 전 SBS인가?` 오탐).

```
'어제 친구를 만난다'    → 만난다→['만났다'][high]
'내일 회의가 열렸다'    → 열렸다→['열린다'][high]
'어제 친구를 만났다'    → (없음)
```

6.13 redundant_causative.py: 불필요한 `-시키다`

`사원을 교육시키다` → `교육하다`. 이미 목적어를 취하는 `-하다`에는 `-시키다`가 군더더기입니다.

어려운 점: `향상시키다`, `변화시키다`, `입학시키다`는 `-하다`가 자동사라 `-시키다`가 필요합니다(`물을 얼음으로 변화시키다`). 태그로는 못 가릅니다. 그래서 **"이미 타동사인 명사 목록"**(교육, 금지, 실현, 주차, 소개, 설득, …)만 씁니다.

```
'사원을 교육시키다'   → 교육시키다→['교육하다'][high]
'물을 변화시키다'     → (없음)
```

6.14 subject_predicate.py: 주술 호응

`내 꿈은 의사가 되고 싶다` → `되는 것이다`. 주어가 추상명사(꿈, 목표, 계획, …)인데 서술어가 사람의 바람(`-고 싶다`)이나 의무(`-어야 하다`)면 주어가 중간에 바뀐 것입니다.

```
'내 꿈은 의사가 되고 싶다'  → 되고 싶다→['되는 것이다'][high]
'꿈은 크다'                  → (없음)
```

`꿈은 크다`는 꿈 자신에 대한 서술이라 건드리지 않습니다. 사람의 태도(`-고 싶다`, `-어야 하다`)만 근거로 삼습니다.

6.15 morph_rules.py: 형태소 배열 보기

문자열 규칙은 활용형을 하나씩 적어야 합니다. 이중피동 규칙 7개가 모두 `~지` 한 형태뿐이어서 `잊혀졌다`, `보여집니다`, `닫혀진`을 전부 놓쳤습니다(2026-09-14 탐침 11/11 미탐). 그래서 **"틀린 형태"가 아니라 "틀린 형태소 배열"**을 봅니다.

```
잊혀졌다 → 잊히/VV + 어/EC + 지/VX + 었/EP + 다/EF
              (피동사 어간) (-어)   (-지다)   ← 이미 피동인데 또 -어지다 = 이중 피동
```

고치는 법: 어간은 그대로 두고 `-어지다`를 빼고 나머지 어미를 `kiwi.join`으로 다시 이어 붙입니다 → `잊혔다`.

6개 규칙

규칙	예	방식
double_passive	잊혀졌다 → 잊혔다	피동사 어간 목록 + `-어지`
ha_contraction	섭섭치 → 섭섭지	안울림 받침 뒤 `하` 통째 탈락(제40항 붙임2)
nonstandard_stem	치루었다 → 치렀다	비표준 어간 → 표준 어간
rieul_eun	거칠은 → 거친	`ㄹ` 받침 + `은` → `ㄴ`
nonstandard_ending	올런지 → 올는지	비표준 어미 표
copula_past	대작이였다 → 대작이었다	`이` + `였` 겹침

```
'잊혀졌다'      → 잊혀졌다→['잊혔다'][high]
'섭섭치 않다'   → 치→['지'][high]
'치루었다'      → 치루었다→['치렀다'][high]
'거칠은 손'     → 거칠은→['거친'][high]
'대작이였다'    → 이였→['이었'][high]
'올런지 모르겠다' → 올런지→['올는지'][high]
```

주목할 설계 판단

- **이중피동 어간은 어휘로 제한합니다.** `-히/이/기/리`로 끝난다는 이유만으로 넣으면 안 됩니다. `밝히다` + `-어지다`(밝혀지다)는 정문이기 때문입니다.
- **`개이다→개다` 규칙은 뺐습니다.** 구어 실측 `정상은 개였다`(`개` + `이었다`)를 날씨 `개다`로 읽었기 때문입니다. "형태만으로는 못 가른다"고 판단했습니다.
- **`ha_contraction`은 단어가 거기서 끝나야 합니다.** `접착치과학`(접착+치과학)을 `접착하+지`로 읽는 오탐이 위키 1건 있었습니다.

6.16 consistency.py: 문체 일관성 (유일하게 글 전체를 보는 검사기)

한 글 안에서 `-습니다`와 `-한다`가 섞였는지 봅니다. **틀린 것이 아니라 통일되지 않은 것**이라서 문장 하나만 보면 전부 정문입니다.

알고리즘 (다수결)

```
1. 문장 단위로 쪼개고, 문장마다 마지막 종결어미(EF)로 문체를 분류
     합쇼체: 습니다, ㅂ니다, 십시오 …    해요체: 어요, 아요, 세요, 죠 …    해라체: 다, ㄴ
다, 었다 …
2. 관용 인사(안녕하세요, 감사합니다 …)는 문체 표지가 아니라서 제외
3. 문체가 2종류 이상이고 다수 문체가 60% 이상이면
4. 소수 문체 문장을 다수 문체로 바꾸자고 제안한다
5. 단, 합쇼체↔해요체는 존댓말 안의 폭이므로 지적만 하고 고쳐 쓰지는 않는다
```

2문장이 1:1이면 다수결이 안 되므로 **첫 문장이 문체를 정한다**는 관례를 쓰되 `low`로 둡니다.

```
'보고서를 제출했습니다. 검토 부탁한다. 확인해 주십시오. 감사합니다.'
  → 부탁한다→['부탁합니다'][medium]
```

(합쇼체 3, 해라체 1이므로 다수는 합쇼체. `감사합니다`는 관용 인사라 표지 제외.)

실제 사고

줄을 손으로 끊어 쓴 글에서 `알려줍니\n다.`처럼 **단어 가운데 줄바꿈**이 있으면 Kiwi가 `알려줍니/NNG + 이/VCP + 다/EF`로 읽습니다. 그대로 문체를 바꾸면 `알려줍니입니다` 같은 **없는 말**이 만들어졌습니다(2026-09-17 실측). 그래서 줄바꿈이 낀 구간은 **제안을 만들지 않고** 지적만 남깁니다.

6.17 이 장에서 가져갈 것

1. 규칙은 **발동 조건 몇 줄 + 예외 수십 줄**로 이루어진다. 예외 하나하나가 실제 오탐 사건이다.
2. 안전을 위해 **"허용 목록"**을 쓴다: 목록에 있는 것만 고치고 나머지는 침묵.
3. 확신이 부족하면 **신뢰도를 low로 내려** 자동 적용만 끈다(탐지는 남긴다).
4. 활용이 얽힌 고침은 **형태소를 고치고 `kiwi.join`으로 다시 조립**한다.
5. Kiwi의 분석 한 번이 틀리면 그 위의 규칙이 틀린다. 그래서 가드는 대개 **"Kiwi가 틀리게 읽는 알려진 모양"**을 막는 것이다.

6.18 확인 문제

1. `결재`를 `결제`로 바꾸는 규칙이 `서류를 결재`를 건드리지 않는 방법은? (`(?<=카드로 )` 같은 앞 문맥 조건)
2. `뭔가`의 `가`를 조사로 오인하지 않는 방법은? (어미 형태가 종성 자모로 시작해 앞 음절에 융합된 것을 확인)
3. `전혀 다르다`를 오류로 잡지 않는 이유는? (`다르` 등 허용 서술어 목록)
4. 이중피동 어간을 `-히/이/기/리`로 끝나는 것 전부로 하지 않은 이유는? (`밝혀지다`처럼 정문이 있어서)

7장. 머신러닝 기초와 혼동쌍 모델

6장의 규칙은 "사람이 조건을 적는" 방식이었습니다. 그런데 `되/돼`처럼 **문맥에 따라 답이 갈리는** 자리는 조건을 다 적기 어렵습니다. 이 장은 "예시를 많이 보여 주고 기계가 스스로 판단 기준을 숫자로 익히게 하는" 방식(머신러닝)과, 다듬이 이것을 어떻게 썼는지를 설명합니다.

> 이 장의 핵심 수치는 제가 배포된 모델 파일(`.ddm`)을 직접 읽어서 손으로 계산한 값입니다. 손계산 결과가 `predict_proba()`와 소수점 넷째 자리까지 일치했습니다.

7.1 규칙 방식과 학습 방식

	규칙 방식	학습 방식
판단 기준	사람이 `if`로 적음	예시에서 기계가 숫자로 익힘
같은 입력	늘 같은 출력	늘 같은 출력 (학습이 끝난 뒤엔 고정)
장점	이유를 설명할 수 있고 예측 가능	조건을 일일이 안 적어도 됨
단점	예외가 끝이 없음	이유가 숫자 더미라서 설명이 어려움, 데이터가 필요

다듬은 **둘을 섞습니다**. 문법으로 답이 정해지는 자리는 규칙이 먼저 확정하고, 규칙이 못 정하는 자리에서만 모델이 발언합니다.

7.2 문제를 "둘 중 하나 고르기"로 바꾸기

`되`와 `돼`는 둘 중 하나입니다. 그래서 문제를 이렇게 바꿉니다.

> 이 문장에서 ▨ 자리에 들어가야 할 것은 `되`인가 `돼`인가?

이런 문제를 **이진 분류**라고 합니다. 입력은 "▨ 자리 주변 글자", 출력은 "`돼`일 확률"입니다.

슬롯과 마스킹 (`app/slots.py`)

슬롯은 "헷갈리는 짝이 나올 수 있는 자리"입니다. 어디가 슬롯인지는 사람이 정규식으로 정합니다.

```
_AN_SLOT    = re.compile(r"(?:(?<=\s)|^)안|않")      # 안: 어절 첫머리일 때만. 않: 항상
_DOE_SLOT   = re.compile(r"[되돼](?!지)")            # 되/돼, 단 바로 뒤에 '지'가 오면 제외
_ROSEO_SLOT = re.compile(r"로[서써]")
_DEN_SLOT   = re.compile(r"(?<=[가-힣])[던든]")
```

왜 이렇게 좁히나:

- `안`은 `안전`, `안내`, `방안` 같은 말 속에 수없이 들어 있습니다. 그래서 어절 **첫머리일 때만** 슬롯으로 봅니다.
- `되지/돼지`는 동물 `돼지`가 코퍼스에 넘쳐서 "`돼` 뒤에 `지`가 온다"는 **정반대 교훈**을 가르칩니다. 그래서 학습과 추론 양쪽에서 아예 뺐습니다. 그 결과 `되지` / `돼지` 혼동은 교정하지 않습니다.

슬롯 주변 글자를 **양옆 12글자씩** 잘라 오고, 슬롯 자리는 `▒`로 가립니다.

```
WINDOW = 12
MARKER = "▒"

def make_feature(text, start, end):
    left  = text[max(0, start - WINDOW) : start]
    right = text[end : end + WINDOW]
    return f"{left}{MARKER}{right}"
```

```
"그렇게 하면 안 돼요."   →   "그렇게 하면 안 ▒요."
```

왜 가리나: 가리지 않으면 모델이 정답 글자 `돼`를 그대로 읽고 "정답은 돼"라고 외우기만 합니다. 가리면 "주변 문맥만으로 판단"하도록 강제됩니다.

실제로 확인한 성질이 있습니다. `안 돼요`와 `안 되요`는 **가린 뒤 똑같은 문자열**(`그렇게 하면 안 ▒요.`)이 됩니다. 그래서 모델의 답도 똑같습니다(둘 다 P(돼)=0.905). 모델은 "쓰인 글자"를 모르고 "문맥"만 보기 때문에, 문맥에 맞는 답은 하나고 쓰인 글자가 그와 다르면 오류라는 구조입니다.

7.3 학습 데이터: 정답이 공짜로 생기는 방법

학습에는 "문제와 정답" 쌍이 많이 필요합니다. 사람이 틀린 문장을 모아 표시하려면 비용이 엄청납니다. 다듬은 **맞게 쓴 문장에서 문제와 정답을 동시에** 만듭니다.

```
원문(위키백과): "그는 의사가 되었다."
   → 슬롯 '되' 발견
   → 문제(특징): "그는 의사가 ▒었다."
   → 정답(라벨): "되"    ← 원문에 쓰인 글자가 곧 정답
```

위키백과 문장은 편집된 글이라 대부분 맞게 쓰였습니다. 그러므로 "쓰인 글자 = 정답"으로 간주합니다. **틀린 문장 코퍼스가 필요 없다**는 것이 이 방식의 가장 큰 이점입니다.

`ml/generate_data.py` 가 하는 일

1. **수집(`fetch`)**: 위키백과 API로 문서 도입부를 무작위로 가져옵니다(20개씩 400번).
2. **검색 수집(`fetch-search`)**: 특정 표현(`term`)이 든 구절을 위키 검색으로 모아 슬롯이 든 문장을 보충합니다.
3. **정제(`_sentences`)**: 마침표 기준으로 문장을 나누고, **길이 10~200자**, **한글이 40% 이상**인 것만 씁니다(수식·외국어 제거).
4. **보강**:
   - `COLLOQUIAL_SEED`: 위키는 격식체라 `안 돼요.`, `가도 돼?` 같은 구어가 적어서, 손으로 쓴 구어 문장 목록을 더합니다.
   - `_augment_doe`: 문장에 `되어`가 있으면 `돼`로 바꾼 변형을 같은 문장 그룹으로 추가합니다(`되어` = `돼`).
5. **슬롯 수확(`build`)**: 문장마다 `(라벨, 문장 그룹, 특징)`을 TSV 파일로 씁니다.

"그룹" 번호가 중요한 이유

```
group = sentence_group(sent)     # 문장의 해시값 = 그 문장에서 나온 모든 슬롯의 공통 번호
```

같은 문장에서 나온 두 슬롯은 주변 12글자가 대부분 겹칩니다. 이 둘이 학습용과 시험용에 갈라져 들어가면, 모델이 "시험 문제를 이미 본 것"과 같아집니다(**누수**). 그래서 그룹 번호를 붙여 **문장 단위로** 나눕니다(7.7).

7.4 특징: 글자 조각 세기 (TF-IDF)

기계는 글자를 못 읽고 숫자만 다룹니다. `그렇게 하면 안 ▒요.`를 숫자로 바꾸는 방법이 **글자 n-gram**입니다.

n-gram은 연속한 n글자 조각입니다. 2~4글자 조각을 전부 뽑습니다(`ngram_range=(2, 4)`).

```
"안 ▒요." 에서
  2글자: "안 ", " ▒", "▒요", "요."
  3글자: "안 ▒", " ▒요", "▒요."
  4글자: "안 ▒요", " ▒요."
```

`char_ngrams(text, 2, 4)`가 이 일을 합니다. 실제 예문 `그렇게 하면 안 ▒요.`에서는 **30개**의 조각이 나왔고 전부 모델 어휘에 있었습니다.

TF-IDF: 조각마다 무게 붙이기

- **TF(Term Frequency):** 그 조각이 이 문장에 몇 번 나왔나(개수).
- **IDF(Inverse Document Frequency):** 그 조각이 전체 학습 데이터에서 **얼마나 드문가**. 드물수록 값이 큽니다.

어디에나 있는 조각(`하는`)은 IDF가 작아 영향이 약하고, 드문 조각일수록 IDF가 커서 판단에 더 크게 반영됩니다. (어느 조각이 실제로 큰 힘을 갖는지는 7.5에서 손계산으로 봅니다.)

```
조각 값 = (개수) × IDF
그 다음 전체를 길이 1로 정규화(L2): 값 ÷ √(값들의 제곱합)
```

정규화를 하는 이유는 문장이 길다고 값이 커지지 않게 하려는 것입니다.

7.5 로지스틱 회귀: 조각마다 투표하기

학습이 끝나면 모든 조각에 **가중치(w)**라는 숫자가 하나씩 붙습니다. 양수면 한쪽 답으로, 음수면 반대쪽으로 미는 힘입니다.

```
z = Σ (조각 값 × 가중치) + 절편
P(클래스 1) = 1 / (1 + e^(-z))      ← 시그모이드: 어떤 z도 0~1 사이 확률로 바꿔 줌
```

- `z`가 0이면 P=0.5, 크면 1에 가깝고, 작으면 0에 가깝습니다.

- **절편**은 “조각을 하나도 못 봤을 때의 기본 치우침”입니다. doe 모델은 −0.0566입니다.

실제 손계산 (doe 모델, 배포된 `.ddm`을 직접 읽음)

`classes_ = ['돼', '되']`이므로 **클래스 1은** `되`입니다. (순서가 중요합니다. 코드에서 `classes.index(...)`로 항상 찾아 씁니다.)

예 ① `그렇게 하면 안 ▒요.`

```
조각 30개 → z = −2.254 → P(되) = 0.0950, P(돼) = 0.9050
되 쪽으로 미는 조각:   ' 하면'(+0.12) '게 '(+0.09) '하면'(+0.08)
돼 쪽으로 미는 조각:   '▒요'(−0.36) ' 안'(−0.27) ' ▒요'(−0.26) '안 '(−0.25)
```

`▒요`(가린 자리 바로 뒤에 `요`)가 가장 강하게 `돼`로 밉니다. “돼요”는 있어도 “되요”는 틀린 표기라는 학습 결과입니다.

예 ② `일이 잘 ▒면 좋겠다.`

```
z = +1.868 → P(되) = 0.8662
되 쪽으로 미는 조각:  '▒면'(+0.73) '▒면 '(+0.48) ' ▒면'(+0.35)
```

`▒면`이 가장 강하게 `되`로 밉니다. 이 손계산 결과가 `model.predict_proba()` 출력 `[0.1338, 0.8662]`과 일치했습니다.

> 이것이 모델의 전부입니다. 거대한 “AI”가 아니라 **조각마다 붙은 숫자표 하나**와 합산 한 번입니다. 어휘 17만 개(doe 모델 175,671개)와 각각의 가중치·IDF가 모델 파일의 전체 내용입니다.

7.6 학습 설정 (`ml/train.py`)

```python
pipe = Pipeline([
    ("tfidf", TfidfVectorizer(analyzer="char", ngram_range=(2, 4),
                              min_df=2, max_features=200_000)),
    ("clf",   LogisticRegression(max_iter=2000, class_weight="balanced")),
])
pipe.fit(X_train, y_train)
```

설정	뜻
`analyzer="char"`	단어가 아니라 글자 단위로 조각을 냄
`ngram_range=(2,4)`	2~4글자 조각
`min_df=2`	학습 데이터에 **2번 미만** 나온 조각은 버림 (우연한 조각 제거)
`max_features=200_000`	어휘 최대 20만 개
`class_weight="balanced"`	한쪽 답이 훨씬 많아도 양쪽을 공평하게 학습 (되가돼보다 많음)
`max_iter=2000`	최대 반복 횟수

빌려 쓴 것: `TfidfVectorizer`, `LogisticRegression`, `Pipeline`은 scikit-learn(sklearn) 라이브러리입니다. 학습 알고리즘은 빌려 오고, **가중치는 이 저장소에서 처음부터 학습한 것**입니다.

7.7 얼마나 믿을까: 시험과 문턱

시험용 분리

```
groups = sorted({g for _, g, _ in rows})
random.Random(SEED).shuffle(groups)                    # SEED = 42
holdout = set(groups[: max(1, int(len(groups) * 0.1))])  # 문장 그룹의 10%
```

문장 그룹 단위로 10%를 시험용으로 떼어 놓습니다. 학습에 한 번도 쓰지 않은 문장입니다.

문턱(threshold)

모델이 "반대 형태일 확률이 0.8"이라고 해도 곧바로 지적하지 않습니다. **문턱 이상일 때만** 지적합니다.

```
P(반대 형태) ≥ 문턱   →   교정 제안
```

문턱이 낮으면 많이 잡지만 멀쩡한 걸 건드리는 일이 늘고, 높으면 안전하지만 놓칩니다.

문턱을 사람이 정하지 않는다

```
FP_BUDGET = 0.005                  # 정문을 잘못 건드리는 비율 0.5% 이하
CANDIDATE_THRESHOLDS = [0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.97, 0.99, 0.995]

for t in CANDIDATE_THRESHOLDS:                # 낮은 값부터
    if t <= min_safe: continue                 # 등록된 회귀 사례 위반 → 건너뜀
    fp_rate, error_recall = _evaluate_threshold(pipe, X_test, y_test, t)
    if fp_rate <= FP_BUDGET:
        chosen = {...}; break                  # 예산을 지키는 가장 낮은 문턱
```

"오탐률 예산을 지키는 가장 낮은 문턱"을 측정이 정합니다.

모델을 아예 안 싣는 조건

```
if len(test) >= MIN_TEST and accuracy >= majority_share + MIN_ACCURACY_MAR
        GIN:
```

- 시험 표본이 100개 미만이면 오탐률 추정이 잡음이므로 **거부**.
- 정확도가 "늘 많은 쪽만 찍는 것"보다 **5%p 이상** 높지 않으면 거부(진짜로 구별을 못 배운 것).
- 어떤 문턱에서도 예산을 못 지키면 거부.

거부된 쌍은 `meta.json`에 `accepted: false`로 기록되고 서버는 그 쌍을 아예 로드하지 않습니다. **"믿을 수 없는 층은 싣지 않는다."**

회귀 사례(forbidden slots)

`eval/qa/regressions.jsonl`에는 "과거에 틀리게 고쳤던 문장"이 등록되어 있습니다. 시험용 균형 데이터로는 이런 특이 사례가 안 보이므로, **문턱이 그 사례들의 반대 확률보다 높아야** 한다는 조건을 문턱 선택에 직접 넣었습니다.

```
min_safe = max(min_safe, proba[1 - classes.index(written)])   # 등록된 '건드리
        면 안 되는' 자리의 반대 확률
...
if t <= min_safe: continue
```

7.8 배포 형식 `.ddm`

학습은 sklearn으로 하지만 **서버에는 sklearn을 설치하지 않습니다.** 대신 학습 결과를 자체 형식으로 내보냅니다.

왜

	피클(`.joblib`)	`.ddm`
모델당 메모리	약 117MB (어휘 20만 개를 파이썬 문자열 객체로 펼침)	5.7MB
sklearn 버전	묶임 (버전이 다르면 읽기 실패 가능)	안 묶임 (순수 데이터)
필요한 패키지	sklearn, numpy, joblib	표준 라이브러리만

구조 (`app/linear_model.py`)

```
[JSON 한 줄: 메타(클래스, n-gram 범위, 절편 등)]
[offsets: 정수 배열 — 각 조각의 위치]
[idf:     실수 배열]
[coef:    실수 배열]
[terms:   정렬된 모든 조각을 UTF-8 바이트로 이어 붙인 덩어리]
```

조각을 찾을 때는 **이진 탐색**(정렬된 목록에서 가운데를 찍어 앞뒤를 가르는 스무고개)을 씁니다.

```
def _index(self, token: bytes) -> int:
    lo, hi = 0, len(offsets) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if terms[offsets[mid] : offsets[mid+1]] < token: lo = mid + 1
        else: hi = mid
    ...
```

어휘가 17만 개여도 약 18번이면 찾습니다.

변환을 믿을 수 있는 이유

수식을 손으로 옮겨 적었으므로 틀릴 수 있습니다. 그래서 `ml/export_models.py`가 **내보낼 때마다 sklearn 원본과 확률을 대조**하고, 차이가 `1e-12`를 넘으면 **파일을 쓰지 않습니다**. 현재 최대 오차는 `5.55e-16`(컴퓨터의 소수 계산 오차 수준)입니다.

7.9 추론: 규칙이 먼저, 모델은 나중에 (`checkers/confusion.py`)

모델 하나를 돌리기 전에 **형태론 게이트**가 먼저 봅니다. 문법으로 정해지는 자리는 확률을 묻지 않고 확정합니다.

```
forced = _decide(text, start, end, written, pair["name"], gates)
if forced is not None:
    if forced != written:
        issues.append(RawIssue(..., confidence="high", ...))
    continue                                  # 모델을 부르지 않음
p_other = ...
if p_other >= threshold:
    issues.append(RawIssue(..., confidence="medium", ...))
```

게이트 일람

쌍	조건	확정	근거
되/돼	뒤에 `고 게 기 니 나 냐 네 는 면 며 자 거 든 어 었 겠 시` 가 옴	`되`	`돼` 는 `되어` 의 준말이라 이런 어미가 바로 붙을 수 없음 (`돼겠` , `돼시` 없음)
되/돼	뒤에 `서 요 야` 가 옴	`돼`	줄어든 꼴에만 붙는 어미
되/돼	어절 끝(뒤가 한글 아님)	`돼`	어간 `되-` 는 어미 없이 못 섬
되/돼	Kiwi가 `되` 를 연결어미(EC)로 읽음	`되`	`적되, 하되` 의 `-되` 는 동사 `되다` 가 아님
안/않	`않` 이 어절 끝	`안`	`않` 은 `않다` 의 어간, 홀로 못 섬
안/않	`-지` 틀 뒤	`않`	`-지 않다` 틀
던/든	어절 끝 `든/EC` 뒤에 명사	`던`	`-든` 은 연결어미라 명사를 못 꾸밈
던/든	`든` 을 ETM(관형사형)으로 읽음	`던`	관형사형 어미는 `-던` 뿐
던/든	이름 속 `든` , 나열 `-든 -든` , 의문사 뒤	`든` (잠금)	비외르든·히든카드, `먹든 말든` , `무엇을 하든`
던/든	`얼마나 ~던지`	`던` (잠금)	회상 감탄
로서/로써	명사형 어미(ETN) 뒤의 `-으로서`	`로써`	행위는 자격이 아니라 수단

> 이 표의 거의 모든 행이 “확률이 자신 있게 틀리는 부류를 문법으로 잠근 것”입니다. 예를 들어 `쓰든 모자의` 를 모델은 `든` 이 맞다고 볼 수 있지만, 명사를 꾸미는 자리에는 `-든` 이 못 오므로 문법이 `던` 을 확정합니다. 실측: 든→던 오류 492건 중 405건(82.3%)을 이 규칙이 잡았고 정문 오탐은 0건.

실제로 돌려 본 결과

```
안/않
  '나는 안 먹었다.'           슬롯 안   P(반대 않)=0.002 → 침묵
  '그는 않 갔다.'             슬롯 않   P(반대 안)=0.998 → 형태론 확정→안
되/돼
  '시간이 되도록 일찍 와.'    슬롯 되   P(반대 돼)=0.192 → 침묵
  '그러면 안 되도 괜찮다.'    슬롯 되   P(반대 돼)=0.798, 문턱 0.6 → 모델: 제안 돼
  '이렇게 해도 되?'           슬롯 되   → 형태론 확정→돼 (어절 끝)
던/든
  '먹든 말든 상관없다.'       슬롯 든   → 형태론 확정→든 (나열 잠금)
  '어릴 때 살든 집이 그립다.' 슬롯 든   → 형태론 확정→던 (관형 수식)
로서/로써
  '칼로서 빵을 잘랐다.'       슬롯 로서 P(반대 로써)=0.517 문턱 0.7 → 침묵(점선 후보)
```

`그러면 안 되도 괜찮다`처럼 **게이트가 못 정하는 자리**(`도`가 `돼도`와 `되도록` 둘 다 가능)에서만 모델이 발언합니다.

7.10 여섯 모델의 성적표 (`meta.json` 실제 값)

모델	학습 행	시험 행	정확도	최다 클래스 비율	문턱	정문 오탐	오류 재현율	실행
an(안/않)	260,639	28,959	99.86%	50.3%	0.5	0.14%	99.9%	✅
doe(되/돼)	35,258	3,917	99.54%	81.4%	0.6	0.15%	99.2%	✅
den(던/든)	13,239	1,470	98.64%	78.2%	0.7	0.41%	94.1%	✅
roseo(로서/로써)	60,464	6,718	87.96%	50.2%	0.7	0.15%	**20.3%**	✅
reo(러/려)	279,787	31,087	98.65%	50.0%	0.8	0.26%	93.7%	⏸ 꺼 둠
dae(데/대)	57,138	6,348	98.03%	50.0%	0.8	0.17%	91.0%	⏸ 꺼 둠

“정확도”와 “최다 클래스 비율”

doe는 정확도 99.54%이지만 `되`가 81.4%라서 “늘 `되`”만 찍어도 81.4%가 나옵니다. 진짜 실력은 그 위의 18%p입니다. roseo는 87.96%인데 최다 비율이 50.2%라 38%p 위에 있어 학습은 됐지만, **문턱 0.7에서의 오류 재현율은 20.3%**에 그칩니다. 이 값은 “모델 혼자 잡는 비율”이고, 실제 엔진의 로서/로써 재현율은 규칙 게이트 덕분에 더 높습니다(평가 0.85, 11장).

reo, dae는 왜 꺼 놓았나

학습 게이트는 넉넉히 통과했습니다. 그러나 **평가할 오류 표본이 거의 없습니다**(구어 오류 1,073행에서 reo 0건, dae 2건). 평가 근거 없이 도는 층은 235MB를 먹으면서 오탐 위험만 지는 “부채”라고 판단해 `_SILENCED_FORMAL = {"dae", "reo"}`로 로드 자체를 건너뜁니다(`DADEUM_PAIRS=dae,reo`로 되살림).

7.11 이 방식의 한계

1. **글자 조각만 봅니다.** 의미를 모릅니다. `칼로서` / `칼로써`는 수단/자격이라는 **의미**의 차이인데 글자 창으로는 잘 안 보입니다(roseo 정확도 88%).
2. **문맥 창이 12글자뿐입니다.** 앞뒤가 멀면 못 봅니다.
3. **학습 데이터가 위키(격식)라서** 구어 표현에 약합니다. 그래서 구어 시드 문장을 손으로 보탰습니다.
4. **어디가 슬롯인지는 사람이 정합니다.** 슬롯 정규식에 안 걸리면 모델은 존재를 모릅니다.

7.12 확인 문제

1. 왜 슬롯 자리를 `▒`로 가리는가? (정답 글자를 그대로 읽어 외우는 것을 막기 위해)
2. 문장 단위로 시험용을 분리하는 이유는? (같은 문장의 슬롯끼리 문맥이 겹쳐 시험 성적이 부풀려지는 누수를 막기 위해)
3. `안 되요`에서 모델이 필요 없는 이유는? (`되` 뒤에 `요`가 오므로 형태론 게이트가 `돼`로 확정)
4. 모델 파일을 sklearn 없이 읽을 수 있게 한 이유는? (배포 이미지 용량·메모리·버전 종속성 때문)

8장. 단어 경계 모델 (gyeol-boundary)

"여러 단어를 붙여 쓴 것"(`하는영화`, `나는매일학교가는버스`)을 어디서 끊을지 정하는 모델입니다. 7장의 방식(글자 조각 + 로지스틱 회귀)을 그대로 쓰지만, **누가 후보를 내고 모델은 무엇을 판정하는가**가 다릅니다. 이 장의 값은 모두 제가 코드를 직접 돌려 얻었습니다.

8.1 풀려는 문제

붙여 쓴 글을 사람이 읽으면 어디서 끊을지 압니다. 컴퓨터는 어렵습니다.

> 하는영화재밌었다 → 하는 영화 재밌었다

Kiwi에는 `kiwi.space(text)`라는 자동 띄어쓰기 기능이 있습니다. 하지만 이 기능은 **한 단어를 억지로 쪼개는** 일이 있습니다.

원문	Kiwi가 제안	옳은가
하는영화	하는 영화	✓
왔다갔다	왔다 갔다	✗ (한 단어)
손병호가	손 병호가	✗ (사람 이름)
도네강과	도네 강과	✗ (강 이름)

규칙으로 막으려면 이름·합성어 목록이 끝이 없습니다. 이름이냐 아니냐는 **문법이 아니라 "그 말을 아느냐"**의 문제입니다. 그래서 "Kiwi가 제안한 자리마다, 진짜 단어 경계인지 학습 모델이 판정"하는 구조를 택했습니다.

8.2 전체 구조

```
check_boundary(text)                              app/checkers/boundary.py
 │
 ├① 후보 찾기   iter_boundary_candidates()        app/boundary.py
 │   문장별로 kiwi.space() → 원문과 비교 → "붙은 덩어리 안에서 Kiwi가 공백을 넣은 자리"
 │
 ├② 갈래 나누기
 │   · 아주 길게 붙인 덩어리(dense) → 모델 없이 Kiwi를 믿고, 희귀 조각 사이만 버림
 │   · 그 밖                              → 모델이 P(띄움)을 계산, 문턱 0.97 이상만
 │
 ├③ 가드      보조용언 허용 / 의문 어미 / 조문 번호 / Kiwi가 고유명사로 읽은 자리
 │
 └④ 결과      high 신뢰도, 제안 = 그 자리에 공백 하나 넣기
```

중요: 후보는 Kiwi만 낼 수 있습니다. 모델은 후보를 **걸러내기만** 하고 새 위치를 만들지 못합니다. 이것은 "모델이 말할 수 있는 범위를 넓히지 않는다"는 안전 설계이기도 하고(오탐이 새로 생길 수 없음), 한계이기도 합니다(Kiwi가 못 본 경계는 영영 못 찾음).

8.3 단계 ①: 후보 찾기 (`app/boundary.py`)

붙은 덩어리: `runon_spans`

```python
_RUN = re.compile(r"[가-힣0-9A-Za-z]+")       # 한글·숫자·영문이 이어진 구간
MIN_RUNON = 3                                  # 3글자 이상

def runon_spans(text):
    return [(m.start(), m.end()) for m in _RUN.finditer(text)
            if m.end() - m.start() >= MIN_RUNON
            and any(_is_hangul(c) for c in m.group())
            and not (m.end() < len(text) and text[m.end()] in _GLOSS_OPENE
        RS)]
```

조건	이유
3글자 이상	2글자 이하는 붙여 쓴 한 단어일 가능성이 압도적
한글이 하나라도 있음	숫자·영문만의 덩어리는 이 모델 대상이 아님
바로 뒤가 괄호면 제외	`민주사회를위한변호사모임(민변)`처럼 **정식 명칭을 일부러 붙여 쓰고** 줄임말을 괄호로 병기하는 한국어 글쓰기 관행

한글만이 아니라 숫자·영문도 같은 덩어리로 봅니다. `동독에서는1937년독일국영철도라는`처럼 숫자가 낀 붙여쓰기가 평가에서 놓치는 것의 5분의 1을 차지했기 때문입니다.

문장 단위로 돌린다

```
for (sentence, base), spaced in zip(sentences, kiwi.space([s for s, _ in s
        entences])):
    if spaced == sentence: continue            # Kiwi가 바꿀 게 없으면 후보 없음
```

문장 하나씩 따로 돌립니다. 같은 문장이 혼자 있을 때와 문서 안에 있을 때 Kiwi가 다른 답을 내는 일이 있었기 때문입니다(`그는또한`이 혼자서는 끊기고 문단 안에서는 안 끊김). 사용자가 예측할 수 없는 판정이라 품질 게이트(P5 불변식)가 막습니다.

후보 = "원문과 Kiwi 결과의 차이 중 공백 삽입"

```
for tag, i1, i2, j1, j2 in diff_opcodes(sentence, spaced):
    if tag != "insert" or i1 != i2 or not spaced[j1:j2].isspace(): continu
        e
    if any(lo < i1 < hi for lo, hi in spans):     # 붙은 덩어리 안쪽이면 후보
        yield sentence, i1, base, is_dense
```

`diff_opcodes`는 두 문자열의 차이를 "삽입/삭제/교체" 목록으로 알려 주는 함수(5장 `base.py`)입니다. "공백만 새로 끼워 넣은 것"만 후보로 삼습니다.

8.4 단계 ②-a: dense(아주 길게 붙인 덩어리) 판별

`medial_closures`: "말을 끝맺고도 계속 이어지는" 횟수

한국어 단어는 조사나 어미로 끝납니다. 긴 **이름**은 조사를 맨 끝에만 달지만, **띄어쓰기를 안 하는 사람**의 글은 중간중간 조사·어미가 있고 그 뒤로 또 내용이 이어집니다.

```
조선민주주의인민공화국의          → 조사 '의'가 맨 끝에만 → 0점
나는매일학교가는버스안에서         → '는'(조사) 뒤에 매일…, '는'(어미) 뒤에 버스… → 2점
```

```
def medial_closures(word, kiwi):
    toks = kiwi.tokenize(word)
    n = 0
    for i, (cur, nxt) in enumerate(zip(toks, toks[1:])):
        tag = cur.tag.split("-")[0]
        if not (tag.startswith("J") or tag.startswith("E")): continue    # 조사·어미가 아니면 패스
        if tag in {"JKG", "ETN"} or nxt가 내용어가 아니면: continue      # 의(속격)·-기(명사형)는 제외
        ...
        n += 1
    return n
```

제외 두 가지: `의`(속격)와 `-기`(명사형)는 이름을 만들 때도 쓰입니다(`사랑의책나누기운동본부`). 어절 첫머리의 1음절 동사(`가재울` → `가/VV + 어 + 재우`)는 Kiwi가 이름을 동사로 잘못 읽은 흔적이라 세지 않습니다.

dense의 정의

```
DENSE_RUNON_CLOSURES = 2      # 2점 이상
DENSE_RUNON_MIN_LEN = 8       # 8글자 이상
dense = hi - lo >= 8 and medial_closures(...) >= 2
```

길이 조건이 붙은 이유: 실측 오탐이 전부 짧았습니다(`나와틀어은`, `호남절의록은`: 5~6음절). "짧은 어절 안의 조사 두 개"는 흔한 형태소 결합이고, 8글자 이상 되어야 "띄어쓰기를 안 하는 사람"의 글로 볼 수 있습니다.

8.5 왜 dense에서는 모델을 안 쓰는가

모델을 돌려 보면 알 수 있습니다. 실제 결과입니다.

```
나는매일학교가는버스안에서 책을 읽었다.      (Kiwi는 다섯 곳을 전부 맞게 띄움)

후보 2   '나는|매일학교가는'         P(띄움)=0.965
후보 4   '나는매일|학교가는버스'     P(띄움)=0.075   ← 진짜 경계인데 거의 0
후보 6   '나는매일학교|가는버스'     P(띄움)=0.276
후보 8   '매일학교가는|버스안에서'   P(띄움)=0.708
후보 10  '학교가는버스|안에서 책을' P(띄움)=0.210
문턱 0.97 → 모델에 맡기면 하나도 못 고침
```

원인은 모델의 한계입니다. 글자 조각(n-gram)을 세는 방식은 "공백이 하나도 없는 문자열 속에서 **이 위치**"를 표현할 수 없습니다. 모든 조각이 주변 70글자의 조각들에 희석됩니다.

해결: **dense 덩어리에서는 모델을 아예 안 쓰고 Kiwi를 믿습니다.** 대신 Kiwi가 알려진 실수를 하는 지점만 막습니다.

8.6 dense에서 버리는 자리: `_keep_dense`

Kiwi는 **모르는 말을 만나면 한 음절씩 쪼갭니다.** 그러면 `학교`가 `핡 교`가 됩니다. 이건 도움이 안 됩니다. `핡교`로 두면 최소한 오타로 읽힙니다.

```
edges  = [lo, *positions, hi]
pieces = [sentence[a:b] for a, b in zip(edges, edges[1:])]
rare   = [not _is_word(p) for p in pieces]          # 사전에서 단어로 보이지 않는 조각
# 양옆 조각이 둘 다 희귀한 경계만 버린다
return {p for i, p in enumerate(positions) if not (rare[i] and rare[i + 1])}
```

조각이 "단어"인지: `_is_word`

```
_PIECE_FLOOR_SYLLABLE = 1000     # 한 음절 조각: 사전에서 1,000번 이상 나와야 단어
_PIECE_FLOOR = 1                 # 두 음절 이상: 한 번이라도 나오면 단어
```

조각	길이	사전 빈도	단어?
수	1	293,000회	✓
때	1	68,000회	✓
교	1	**59회**	✗ (1,000 미만)
핡	1	0회	✗
완주군	3	225회	✓

한 음절은 어디에나 있어서 기준을 높게, 두 음절 이상은 존재만 해도 단어로 봅니다. 실제 결과(어절 사전 181만 어절을 직접 조회):

```
나는매일핡교가는버스안에서 책을 읽었다.
Kiwi:     나는 매일 핡 교 가는 버스 안에서
사전:     핡=0, 교=59 (둘 다 희귀)
결과:     나는 매일 핡교 가는 버스 안에서     ← '핡|교' 경계만 안 끊음
```

(실제 최종 `check_boundary` 출력은 `(2 매), (4 핡), (6 가), (8 버), (10 안)`입니다. `핡`과 `교` 사이 5번 자리는 제안되지 않습니다.)

8.7 단계 ②-b: 모델 판정 (dense가 아닐 때)

특징 만들기

```
WINDOW = 10
MARKER = "▣"

def make_boundary_feature(text, pos):
    left  = text[max(0, pos - WINDOW) : pos]
    right = text[pos : pos + WINDOW]
    return f"{left}{MARKER}{right}"
```

```
"어제 하는영화 재밌었다."  후보 위치 5  →  "어제 하는▣영화 재밌었다."
```

7장의 `▒`는 **정답을 가리는** 표시였습니다. 여기 `▣`는 **"여기가 후보 자리"라고 알려 주는** 표시입니다(끊을지 말지가 정답이므로 가릴 글자가 없습니다).

판정

```
if not dense and proba[space_col] < threshold:     # 문턱 0.97
    continue
```

7장과 같은 방식(2~4글자 n-gram + TF-IDF + 로지스틱 회귀)으로 `P(띄움)`을 구해 0.97 이상일 때만 통과시킵니다.

8.8 단계 ③: 가드

가드	코드	이유
보조용언 허용	`pos in allowed_attached` (dense 갈래)	맞춤법 47항: `도와드리다` 처럼 붙여 써도 맞는 자리
의문 어미 `-ㄴ가`	`_is_question_ending`	`없는가` 의 `가` 를 동사 `가다` 로 읽어 `없는 가` 로 끊는 오류
조문 번호	`_article_number`	`제297조의2` 는 붙여 쓴다 (위키 정문에서 `…의` 뒤 숫자가 붙은 어절 2건은 둘 다 조문 번호)
고유명사	`_proper_noun_spans`	Kiwi가 NNP 한 덩어리로 읽은 안을 가르면 이름이 깨짐 (`손병호` · `도네강` · `해선리` · `낙은재`)

8.9 학습 데이터 만들기 (`ml/boundary_data.py`)

7장과 같은 발상입니다: **정문에서 문제와 정답을 동시에 만든다.**

```
원문(위키):   흐르고 도네강과                  ← 띄어쓰기가 맞는 문장
붙이기:       흐르고도네강과                  ← 일부러 이웃 단어를 붙임
Kiwi 후보:    3번 자리(흐르고|도)  → 원래 이음매였음 → 정답 "space"
              6번 자리(도네|강)    → 원래 한 단어    → 정답 "join"
```

만드는 규칙

- 문장마다 이웃한 **2~3단어를 붙인 곳을 최대 4개** 만듭니다.
- 3어절 이상이고 전부 한글인 문장은 **문장 전체를 붙인 것**도 하나 더 만듭니다 (`_WHOLE_SENTENCE_MIN_WORDS = 3`).
- 붙이지 않은 어절 안에서 Kiwi가 끊자고 한 자리도 **"join"(끊지 마)**으로 씁니다. 위키는 편집된 글이라 그 어절은 원래 맞게 붙어 있다고 보는 것입니다. 이것이 `도네강` · `해선리` 같은 **어려운 오답 사례**입니다.

> 이 "어려운 오답"을 빼 보았더니, 같은 오탐률을 지키는 문턱이 0.97에서 0.995로 올라가서 이 층이 사실상 아무 일도 안 하게 됐다는 기록이 코드에 있습니다.

시험 문제 미리 보기 방지

```
excluded = _eval_sentences()          # 평가에 쓰는 모든 문장
rows = [(s, c) for s, c in rows if s not in excluded]
```

평가셋(`real_errors`, `corpus_suite`)에 있는 문장은 학습에서 **정확히 같은 문장이면 뺍니다.**

구어 코퍼스를 끈 이유

nsmc(영화 리뷰) 6만 2천 문장을 넣어 봤더니 학습 행이 31만에서 79만으로 늘었는데, 같은 문턱에서 **정상 문장을 잘못 끊는 비율이 15배(`.00021 → .00327`)**가 됐고 실사용 재현율은 그대로(126건)였습니다. 구어는 "정문"이라고 해도 두 문장 중 한 문장 꼴로 띄어쓰기가 빠져 있어 "join" 정답이 오염되기 때문입니다. 기본값은 꺼짐입니다.

결과 규모

학습 141,388행, 시험 15,977행. 정답 비율은 `join` 62.7%.

8.10 학습과 문턱 고르기 (`ml/train_boundary.py`)

방식은 7장과 같지만 **예산이 둘**입니다.

예산	값	무엇을 재나
`FP_BUDGET`	0.01	시험 후보 중 "join"을 잘못 띄우는 비율
`SENTENCE_FP_BUDGET`	0.0005	**손대지 않은 정문 문장**(3,169개)에서 한 군데라도 끊기는 문장의 비율

두 번째가 더 중요합니다. 첫 번째만 쓰면 "붙여 놓은 쌍"이라는 쉬운 분포에서 잰 값이라 실제 문장에서 오탐이 세 배가 됐습니다. 그래서 **진짜 정문 문장을 통째로 넣어서** 재는 예산을 따로 둡니다.

`0.0005`는 임의의 숫자가 아닙니다. 제품 전체 오탐 하한이 0.002인데 규칙 층들이 이미 0.0017을 쓰고 있어서, **새 층에 남은 몫**이 이만큼입니다.

문턱 표 (실제 `boundary_meta.json`)

문턱	후보 단위 오탐	진짜 경계 재현	문장 단위 오탐
0.5	17.6%	89.0%	12.5%
0.7	4.7%	74.6%	3.9%
0.8	1.4%	62.0%	1.1%
0.9	0.28%	39.7%	0.28%
0.95	0.10%	18.9%	0.063%
0.97 ← 채택	**0.05%**	**9.3%**	**0.032%**
0.99	0%	1.6%	0%

문턱 0.95는 문장 오탐 0.063%로 예산 0.05%를 넘어서 탈락, 0.97이 처음 두 예산을 모두 지킵니다.

숫자를 읽는 법

- README의 "정확도 86.54%"는 문턱 0.5 근처의 값입니다.
- **실제 쓰는 문턱 0.97에서는 진짜 경계의 9.3%만 모델 혼자 고칩니다.**
- 즉 이 모델은 "많이 잡는 모델"이 아니라 **"아주 확실한 것만 잡고 나머지는 침묵하는 모델"**입니다. 이 프로젝트의 원칙(오탐 공포)이 그대로 드러난 설계입니다.

8.11 실제로 돌려 본 결과

문장	후보와 판정	결과
어제 **하는영화** 재밌었다	후보 5, P=0.995	✓ `하는 영화`
나는매일학교가는버스안에서	dense → Kiwi를 믿음	✓ 다섯 곳 모두 띄움
배우 **손병호가**	Kiwi가 제안 자체를 안 함	✓ 그대로
하루 종일 **왔다갔다**	후보 11, P=0.575	✓ 그대로(한 단어라 정답)
흐르고도네강과	Kiwi 제안 `흐르고도 네 강과`: P=0.674, 0.227	✓ 잘못 끊는 것은 막음 / △ 진짜 경계(`흐르고|도네강과`)는 후보에 없음
돈주고볼영화는	7글자라 dense 아님. P=0.445, 0.815, 0.809	✗ 경계 모델은 전부 놓침 (다른 규칙이 일부를 잡음, 10장)
핡교가는길에 비가왔다	후보 4: P=0.914 등 전부 문턱 미달	✗ 놓침

8.12 한계 (코드에 적힌 실측 기록)

1. **걸러내기 전용입니다.** Kiwi가 제안하지 않은 경계는 찾지 못합니다.
2. **8글자 미만으로 길게 붙인 문장이 사각지대입니다.** dense 기준에 못 미쳐서 모델에 넘어가는데, 모델은 0.97을 거의 못 넘깁니다.
3. **데이터를 늘려도 나아지지 않았습니다.** 5번 재학습하며 위키를 5만 1천 → 9만 8천 문장으로, 구어 6만 2천을 더했지만 실사용 재현율은 제자리(dev 220-227/356, holdout 121-126/204)였습니다. 데이터가 많을수록 문장 오탐 추정이 정확해져 문턱이 올라가고, 얻은 만큼 도로 잃기 때문입니다.
4. **품사 태그를 특징에 넣어도 이득이 없었습니다.** 오프라인 재현율이 0.155 → 0.367로 올랐지만 그 문턱(0.97)에서 정문 12건을 잘못 끊었고(`경기도|리그`, `동음이|의`), 쓸 수 있는 문턱(0.99)에서는 태그 없는 모델과 같았습니다. "오프라인 곡선이 실사용을 따라가지 않는다"는 것이 교훈입니다.
5. **한계는 데이터가 아니라 모델의 종류**에 있습니다. 코드의 기록: 문장 전체를 붙인 샘플을 더해도 `나는매일|학교`의 점수가 0.056 → 0.094밖에 안 올랐습니다. 글자 조각 세기 방식으로는 "공백 없는 긴 문자열 속의 위치"를 표현할 수 없습니다. 샘플은 "다음 모델 종류를 위해 남겨 둔다"고 적혀 있습니다.

8.13 확인 문제

1. 경계 모델이 새로운 경계를 스스로 찾지 못하는 이유는? (후보를 Kiwi만 내고 모델은 판정만 함)
2. `나는매일학교가는버스안에서`는 왜 모델에 안 맡기나? (공백 없는 긴 문자열에서 n-gram 모델이 위치를 구별 못 해서 실제 경계도 P<0.97)
3. 문턱이 0.95가 아니라 0.97인 이유는? (문장 단위 오탐 예산 0.05%를 0.95는 0.063%로 넘음)
4. `학 교`로 끊지 않는 이유는? (양옆 조각이 모두 사전이 거의 모르는 말이라 쪼개기보다 붙여 두는 것이 낫다)

9장. 오타 · 어절 사전 · 언어모델 · 이상탐지

앞 장들은 “알려진 틀린 모양”을 잡았습니다. 이 장은 **사전에 없는 말**, 즉 아무도 목록에 적지 않은 오타를 다룹니다. `매읽`은 `매일`의 틀린 표기가 아니라 “손가락이 ㄱ을 한 번 더 누른 것”일 뿐이라서 어떤 규칙 목록에도 없습니다.

이 장에는 부품이 네 개 나옵니다. 서로 이렇게 얽혀 있습니다.

```
어절 사전(lexicon)  ──▶ 오타 검사(typo)  ◀── 키보드 거리(keystrokes)
        │                       ▲
        └─▶ 이상탐지(anomaly) ─┘ "어느 쪽이 문장에 맞나"
                                │
                  언어모델 gyeol-lm (rescore)
```

이 장의 수치는 제가 직접 돌려 얻은 값입니다.

9.1 어절 사전 (`app/lexicon.py`): 181만 개의 “아는 말” 목록

무엇인가

한국어 위키백과에서 **어절이 몇 번 쓰였는지** 센 표입니다. `count("매일") = 3005`, `count("매읽") = 0`. 사전 크기는 **1,806,050개**입니다.

질문 두 개에 답한다

질문	방법
이 어절이 말인가?	`count(word) > 0` (`knows`)
얼마나 흔한 말인가?	`count(word)` 값 자체

두 번째가 중요합니다. 이름(`발맹`, `모리요`)도 사전에는 몇 번 나옵니다. **오직 횟수만이** “사람이 쓰려던 흔한 말”과 “이름”을 구별합니다.

저장 방식: 해시 + 이진 탐색

181만 개 단어를 파이썬 문자열로 들고 있으면 메모리가 너무 큽니다. 그래서 단어 대신 **64비트 지문(해시)**을 저장합니다.

```python
def eojeol_hash(word):
    digest = hashlib.blake2b(word.encode(), digest_size=8).digest()
    return int.from_bytes(digest, "big") - (1 << 63)
```

파일 구조: `[메타 JSON 한 줄] [해시 배열(정렬됨)] [횟수 배열(같은 순서)]`. 단어를 찾을 때는 해시를 계산하고 **이진 탐색**(정렬된 목록에서 가운데를 찍어 앞뒤를 가르는 스무고개)으로 찾습니다.

```python
def count(self, word):
    h = eojeol_hash(word)
    i = bisect_left(self._hashes, h)
    if i < len(self._hashes) and self._hashes[i] == h:
        return self._counts[i]
    return 0
```

해시의 위험과 안전한 방향

해시는 서로 다른 단어가 같은 지문을 가질 수 있습니다(충돌). 181만 개에서 확률은 약 천만분의 일입니다. **충돌이 나면 "모르는 말을 안다고 착각"합니다.** 그러면 오타를 놓치는 쪽(미탐)으로 틀리지, 멀쩡한 말을 고치는 쪽(오탐)으로 틀리지 않습니다. 이 프로젝트는 이 방향을 일부러 택했습니다.

왜 `lexicon.bin`을 따로 만들었나

원본 통계 파일(`stats_ko.json`, 60MB)은 용량·라이선스 정책상 저장소에 없습니다(`.gitignore`). 그러면 오타 검사는 CI·도커·새 체크아웃에서 **조용히 꺼집니다.** 그래서 오타 검사에 필요한 부분만(해시 + 횟수) 추려 **21.7MB**로 커밋했습니다(`ml/export_lexicon.py`). 쓰기 전에 무작위 2만 개를 다시 읽어 횟수가 같은지 검증합니다.

9.2 키보드 거리 (`app/keystrokes.py`): "손가락이 한 번 헛디딘 것"

왜 글자 거리가 아니라 키 거리인가

```
매읽 ↔ 매일     ㄹ(ㄹ+ㄱ)이 ㄹ로: 키 하나를 더 누른 것   → 손 실수
구촌 ↔ 구몬     ㅊ과 ㅁ: 키보드 양 끝                    → 다른 단어
```

글자 단위(음절)로나 자모 단위로 봐도 두 쌍 모두 “한 군데 다름”입니다. 그러나 사람의 손은 `ㅊ`을 누르려다 `ㅁ`을 누르지 않습니다. 그래서 **자판 위치**를 계산합니다.

자판 배치 (두벌식)

```
_ROWS = (
    "ㅂㅈㄷㄱㅅㅛㅕㅑㅐㅔ",      # 윗줄
    "ㅁㄴㅇㄹㅎㅗㅓㅏㅣ",        # 가운뎃줄
    "ㅋㅌㅊㅍㅠㅜㅡ",            # 아랫줄
)
_SHIFTED = {"ㅃ":"ㅂ", "ㅉ":"ㅈ", "ㄸ":"ㄷ", "ㄲ":"ㄱ", "ㅆ":"ㅅ", "ㅒ":"ㅐ",
        "ㅖ":"ㅔ"}
```

- 각 자모에 `(줄, 칸)` 좌표를 줍니다.
- 쌍자음(`ㅃ` 등)은 같은 키에 Shift를 누른 것이라 같은 위치로 봅니다(Shift를 놓친 것은 실수).
- **이웃**: 줄 차이 ≤ 1 이고 칸 차이 ≤ 1.

글자를 키 입력열로 풀기

```
def keystrokes(word):
    for ch in word:
        initial, vowel, final = decompose(ch)          # 2장의 공식을 거꾸로
        out.append(INITIALS[initial])
        out.extend(_COMPOUND_VOWEL.get(VOWELS[vowel], VOWELS[vowel]))
        if final: out.extend(_COMPOUND_FINAL.get(FINALS[final], FINALS[fin
        al]))
```

겹받침·겹모음은 **두 번 누르는 키**로 풀립니다(`ㄺ` → `ㄹ`, `ㄱ`).

```
매읽  → ㅁ ㅐ ㅇ ㅣ ㄹ ㄱ   (6키)
매일  → ㅁ ㅐ ㅇ ㅣ ㄹ      (5키)
```

한 번의 실수 판정: `one_slip`

```
def one_slip(typed, intended):
    a, b = keystrokes(typed), keystrokes(intended)
    if abs(len(a) - len(b)) > 1: return False          # 키 수가 2개 이상 다르면 아님
    if len(a) == len(b):                                # 같은 길이: 한 키만 달라야, 그것도 이웃 키
        differing = [(x, y) for x, y in zip(a, b) if x != y]
        return len(differing) == 1 and adjacent(*differing[0])
    # 길이가 1 다름: 한 키를 더 눌렀거나 빼먹은 것
    shorter, longer = (a, b) if len(a) < len(b) else (b, a)
    return any(longer[:i] + longer[i+1:] == shorter for i in range(len(longer)))
```

세 가지 실수만 인정: **한 키를 더 누름 / 빼먹음 / 이웃 키를 누름**.

후보 만들기: `near_misses`

단어의 글자 하나에서 초성·중성·종성 중 **하나만** 바꿔 만든 모든 단어입니다(유효한 한글만). `매읽`에서 만든 후보는 **130개**였고, 그중 `one_slip`을 통과한 것은 **54개**입니다. 54개 중에서 다음 단계(빈도 기준)를 통과한 것이 `매일` 하나였습니다.

9.3 오타 검사 (`checkers/typo.py`): 여섯 개의 문

문마다 실측으로 정해진 사연이 있습니다. `매읽 그 영화 봤다.`에서 직접 따라가 봅니다.

단계	문	`매읽`의 결과
1	한글 어절이고 **사전에 없다**	`count("매읽") = 0` ✓
2	**Kiwi도 못 읽었다**(머리를 NNG/NNP로 추측만 함, 뒤에는 조사·어미뿐)	`_guessed_head` = (0, 2) ✓
3	머리가 2~3음절	2음절 ✓
4	**형태소 사전**(gyeol-lm)에서도 모른다	`_known_morpheme` = False ✓
5	**키 한 번 실수**로 닿는 흔한 말(2음절 600회+, 3음절 100회+)	후보 130 → 54 → `매일`(3005회) ✓
6	후보가 **정확히 하나** 남거나 언어 모델이 고름	하나 ✓

문 2의 근거: Kiwi가 읽을 수 있으면 오타가 아니다

`갈래는`은 어절 전체로는 사전에 없지만 `갈래` + `는`으로 분명히 읽힙니다. 그래서 Kiwi가 **"추측했다"**는 흔적(머리가 NNG/NNP 하나이고 뒤에 조사·어미만)이 있을 때만 후보로 삼습니다. 이 문 하나로 정문 오탐이 **18.8% → 3.0%**로 줄었습니다.

문 3의 근거: 1음절은 위험하다

한 음절 문자열은 거의 다 단어이고, 아닌 것은 여러 단어에 한 번씩 닿습니다. 긴 어절은 이름·합성어라 사전에 없다는 사실이 아무것도 말해 주지 않습니다. 그래서 2~3음절만 봅니다.

문 5의 근거: 이름도 "아는 말"이다

`발맹`, `모리요`, `허준로`도 몇 번씩 나옵니다. 오직 횟수만이 "사람이 쓰려던 말"과 "이름"을 가릅니다. 2음절은 더 많은 이웃과 충돌하기 쉬워 **600회**, 3음절은 **100회**를 기준으로 합니다. 코드 주석의 측정: 3음절을 2음절 기준(600)에 맞추면 복원할 수 있는 것의 3분의 1을 잃는데(`완주군` 225회, `모나코`), 정문 점선 밑줄 감소는 0.3%p뿐이었습니다.

9.4 이 오타를 "고칠" 것인가 "물어볼" 것인가

여섯 개의 문을 모두 통과해도 대부분은 **low(점선)**로만 냅니다. 왜냐하면:

> 이름은 끝이 없습니다. `딕터`, `라나카`, `하찌`, `항가리`는 모두 사전에 없고 모두 흔한 말에서 한 번 실수한 모양입니다. 정문에서 측정하면 0.35%(위키), 0.37%(구어)가 걸리는데 이는 자동 적용 예산의 두 자릿수 배입니다.

그러나 **조건이 모두 맞으면 high로 올립니다** (코드의 `sure`).

```
_CONFIDENT_GAP = 10.0
sure = gap >= _CONFIDENT_GAP and _is_one_off_common_noun(text, tokens, hea
        d_start, head_end, word)
confidence = "high" if sure else "low"
```

조건	뜻
`gap ≥ 10`	언어모델이 "고친 쪽이 10점 이상 더 자연스럽다"
`_is_one_off_common_noun`	Kiwi가 NNG 하나로 읽었고, **글 안에서 한 번만** 나옴

한 번만 나와야 하는 이유

코드 주석: 정문에서 high로 오탐한 사례가 모두 세 가지 모양이었고, 모델 없이도 보였습니다.

모양	예	왜 위험
합성어	`학술직` → Kiwi가 `학술` + `직` 으로 읽음	모르는 말이 아니라 합성어
고유명사 추측	`켐퍼스` → NNP	Kiwi의 추측이 "이름"
같은 글에서 두 번 이상	`주허`	**오타는 한 번뿐이고, 의도한 이름은 되돌아온다**

실제 결과

```
'매읽 그 영화 봤다.'     → 매읽→['매일'] high   (margin 11.73, 한 번뿐인 NNG)
긴 예문 속 '매읽'         → low                    (margin 8.03 < 10)
'나는 매읽 운동한다.'    → 지적 없음(typo는 침묵, anomaly 점선만)
```

세 번째가 **한계의 실제 사례**입니다. Kiwi가 `매읽`을 `매/NNG + 읽/VV`로 "분석에 성공"해 버려서 문 2(Kiwi가 추측했는가)를 통과하지 못합니다.

9.5 언어모델 gyeol-lm (`app/ngram_lm.py`): "어느 쪽이 더 한국어다운가"

무엇을 하는 모델인가

두 가지 쓰기 중 어느 쪽이 더 자연스러운 한국어인지 **점수**를 냅니다. 고칠 말을 만들지는 않습니다. 점수가 높을수록 자연스럽습니다.

```
나는 학교에 갔다.    -27.61
나는 학교을 갔다.    -35.50      ← 7.89점 낮음
```

어떻게: 세 개씩 묶어 세기 (3-gram)

위키 문장을 형태소로 쪼개서 **연속한 세 형태소가 몇 번 나왔는지** 셉니다.

```
"나는 학교에 갔다."   →   ['<s>', '_나', '는', '_학교', '에', '_가', '_었', '다',
'.']
                          3-gram:  <s> _나 는  /  _나 는 _학교  /  는 _학교 에
...
```

- `<s>`는 문장 시작 표시.
- `_`**(밑줄 막대)는 어절의 첫 형태소 표시**입니다. 이게 없으면 `영화보고`와 `영화 보고`가 같은 형태소 세 개라서 구별이 안 됩니다.

실제로 확인한 규모: 1-gram 1,222,405개 / 2-gram 1,224,952개 / 3-gram 534,188개, 전체 토큰 143,404,308개. 파일 36MB.

점수 계산: 멍청한 백오프(stupid backoff)

"앞의 두 형태소가 이랬을 때 다음이 이것일 확률"을 로그로 합산합니다.

```
P(고 | _영화 _보) = count(_영화 _보 고) / count(_영화 _보) = 280 / 2034 = 0.1377
log P = -1.983          ← 실제 모델 출력과 일치
```

세 개 조합이 학습 데이터에 없으면 **한 단계씩 낮춰서** 보되, 단계마다 점수에 `log 0.4`(=−0.916)를 깎습니다.

```
P(c | a b) 가 없으면  →  0.4 × P(c | b)
그것도 없으면         →  0.4 × 0.4 × (c의 단독 빈도, +1 보정)
```

실제로 학습에 없는 조합(`_영화 _뛟 고`)의 점수는 −6.45입니다(−1.83은 두 번 깎은 값).

왜 "멍청한"인가: 확률이 합이 1이 되도록 정교하게 정규화하지 않기 때문입니다. 이 용도에서는 **두 후보의 순서**만 필요해서 가장 싼 방식으로 충분합니다. 논문 기준값 0.4를 그대로 썼습니다.

두 후보 비교: `margin` (`app/rescore.py`)

```
def margin(text, start, end, replacement, kiwi):
    lo, hi = _window(text, start, end)              # 고친 자리 양옆 20글자(어
        절 경계로 넓힘)
    before = text[lo:hi]
    after  = text[lo:start] + replacement + text[end:hi]
    return model.score(morphemes(after, kiwi)) - model.score(morphemes(bef
        ore, kiwi))
```

양수면 "고친 쪽이 더 자연스럽다". 양옆 20글자만 보는 이유: **문서 반대쪽 문장 때문에 값이 달라지면 안 되기 때문**입니다.

중요한 제한: 만들지 않고 고르기만

모듈 설명(`rescore.py`)에 이렇게 적혀 있습니다.

> 생성하는 언어모델은 이 엔진이 말하는 범위를 넓히고, 그 모델이 지어낸 오탐은 나머지 엔진이 맞춰 둔 측정 밖에 있게 된다. 그래서 이 모델은 **절대 제안하지 않는다.** 검사기가 규칙으로 후보를 만들면, 이 모듈은 순위만 매긴다. 지적할 수 있는 것은 줄거나 순서가 바뀔 뿐 **늘지 않는다.**

시도했다가 안 된 용도: 띄어쓰기 심판

"`영화보고`가 `영화 보고`보다 낮다"는 신호가 실제로 있습니다(위 예: −21.34 vs −21.02). 그러나 사람이 라벨링한 16개 편집에서 재 보니 띄어야 할 8개(−19.7~+2.0)와 띄우면 안 되는 8개(−17.8~+3.1)의 값이 **완전히 섞였습니다**. 신호(0.46점)가 문장을 다시 분석할 때 생기는 변동에 묻힙니다. 그래서 띄어쓰기 판정은 8장의 경계 모델(판별 학습)이 계속 맡습니다.

모델 크기의 판단

학습 코퍼스: 위키 약 450만 문장 + 구어(nsmc) 문장을 8배 반복. 가지치기(횟수가 기준 미만인 조각 삭제): `{1:1, 2:8, 3:20}`. 36MB. 기준을 `{2, 3}`으로 풀면 127MB가 되고 오타 벤치마크가 11.9% → 12.4%로 오를 뿐이라(531문장 중 약 한 문장 반) 90MB를 더 싣지 않았습니다.

알아 둘 점

`_`(어절 첫머리 표시)를 붙이는 조건은 `token.start가 0이거나 앞 글자가 공백`입니다. 그런데 `갔`처럼 한 글자에 두 형태소가 녹아 있으면 둘 다 같은 `start`를 가져서 **둘 다 `_`이 붙습니다**(위 예: `_가 _었`). 학습과 추론이 같은 함수를 쓰므로 일관되지만, 읽으면 의아해 보이는 부분입니다.

9.6 이상탐지 (`app/anomaly.py`): "이 어절은 낯설다"

질문

사전에도 없고(`known=False`), 글자가 이어지는 모양도 한국어답지 않은 어절이 있나? **고칠 말은 만들지 못합니다.** "여기가 수상하다"만 말하며 항상 low(점선)입니다.

점수 공식

어절 앞뒤에 시작 `^`, 끝 `$`를 붙이고, 각 글자가 **앞 두 글자 다음에 올 확률**을 구해 `-log P`가 **가장 큰 글자**를 그 어절의 점수로 삼습니다.

```
score(어절) = max_i  -log P( c_i | c_{i-2}, c_{i-1} )
```

왜 평균이 아니라 최댓값인가

긴 어절에 이상한 이음매가 딱 한 군데 있으면 평균이 그것을 희석합니다. 코드 주석: 실측에서 평균 기준은 오문이 정문보다 오히려 낮게 나왔고, 최댓값으로 바꾸자 분리가 생겼습니다.

확률: 세 종류를 섞기 (선형 보간)

```
P = 0.70 × P(3-gram) + 0.25 × P(2-gram) + 0.05 × P(1-gram)
```

3글자 조합이 신호의 중심이고 1글자 확률은 "0이 되지 않도록" 바닥을 깔아 줍니다. 단일 모델보다 같은 오탐률에서 탐지율이 6.8% → 10.5%로 올랐다고 주석에 있습니다.

판정

```
def detect(self, text, threshold):
    for word in text.split(" "):
        if self.is_known(word): continue          # 사전에 있으면 건너뜀
        score = self.word_score(word)
        if score > threshold: ...                 # THRESHOLD = 11.25
```

실제 점수

어절	사전에 있나	점수	결과
학교를	✓	2.49	—
친구가	✓	2.56	—
독재자	✓	4.02	—
ㅋㅋ	✓	0.00	—
핽교	✗	7.81	임계값 아래, 침묵
매읽	✗	**12.44**	**점선**
만낫는데	✗	**12.82**	**점선**
뿱뿱	✗	19.42	점선

임계값 11.25는 누가 정했나

“오탐 예산 0.5%”를 지키는 값으로 평가 게이트(`eval/detect_eval.py`)의 스윕이 정한 **운영값**입니다. 낮추면 탐지율은 오르지만 예산을 넘습니다. 값을 바꾸면 게이트도 다시 재야 합니다.

메모리 문제와 해결 (사연)

통계를 파이썬 `dict`로 들면 메모리가 **581MB 늘었습니다**(Kiwi만 380MB → 961MB). 항목이 350만 개이고 각각이 객체를 잡기 때문입니다. 서버 한 워커 예산이 300~500MB인데 그 3배라서 이 층은 **탐지율 4.3% → 23.6%까지 올려 놓고도 서버에 못 붙이고** 있었습니다.

조치	내용
어절 사전	값이 필요 없으므로 해시 배열 하나로
n-gram	글자 종류가 약 3천 개뿐이라 `3-gram` 도 $V^3 < 2^{63}$. **글자 조합을 정수 하나로 인코딩**한 뒤 정렬 배열 + 이진 탐색
JSON 파싱 봉우리	상주는 961→278MB로 줄었지만, JSON을 읽는 순간 봉우리가 741MB. 첫 실행 때 배열을 그대로 `.compact` 파일에 붓고 이후엔 그 파일만 읽음 (원본의 수정 시각으로 무효화)

결과: **+54MB, 로드 0.02초**로 붙였습니다.

> 이 파일(`stats_ko.json` , `.compact`)은 저장소에 없습니다. 새로 받은 환경에서는 이 층이 조용히 꺼져 있습니다(`/api/health` 에서 확인).

9.7 시도했다가 접은 것

코드에 "Tried and rejected"로 남아 있는 것들입니다. **무엇을 안 했는지가 무엇을 했는지만큼 중요합니다.**

겹친 음절을 오타로 보기 (`감감독` , `유럽럽`)

퍼징 도구가 중복 음절 오류의 3%만 탐지하는 것은 사실이라 시도했습니다. 그러나 24,766개 정문 중 15번 발동해 **13번이 틀렸습니다**: `한한다고` (한하-+ㄴ다고), `주초석석재` , `발기인인` , `출시시` , `레이다다` , `터터` , `살라라` . 한국어는 형태소 경계에서 같은 음절이 붙는 일이 흔합니다. 결론: "음절을 하나 빼려면 **모양이 아니라 단어를 알아야** 한다."

이상탐지 평균 점수, 단일 add-α 스무딩

위에서 설명한 대로 둘 다 실측으로 최댓값·선형 보간에 졌습니다.

9.8 확인 문제

1. `구촌`을 `구몬`으로 고치지 않는 이유는? (`ㅊ`과 `ㅁ`은 자판 양 끝이라 손 실수로 보기 어려워서 `one_slip`이 False)
2. 언어모델이 후보를 만들지 못하게 한 이유는? (엔진이 말하는 범위가 넓어지면 오탐이 측정 밖에서 생김)
3. 이상탐지가 평균 대신 최댓값을 쓰는 이유는? (긴 어절의 이상한 이음매 한 곳을 평균이 희석)
4. 같은 글에서 두 번 나온 낯선 말이 high 교정에서 빠지는 이유는? (오타는 한 번뿐이고 의도한 이름은 되돌아옴)

10장. 띄어쓰기: 가장 큰 검사 영역

띄어쓰기는 한국어 맞춤법에서 **오류가 가장 많고 동시에 허용이 가장 많은** 영역입니다. "맞는 걸 틀렸다고 하는" 위험이 가장 큰 곳이라서 다듬 코드의 띄어쓰기 부분은 가장 조심스럽고 가장 큽니다 (`spacing.py` 758줄 + `spacing_extra.py` 922줄 + `spacing_tiers.py` 103줄).

이 장의 실행 결과는 제가 `check_spacing`을 직접 불러 얻은 값입니다.

10.1 문제의 모양

띄어쓰기 오류는 두 방향입니다.

방향	예	처리
붙여 써야 하는데 띄어 씀 (삭제)	`안녕  하세요` → `안녕하세요`	공백을 지움
띄어 써야 하는데 붙여 씀 (삽입)	`너무많아서` → `너무  많아서`	공백을 넣음

그리고 **둘 다 맞는 자리**가 많습니다(맞춤법 제47·48·49항: 보조용언, 전문 용어, 고유명사). 이것이 가장 어려운 점입니다. 모델이 "이쪽이 더 흔하다"고 해서 고쳐 버리면 **맞는 글을 틀렸다고 하는** 도구가 됩니다.

10.2 3단계 판정: 필수 / 권장 / 허용 (`spacing_tiers.py`)

설계 원칙이 이 파일 맨 위에 있습니다.

단계	뜻	동작	신뢰도
필수	확정 오류	자동 교정	high
권장	근거는 있으나 확정 못 함	제안만	low (점선)
허용	규범이 둘 다 허용하거나 근거가 약함	**침묵**	—

> "허용 범위를 넓게 잡는다." 애매하면 전부 '허용'으로 보낸다. 침묵이 기본값이다.

```
ADVISE_MIN = 0.90            # 모델 확률이 이 위여야 '권장'으로라도 올라간다
MODEL_NEVER_AUTOFIXES = True # 모델 단독으로는 자동 교정 권한을 주지 않는다
```

"필수는 모델이 정하지 않는다." 자동 교정(high)은 오직 **규칙이 정한 자리**에서만 나옵니다. 모델(태거)의 말은 아무리 확신해도 low(점선)까지만 올라갑니다.

10.3 전체 구조: 네 갈래가 한 목록으로

`check_spacing(text, kiwi)`가 네 가지 길을 모아서 정리합니다.

```
                    ┌─ ① 삽입  _insertions   BiGRU 태거 (확률 ≥ 0.90, 한글–한글 경계)   → low
check_spacing ──────┼─ ② 삭제  _deletions    kiwi.space() 가 지운 공백                  → medium
                    ├─ ③ 규칙(spacing.py)    정도부사·부정부사·보조용언·시간명사·수+단위·명사+있다/없다·관형사+명사·부사+용언·조사·접미사
                    └─ ④ 규칙(spacing_extra) 문장부호·합성어·접두사·서술격조사·어미·연어·경계 판별·명사형·합성용언 … 20여 종   → high
```

합치는 법 (코드 끝부분)

```
taken = set()
uniq = []
for h in rule_hits:                          # 규칙 결과 중 같은 자리는 하나만
    if (h.start, h.end) in taken: continue
    taken.add((h.start, h.end)); uniq.append(h)
return uniq + [i for i in out if (i.start, i.end) not in taken]    # 태거·삭제는 규칙이 안 차지한 자리만
```

규칙이 확정한 자리는 태거의 저신뢰 제안보다 우선합니다. 같은 자리에 두 개가 뜨면 중복 밑줄로 보이기 때문입니다.

10.4 ① 삽입: BiGRU 태거

BiGRU가 무엇인가

GRU는 글을 앞에서 뒤로 읽으며 "지금까지 본 내용"을 기억에 쌓아 가는 작은 신경망입니다. **Bi**는 양방향이라는 뜻으로, 앞→뒤와 뒤→앞을 둘 다 읽어 합칩니다.

```
입력:  하 는 영 화 재 밌 었 다        (공백을 전부 지운 글자열)
출력:  0.1 0.97 0.1 0.8 0.1 0.1 0.1 0     ← 각 글자 "뒤에 공백이 올 확률"
```

구조(`ml/spacing_model_nn.py`):

```python
class SpacingTagger(nn.Module):
    def __init__(self, vocab, emb=64, hidden=192, layers=2, dropout=0.2):
        self.emb = nn.Embedding(vocab, emb)                    # 글자 → 64
        개 숫자
        self.rnn = nn.GRU(emb, hidden, num_layers=2, bidirectional=True)
        self.out = nn.Linear(hidden * 2, 1)                    # 글자마다 점
        수 1개
    def forward(self, x):
        h, _ = self.rnn(self.emb(x))
        return self.out(self.drop(h)).squeeze(-1)
```

왜 BiGRU인가 (코드 설명): 띄어쓰기 판단은 **양방향 국소 문맥**이 거의 전부입니다. `할수`를 띄울지는 앞의 관형형과 뒤의 `있다`를 같이 보면 정해집니다. 장거리 의존은 거의 필요 없어서 작은 모델로도 충분하다고 보았습니다. 사전학습 모델(RoBERTa 등)을 미세조정하는 방식과 정면으로 비교하기 위해 만든 것입니다.

추론할 때: 공백을 일부러 전부 지운다

```python
stripped_chars = [ch for ch in text if not ch.isspace()]
probs = tagger.predict_proba("".join(stripped_chars))
```

학습할 때 "공백을 지운 문장 전체"를 입력으로 했으므로 추론에서도 똑같이 넣습니다. 어절을 하나씩 따로 넣으면 문맥이 사라져 판정이 나빠집니다. 그런 다음 원문의 **어절 내부 경계만** 봅니다. 이미 공백이 있는 자리는 건드리지 않습니다(삭제는 ②의 일).

통과해야 하는 문들

```python
if probs[k] < ADVISE_MIN: continue             # 0.90 미만이면 침묵
if i1 != orig_index[k + 1]: continue           # 원문에 이미 공백이 있음
if not (_is_hangul(text[i1-1]) and _is_hangul(text[i1])): continue   # 한글–
        한글 경계만
if i1 in aux_starts: continue                  # 47항 허용형이면 침묵
if i1 in inside_morph: continue                # 형태소 안쪽이면 불가능
```

문	이유
한글-한글만	숫자·영문·부호 옆 분리(`17일 ~ )`)가 정문 오탐의 최대 부류였음
형태소 안쪽 제외	태거는 위키(격식)로 학습해서 구어 어미를 모름. `도와드릴게요` 의 `ㄹ게요` 를 `…ㄹ게 | 요` 로 쪼개려 한 적이 있음
`aux_starts`	보조용언이 `-아/-어` 나 관형사형 뒤에 있으면 붙여 써도 맞음

태거의 한계 (코드 설명에 명시)

> 태거를 "처음부터 다시 띄우는" 과제로 재면 **문장 완전일치가 44.8%**라서, 절반 이상의 문장에서 최소 한 군데를 틀린다. 그대로 자동 교정에 쓰면 안 된다.

F1 점수 0.945는 "경계 하나하나"의 점수이고, **문장 전체를 다 맞히는 비율은 44.8%**입니다. 그래서 low(점선) 이상은 못 올라갑니다.

환경에 따라 다릅니다

이 태거는 torch(딥러닝 라이브러리)가 있어야 읽힙니다. 배포 서버에는 torch를 설치하지 않으므로 **배포본에서는 이 층이 빠집니다.**

```
try:
    from ml.spacing_model_nn import load_tagger
except ImportError:
    return None       # torch 가 없으면 이 층만 조용히 빠진다
```

`/api/health` 의 `spacing_tagger` 가 `false` 면 꺼진 것입니다. "내 컴퓨터에선 되는데 서버에선 안 돼"의 주요 원인입니다(실제로 CI가 이 때문에 죽은 적이 있습니다. 모델 파일이 없어서 import에서 예외가 나 파이프라인 전체가 실패했고, 지금은 `None` 을 돌려주도록 고쳐졌습니다).

10.5 ② 삭제: Kiwi의 `space()`

`kiwi.space(sentence)` 가 돌려준 문장과 원문을 비교해서 **공백이 지워진 자리만** 봅니다.

```
for tag, i1, i2, j1, j2 in ops:
    if tag != "delete" or not text[i1:i2].isspace(): continue      # 공백만 지
        운 것
    if i1 in protected: continue                                    # 보조용언
        허용 자리
    if not (한글 양옆): continue
    → source="kiwi_space", confidence="medium", message="띄어쓰기를 확인하세요."
```

문장 하나씩 따로 돌립니다(`_sentences`). `kiwi.space()`가 건네받은 문자열 전체를 읽으므로 같은 문장이 혼자일 때와 문서 안에 있을 때 답이 달랐기 때문입니다(`가리킨 다`가 문서 안에서는 사라짐). P5 불변식이 이를 지킵니다.

공백 이동 문제

원문 `검토 해보다`가 `검토해 보다`로 바뀌는 경우 diff는 "한 글자 삽입 + 한 글자 삭제"로 나옵니다. **공백만 지운 순수 삭제만 받는** 조건이라 이런 "공백 이동"은 절반이 음절이라 걸러집니다. 이 한 줄 조건이 코드 주석에서 "위험을 녹여 없앤다"고 표현됩니다.

10.6 ③ 규칙 (`spacing.py`)

여기서부터 모두 **high**입니다. 각 규칙은 "위키 정문 10,000문장에 먼저 걸어 보고 형태를 좁혔다"는 방식으로 만들어졌습니다.

규칙	근거 조항	예	핵심 조건
`_degree_adverb_rule`	제2항	`너무많아서` → `너무 많아서`	정도부사 목록(너무·매우·아주·몹시·굉장히·무척·상당히·꽤·되게·하도·약간·훨씬·더욱)만
`_aux_verb_rule`	제47항	`하고있다` → `하고 있다`	`-지/-게/-고/-어야` 뒤의 보조용언만 (-아/-어 뒤는 허용이라 제외)
`_negation_rule`	제2항	`안되요` → `안 되요`	`안·못` + 용언. 굳은 말(`못하다` , `안되다`) 제외
`_time_noun_rule`	제42항	`갈때` → `갈 때`	앞말 종류마다 따로 좁힘
`_numeral_unit_rule`	제43항	`세번째` → `세 번째`	수 관형사 + 단위 명사
`_noun_exist_rule`	제2항	`재미 없다` ↔ 붙임	명사 + 있다/없다/같다
`_determiner_noun_rule`	제2항	`이런영화` → `이런 영화`	관형사 + 명사
`_adverb_verb_rule`	제2항	부사 + 용언	굳은 부사 제외
`_josa_space_rule`	제41항	`친구 와` → `친구와`	조사는 앞말에 붙임
`_suffix_space_rule`	—	`안녕 하세요` → `안녕하세요`	파생 접미사 -하다·-되다·-시키다

규칙마다 이어진 “빼기” 목록의 실제 예

`_negation_rule` (안/못 + 용언):

빼는 것	이유 (코드 주석의 실측)
`못하다`, `못되다`, `안되다`	위키 `못하다` 13건이 전부 정문. `못되다`(성품)·`안되다`(가엾다)는 뜻으로만 갈려 글자로 못 가름
앞말에 붙은 `안` / `못`	`편안`, `잘못`의 일부일 수 있음. 조사 뒤(`B급도안`)만 받음
《》·〈〉·『』·「」 안	작품 제목은 만든 이가 쓴 그대로가 이름. `《짱구는 못말려》`가 오탐이었음

`_time_noun_rule` (때/동안/전/후/중/이상/이하 …): 앞말 종류에 따라 규칙이 7갈래입니다. 코드 주석:

앞말	허용	근거
관형형	때·동안·내내·무렵·즈음	`끝날 때`
명사	`때문`	의존명사
수량	이상·이하	명사 뒤는 `정신이상` 합성어
명사형(-기)·시간 단위	전·후	**명사 뒤 `전·후`는 위키 9+9건이 거의 전부 오탐**: 격투전(戰)·개막전·기원전·얼후(악기)·곽상훈(이름)
두 음절 이상 일반명사	때·중	위키 `중` 39건 중 오탐 11건(`혈중`, `작중`, `초중`, `윤희중`)을 이 조건이 걸러 냄

시도하고 버린 것도 기록돼 있습니다: `영상속에`, `환경속에도` 같은 `속`. 생물 분류 `속(屬)`(마타리속, 스콜로플락스속)은 붙여 쓰는데, 앞뒤 어디에도 둘을 구별할 신호가 없고 "그 단어를 아는지"만이 구별합니다.

허용 형태: "지켜야한다"의 숨은 불일치

실행해 보았습니다.

```
'도와드리다'   → (없음)                                  ← 본용언 + -아/-어 + 보조용언: 허용
'지켜야한다'   → [aux_verb_rule|high] '한' → ' 한'        ← 규칙은 띄우라고 함
```

`_aux_verb_rule`의 설명대로 `-어야` 뒤는 47항의 붙여 쓰기 허용 목록(`-아/-어`)에 없습니다. 그런데 같은 파일의 `_delete_protected` 설명문은 `지켜야 한다 ↔ 지켜야한다`를 "둘 다 맞는 예"로 들고 있습니다. **코드 주석 두 곳이 서로 어긋납니다.** 동작은 앞의 규칙(띄우라)입니다. 규범상 보조사 `야`가 낀 `-어야`는 허용이 아니라는 쪽이 맞지만, 이 불일치는 문서 정리 후보입니다.

10.7 ④ 추가 규칙 (`spacing_extra.py`)

2026-09-14 탐침에서 **로컬 엔진이 아무 표시도 못 한** 부류들을 메우려고 만든 파일입니다.

만들어진 이유

> Kiwi는 대부분의 정답 띄어쓰기를 이미 알고 있었다(`kiwi.space`). 그런데 `spacing.py`가 **삭제만** 쓰고 삽입은 태거 0.90 문턱에 맡겨 버려지고 있었다. "Kiwi 삽입 ∧ 태거"를 그대로 켜면 위키 정문에서 p≥0.9로도 **395건**이 발동했다(대부분 허용 합성어 `총괄 프로듀서`). 쓸 수 없었다. 그래서 **부류별 규칙**이다.

즉 Kiwi의 제안을 통째로 믿을 수 없어서 **"어떤 부류인지 명시한 뒤에만"** 믿습니다.

규칙 목록 (`_RULES`)

이름	하는 일	예 (실행 결과)
punct_space	문장부호 뒤 다음 문장이 붙음	`재밌다.다음에` → `다. 다음에`
punct_cling	부호 앞의 공백	`많다 .` → `많다.`
compound_predicate	명사 + 용언 합성(재미없다 등)	`재미 없다` → `재미없다`
question_mark	의문형 어미 뒤 마침표	(6장)
compound_time	지난주·그동안·매년 / 이번 주·다음 달	
prefix_space	`제-` 는 숫자에 붙임	
numeral_spacing	수 + 단위 (일 년, 열 시)	
copula_space	서술격 조사 `이다` 는 앞말에 붙임	
ending_space	`-ㄹ수록` , `-는지`	
pronoun_josa	대명사 뒤 `마저·뿐`	
noun_verb	명사 + 용언 연어(짜증 나다)	
josa_boundary	격조사 뒤에 용언이 붙음	`기초로하면서`
runon_boundary	여러 단어를 붙여 쓴 어절의 조사·어미 뒤	아래
bound_noun_extra	관형형+적·채·뒤·한·일, 수+단위	
ending_join	`-자마자` · `-ㄹ뿐더러`	
compound_verb	사전 합성용언(알아내다)	`알아 내다` → `알아내다`
negation_context	`-면 안 된다` , `잘되든 안 되든`	
den_choice	선택의 `-든 말든`	
etm_noun	관형형 + 명사	`하는영화` → `하는 영화`
adverb_space	부사 + 뒷말	
etn_verb	`-기` + 용언	`쓰기 위해`
derived_verb_join	파생 동사 붙임	
noun_light_verb	명사 + 경동사	`영화 보다`

`runon_boundary`의 사연 (경계 모델 이전의 방식)

붙여 쓴 어절의 **조사·어미 바로 뒤**를 띄우는 규칙입니다. 이 규칙은 8장의 경계 모델이 나오기 **전**의 방식이고, 지금도 보조로 돕니다.

조건이 많은 이유는 오탐 사례가 그대로 조건이 되었기 때문입니다.

조건	막는 실제 오탐
한 음절 앞말은 **단어 첫머리일 때만** 믿는다	`면역글로|불린`, `피앙케|토하는` (외래어 조각)
관형형 + 명사: 동사·보조용언 뒤나 두 음절 이상 형용사 뒤만	`짠맛`, `큰키나무`, `흰얼굴`, `같은해` (합성어)
어절 첫머리 관형형 + 한 음절 명사 제외	`마른내로` (도로명)
어미 뒤 `서` 제외	`옮기므로|서` (`서다`를 잘못 읽음)
굳은 부사(`_LEXICAL_ADVERBS`: 잘·못·안·늘·함께·같이·가득…) 제외	합성어를 만드는 부사

한계도 명시돼 있습니다. 8장의 서문대로 이 규칙은 **실사용 오류 36개 경계 놓침 중 4개만** 회복했습니다.

10.8 실행 결과 정리 (제가 직접 돌림)

입력	결과	출처
너무많아서 놀랐다	`많` → `많`	degree_adverb_rule (high)
그렇게 하면 안되요	`안` → `안`	negation_context_rule (high)
하고있다	`있` → `있`	aux_verb_rule (high)
도와드리다	(없음)	47항 허용
안녕 하세요	공백 삭제	suffix_space_rule (high)
세번째 갔다	`번` → `번`	numeral_unit_rule (high)
재밌다.다음에 보자	`다` → `다`	punct_space_rule (high)
친구 와 갔다	공백 삭제	josa_space_rule (high)
하는영화를 봤다	`영` → `영`	etm_noun_rule (high)
알아 내다	공백 삭제	compound_verb_rule (high)
이 영화 재미 없다	`재미 없다` → `재미없다`	compound_space_rule (high)
핽교가는길에 갔다	`길` → `길`	tagger_space (low)
지난주에 갔다 / 영화 보다 / 짜증난다 / 점심때 먹었다 / 총괄 프로듀서	(없음)	정문

`총괄 프로듀서`, `점심때`가 **조용한 것**이 설계의 성공입니다. 둘 다 위키에서 태거가 틀리게 띄우고 싶어 하던 허용 합성어입니다.

10.9 이 장에서 가져갈 것

1. 띄어쓰기는 **허용이 많은 영역**이라 "필수 / 권장 / 허용" 3단계로 나누고 **침묵이 기본값**이다.
2. 자동 교정(high)은 **규칙이 정한 자리**에서만. 모델은 점선까지만.
3. 규칙 하나하나가 **위키 정문에서 발동한 것을 전수로 읽고 좁힌 결과**다. 조건이 많은 것은 오탐 사례의 흔적이다.
4. 같은 코드가 두 곳에서 어긋날 수 있다(`지켜야한다` 설명 불일치). 문서도 코드와 같이 검증해야 한다.
5. 층이 환경에 따라 켜지고 꺼진다. 결과가 다르면 `/api/health`부터 본다.

10.10 확인 문제

1. 태거가 `총괄프로듀서`를 띄우라고 해도 자동 교정이 안 되는 이유는? (모델 단독으로는 low까지만, `MODEL_NEVER_AUTOFIXES`)
2. `못하다`를 `못 하다`로 안 고치는 이유는? (위키 13건이 전부 정문, 굳어진 말)
3. 규칙 결과와 태거 결과가 같은 자리면 누가 남나? (규칙이 차지한 `(start,end)`는 태거 후보에서 제외)
4. 태거가 torch 없는 배포본에서 빠졌는지 확인하려면? (`/api/health`의 `spacing_tagger`)

11장. 평가: 좋은지 어떻게 아는가

코드가 잘 돈다고 해서 엔진이 **좋은** 것은 아닙니다. "좋다"를 숫자로 재는 장치가 따로 필요합니다. 이 프로젝트는 코드의 상당 부분(`backend/eval/`, `backend/tests/`)이 이 장치입니다. 엔진 본체보다 평가 코드가 더 많습니다.

이 장의 모든 수치는 제가 **이 저장소를 받아 직접 돌려서** 얻은 값입니다(`python -m eval.pipeline`, 약 5분).

11.1 먼저 용어

용어	뜻	쉬운 말
정밀도(precision)	엔진이 "고쳐라"고 한 것 중 **실제로 맞게 고친** 비율	말한 것의 정확도
재현율(recall)	실제 오류 중 엔진이 **잡아낸** 비율	놓치지 않는 정도
오탐(FP, false positive)	맞는 글을 틀렸다고 함	헛짚음
미탐(FN)	틀린 글을 못 잡음	놓침
오탐률(FPR)	정문 중 엔진이 건드린 비율	
과교정	정문을 **틀리게 바꿔** 버림	오탐의 가장 나쁜 형태
정문	맞게 쓴 문장	
코퍼스	문장을 대량으로 모은 자료	

왜 두 숫자가 모두 필요한가

"전부 틀렸다고 하는" 엔진은 재현율이 100%지만 쓸 수 없습니다. "아무것도 안 건드리는" 엔진은 오탐이 0이지만 쓸모가 없습니다. 다듬은 **오탐을 훨씬 무겁게** 봅니다(1장 원칙). 그래서 하한을 오탐 쪽에 가장 엄격하게 겁니다.

11.2 평가 데이터 네 종류

데이터	규모	만드는 법	무엇을 재나
`clean.jsonl`	위키 정문 9,765	위키백과 문장	오탐·과교정
`errors.jsonl`	합성 오류 3,871	정문에 규칙으로 오류를 주입	재현율(카테고리별)
`gold.jsonl`	225	국립국어원 규범 예문	규범과의 일치
`casual_*.jsonl`	구어 정문 3,000 / 오류 1,073	nsmc 영화평	구어 도메인
`real_errors/`	dev 300 · holdout 199 · holdout2 150 문장	**사람이 실제로 쓴 문장**을 규범 기준으로 주석	**실사용 재현율**

합성 오류의 함정: 순환

합성 오류는 "규칙이 오류를 만들어 넣고, 규칙이 그것을 잡는" 구조입니다. 점수가 높게 나오지만 **규칙이 모르는 오류가 얼마나 있는지는 원리상 못 잽니다.**

> 2026-09-13 nsmc 구어 300문장을 규범 기준으로 직접 주석해 보니, 합성셋 로컬 재현율은 **93.8%**인데 실사용 오류의 자동교정 재현율은 약 **15%**였다. 놓친 것의 대부분은 소수 부류(의존명사 거/걸/건, 시간 명사 때·전·중, 부정 부사 안/못 …)에 몰려 있었다.

이 발견이 프로젝트의 방향을 바꿨습니다. "합성 점수를 올린다"가 아니라 **"실사용 오류를 잡는다"**가 기준이 됐습니다(13장).

실사용 평가셋의 구조 (`eval/real_errors/`)

파일 두 개가 한 세트입니다.

```
holdout_sentences.txt   한 줄에 한 문장
holdout_gold.txt        문장번호|||틀린부분|||고친부분|||부류      (한 행 = 편집 하나)

실제 예:
1|||전개 마저|||전개마저|||DEL
2|||재미 없었고|||재미없었고|||DEL
2|||별로 였다|||별로였다|||DEL
```

부류 표기: DN(의존명사), DET(관형사), EXV(있다/없다 합성), DEL(붙여쓰기), TN(시간명사), NEG(부정 부사), JOSA(조사), M-어휘·M-활용·M-구어(맞춤법 계열) 등.

점수를 매기는 법

한 오류마다:

1. 문장에서 `틀린부분`을 찾고, `고친부분`으로 바꾼 **기대 문장**을 만듭니다.
2. 엔진을 돌려 그 자리와 겹치고, **적용했을 때 기대 문장이 되는** 지적을 찾습니다(`hit`).
3. 그 지적이 **자동 교정 등급**(low가 아니고 제안이 있음)이면 `auto` 성공, 점선이라도 있으면 `any` 성공.

```
hit  = [i for i in issues[sid] if i.start < w1 and i.end > w0 and _apply(t
        ext, i) == expected]
auto = any(_actionable(i) for i in hit)
```

여기서 `_actionable`의 사연이 있습니다.

```
# 제안이 **빈 문자열**인 것은 공백을 지우는 교정이다(붙여 쓰기). 이걸 bool(s) 로 걸러 내는
        바람에
# 붙여쓰기 교정이 전부 '저신뢰만' 으로 찍혀 DEL 부류가 실제보다 낮게 나왔다(29% vs 실제 7
        9%).
if iss.confidence == "low" or not iss.suggestions: return False
return iss.suggestions[0] != iss.original
```

빈 문자열 `""`은 "공백 삭제"라는 정당한 교정인데, 파이썬에서 빈 문자열은 거짓(False)처럼 취급됩니다. `if suggestion:`처럼 쓰면 "제안 없음"과 구분이 안 됩니다. 이 사소한 차이로 한 부류의 점수가 29%로 찍혔고 실제는 79%였습니다. **교훈:** 측정 도구의 버그는 엔진의 버그보다 오래 숨습니다.

11.3 실제 결과 (제가 돌린 값)

실사용 오류 재현율

평가셋	오류 수	자동 교정	점선 포함
dev (300문장)	356	**265 (74%)**	287 (81%)
holdout (199문장)	204	**146 (72%)**	153 (75%)
holdout2 (150문장)	178	**120 (67%)**	129 (72%)

주의: README는 holdout2를 **63%**로 적고 있습니다(2026-09-17 측정). 제 환경에서 지금 돌린 값은 **67%**입니다. 4%p 차이의 원인은 확인하지 않았습니다(환경에서 켜진 층, README 이후의 변경 등이 후보). 다음 절의 "층이 켜졌는가"가 그 후보 중 하나입니다.

세 평가셋을 따로 두는 이유

> holdout은 규칙을 만들기 **전에** 주석하고 봉인했지만 그 뒤로 여러 번 들여다봤습니다. holdout2는 **처음 보는** 문장입니다. 세 수의 차이가 곧 **과적합의 크기**이고, 앞으로의 개선은 holdout2에서 주장합니다.

과적합은 "내가 본 문제에는 강하지만 새 문제에는 약한" 상태입니다. 시험 문제를 알고 공부하면 그 시험 성적이 좋아지는 것과 같습니다. dev(74%)는 규칙을 만들 때 계속 보는 셋이라 가장 높고, 처음 보는 문장인 holdout2(67%)가 실제 실력에 가장 가깝습니다.

부류별로 보면 (dev 기준 일부)

부류	오류	자동교정	비고
DN 의존명사	71	94%	가장 많이 잡음
TN 시간명사	24	88%	
EXV 있다/없다	37	73%	
OTH 기타	29	**34%**	가장 약함
M-구어	8	38%	구어 표기
JOSA 조사	28	57% (점선 포함 86%)	조사는 점선으로 많이 잡힘

오탐 후보 목록

평가는 "정답과 안 맞는 자동교정"도 출력합니다. dev에서 15건이었습니다. 몇 가지 예:

```
#41  '나' → ' 나'    [noun_verb_space_rule]
#132 '써져' → '쓰여' [dict_rule]
#224 '않' → '안'     [ml_confusion]
#276 '다르게 보이네요' → '다르게 보인다' [consistency_rule]
```

이것은 **오탐이 확정된 목록이 아닙니다.** "주석에 없는 교정"이라서, 진짜 오탐일 수도 있고 **주석자가 놓친 진짜 오류**일 수도 있습니다. 사람이 하나씩 읽어 갈라야 합니다.

11.4 품질 파이프라인: `python -m eval.pipeline`

모든 변경이 이 한 명령을 통과해야 합니다. **순서가 설계**입니다.

```
[layers]         어느 층이 켜졌나
[per-rule fpr]   어느 규칙이 정문을 건드리나
[coverage]       그 구문을 재고는 있나
단위테스트        1,020개
모델·사전 대조    배포본이 원본과 같은 값을 내나
QA 게이트          불변식 · 회귀 · 코퍼스 스캔 · 퍼즈
메트릭 하한       문어 6개 · 구어 1개
```

실제로 돌렸을 때의 요약입니다.

```
✓ unit-tests       1020 passed in 37.6s
✓ model-export     .ddm이 sklearn 원본과 일치
✓ lexicon          어절 1,806,050개 · 21.7MB
✓ qa-gates
✓ metric-floors
✓ casual-floors
파이프라인 통과 (308.9s)
```

단계 ① `[layers]`: 어느 층이 켜졌나를 먼저 찍는다

```
ok  띄어쓰기 태거(BiGRU)
ok  C 경로(이상탐지)
ok  1층 규칙 · 2층 혼동쌍 · D 문체
```

이유: 모델 파일이나 의존성이 없으면 그 층은 조용히 빠집니다(앱이 죽지 않게 하는 설계). 그러면 지표는 **"다른 엔진"을 잰 값**이 됩니다. 실제로 CI가 torch 없이 돌면서 태거 없는 파이프라인을 재고 있던 일이 있었습니다(2026-08-30). 그 뒤로는 실행마다 어느 층이 켜졌는지 먼저 찍습니다.

단계 ② `[per-rule fpr]` : 규칙별 오탐률

전체 오탐률은 규칙 수백 개를 한 숫자로 뭉갭니다. 그래서 **규칙마다 자기 표본에서만** 따로 잽니다.

```
전체 fpr           9 / 10,000 = 0.0009
confusion:den      4 /     30 = 0.133     ← 같은 엔진, 148배
```

희석이 사라지면 값이 전혀 다르게 보입니다. 실제 결과의 일부:

```
ok  checker:compound_space_rule     3/30 = 0.100
ok  checker:yul_ryul_rule           2/30 = 0.067
ok  confusion:doe                   0/30 = 0.000  (상한 <= 0.050)
ok  checker:boundary_model          0/30 = 0.000  (상한 <= 0.050)
```

위 두 줄(0.100, 0.067)은 숫자만 보면 높아 보입니다. 이 규칙들은 **상한이 등록되지 않아** 값을 보고만 하고 게이트하지 않으므로 `ok`로 찍힙니다. 실제 출력은 발동한 문장을 같이 보여 주는데(`종류에 관계 없이…`처럼 원문이 `관계없이`를 띄어 쓴 경우가 보입니다), **세 건이 모두 원문의 진짜 오류인지는 제가 하나씩 읽어 확인하지 않았습니다.** 이런 줄을 사람이 읽어 갈라야 한다는 것이 이 단계의 요점입니다.

표본이 부족하면 숫자를 내지 않는다

```
insufficient_sample 36개 (표본 < 20) — 수치를 내지 않는다
0% 가 아니라 '못 쟀다' 이다.
```

0%와 "못 쟀다"는 전혀 다른 말인데, 숫자로 뭉개면 후자가 전자로 읽힙니다. 그래서 표본이 20건 미만이면 숫자를 내지 않고 "사각지대"라고 적습니다.

단계 ③ `[coverage]` : 그 구문을 재고는 있나

```
ok  되/돼                1399건  (하한 >= 1300)
ok  안/않                 378건  (하한 >= 180)
ok  연결어미 -되           2건  ← 코퍼스로는 못 잼(테스트가 지킴)
ok  율/률                 23건  ← 코퍼스로는 못 잼(테스트가 지킴)
```

이 게이트가 없어서 실제로 버그를 놓친 적이 있습니다. 연결어미 `-되`(적되·하되)를 `돼`로 고치는 과교정이 medium으로 나가고 있었는데, 정문 10,000문장에 그 구문이 **1건**뿐이라 아무리 잘못 고쳐도 오탐률이 움직이지 않았습니다. 지표를 믿기 전에 **"그 구문의 정문 표본이 충분히 있는가"**를 먼저 봅니다. 없는 구문은 "코퍼스로는 못 잼, 테스트가 지킴"이라고 정직하게 적고, 대신 표적 단위 테스트를 연결합니다.

단계 ④ 단위 테스트 1,020개

테스트 파일이 검사기마다 하나씩 있습니다(`test_josa.py`, `test_tense.py`, `test_confusion.py` …). 구조는 모두 같습니다.

```python
def test_...():
    issues = check_xxx("틀린 문장", kiwi)
    assert issues[0].suggestions == ["고친 말"]

def test_..._is_never_flagged():
    assert check_xxx("맞는 문장", kiwi) == []      # 정문을 건드리지 않는다
```

"정문을 건드리지 않는다"는 테스트가 "잡는다"는 테스트만큼 많습니다.

단계 ⑤ 모델·사전 대조

```
model-export: .ddm이 sklearn 원본과 같은 확률을 내는가 (최대 오차 1e-12 이내)
lexicon:      어절 사전이 원본과 같은 횟수를 내는가
```

"배포본(`.ddm`)"과 "학습본(`.joblib`)"이 어긋나면 `meta.json`의 문턱이 **지금 돌지 않는 모델**을 설명하게 되므로 매번 대조합니다.

단계 ⑥ QA 게이트 (`eval/qa/`)

예시 한 개가 아니라 **불변식**을 겁니다. "어떤 입력이어도 이것만은 참이어야 한다"는 규칙입니다.

불변식	내용	실행 결과
P1 무악화	자동 교정을 적용하면 형태소 분석이 나빠지지 않는다(모르는 형태소가 늘지 않음)	위반 0
P2 멱등	고치고 다시 검사해 또 고쳐도 더 이상 안 바뀐다	위반 0
P3 정문 침묵	정문 코퍼스에서 high가 거의 0	오탐 0건
P4 편집 최소성	교정이 원문과 너무 멀리 가지 않는다(`면→수면` 같은 도약 금지)	위반 0
P5 문맥 불변	문장 하나에서 찾은 교정은 그 문장이 문서 속에 있어도 똑같이 찾아진다	(코드 수준 불변식)

그 밖에:

```
[회귀 스위트] 등록 22건, 실패 0건
[코퍼스 스캔] 3000문장, high-conf 오탐 0건
[퍼즈]        500문장 변형 ...
```

- **회귀 레지스트리(`regressions.jsonl`, 22건):** 과거에 틀리게 고쳤던 문장을 영구 등록해 "다시는 안 틀린다"를 게이트로 둡니다. 형식은 `{"sentence": ..., "mode": "absent", "original": "든", "suggestion": "던", "note": "foreign proper-noun mis-parse (세든->세던)"}`. `absent`는 "이 교정이 나오면 안 된다"입니다.
- **퍼즈(fuzz):** 정문에 무작위로 오류를 주입해(`add_space`, `delete_char`, `keyboard` …) 검출률을 봅니다.

변형	검출률	의미
an_anh	100%	안/않 바꾸기는 다 잡음
doe_dwae	94%	
josa_swap	58%	
add_space	35%	
delete_char	4.6%	글자 하나 빠짐은 거의 못 잡음
dup_char	3.2%	글자 중복은 거의 못 잡음
keyboard	7.2%	자판 오타

낮은 수치를 숨기지 않습니다. 이 표가 엔진의 약점 지도입니다. `delete_char`·`dup_char`가 낮은 것은 9장 끝의 "음절 하나를 빼려면 단어를 알아야 한다"는 한계와 같은 이야기입니다.

단계 ⑦ 메트릭 하한(floors)

```
ok  precision            0.9921  (floor >= 0.97)
ok  fpr                  0.0000  (floor <= 0.002)
ok  over_correction_rate 0.0000  (floor <= 0.002)
ok  l1_recall            0.9791  (floor >= 0.97)
ok  l2_recall            0.9924  (floor >= 0.99)
ok  roseo_recall         0.8503  (floor >= 0.82)
```

지표	하한	실측	뜻
precision	≥ 0.97	0.9921	고친 것 중 맞은 비율
fpr	≤ 0.002	0.0000	정문을 건드린 비율
over_correction_rate	≤ 0.002	0.0000	정문을 **틀리게 바꾼** 비율
l1_recall	≥ 0.97	0.9791	규칙 층이 잡는 비율
l2_recall	≥ 0.99	0.9924	혼동쌍 모델 층
roseo_recall	≥ 0.82	0.8503	로서/로써

하한은 후퇴 금지선입니다. 코드 주석의 뜻(의역): 엔진이 좋아지면 올리고, 변경을 통과시키려고 **내리지 마라. 하한은 품질 목표가 아닙니다.** "여기서 더 나빠지면 안 된다"일 뿐 "여기까지면 충분하다"가 아닙니다.

하한 하나를 폐기한 일

처음엔 `l3_recall`(계층 가중평균)로 걸었다가 걷어냈습니다. 성격이 전혀 다른 두 집단(합성 roseo 0.83 vs gold의 문맥·시제 0.013)을 섞기 때문에 **성능이 그대로여도 구성비만 바뀌면 값이 움직였기** 때문입니다(0.6768 → 0.5850, 그런데 같은 엔진의 roseo 재현율은 0.8205 → 0.8305로 오히려 올랐음). **구성비에 흔들리는 하한은 회귀 방지선 노릇을 못 합니다.** 카테고리 단위로 바꿨습니다.

단계 ⑧ 구어 도메인은 따로 잰다

```
casual 정문 2,936 · 오류 1,073
  precision      0.3096
  recall         0.9515
  casual_fpr     0.3835
  raw_touch_rate 0.3967   (<= 0.40)
```

구어 정밀도·오탐률은 하한이 걸려 있지 않습니다. 이 값이 재는 것은 엔진이 아니라 **코퍼스**이기 때문입니다.

> 띄어쓰기 규칙이 들어오기 전 구어 과교정률은 1.84%였는데 규칙이 들어오자 31%로 뛰었다. 발동한 것을 무작위로 뽑아 읽으니 **20건 중 19건이 원문의 진짜 띄어쓰기 오류**였다(웃긴부분은 , 높은거지 , 남편때문에). nsmc의 "정문"은 오류를 주입하지 않았다는 뜻일 뿐, 띄어쓰기를 지켰다는 뜻이 아니다.

그래서 구어에서 게이트로 거는 값은 **사람이 실제로 보는 지적 빈도**(`raw_touch_rate` ≤ 0.40, 측정 0.3967)입니다. "두 문장 반에 하나꼴로 밑줄이 뜬다"는 뜻이고, 그것을 낮출지 말지는 **측정이 아니라 제품 결정**이라고 문서가 못 박고 있습니다.

11.5 이 장의 교훈 정리

교훈	근거 사건
측정이 정확해질수록 지표가 **나빠** 보일 수 있다	빈 제안(공백 삭제)을 자동교정으로 세지 않던 버그 수정 후 구어 지적률 .3543 → .3860
평균 하나는 결함을 희석한다	전체 fpr 0.0009 vs `den` 0.133
"0%"와 "못 쟀다"는 다르다	표본 20건 미만이면 수치를 내지 않음
재고가 없으면 지표를 믿지 말라	연결어미 `-되` 표본 1건
규칙이 만든 시험은 규칙을 칭찬한다	합성 93.8% vs 실사용 15%
시험을 본 사람은 점수가 부풀려진다	dev 74% → holdout 72% → holdout2 67%
측정 도구도 코드다	빈 문자열 버그 29% vs 79%
같은 것을 두 길로 재면 서로 대조하라	`style` 카테고리 500 사건

11.6 직접 돌려 보기

```
cd dadeum/backend
source .venv/bin/activate

python -m eval.pipeline --fast              # 단위테스트만 (약 40초)
python -m eval.pipeline                      # 전체 (약 5분)
python -m eval.real_errors.score holdout2    # 실사용 재현율
python -m eval.real_errors.score dev --misses DN    # 한 부류의 놓친 것만 보기
python -m eval.qa.run_qa                     # QA 불변식만
```

11.7 확인 문제

1. 정밀도 99%, 재현율 50%인 엔진은 어떤 엔진인가? (말한 것은 거의 맞지만 오류의 절반을 놓침. 다듬이 선호하는 쪽에 가까움)
2. dev 점수가 holdout2보다 높게 나오는 이유는? (규칙을 만들 때 계속 보는 셋이라 과적합됨)
3. 표본이 5건뿐인 규칙의 오탐률을 "0%"라고 쓰지 않는 이유는? (0%와 "못 잤다"를 구별하기 위해)
4. 하한을 "통과시키려고 내리면" 안 되는 이유는? (후퇴 금지선이 침식돼 서서히 나빠짐을 못 막음)